COBOL

CHEAT SHEETS

Enterprise COBOL for z/OS, with GnuCOBOL for practice anywhere

Divisions · Fixed Format · PICTURE · USAGE · COMP-3 · Level 88 · REDEFINES · MOVE · STRING · INSPECT · COMPUTE · EVALUATE · PERFORM · OCCURS · SEARCH ALL · FILE STATUS · VSAM · SORT · Control Breaks · Copybooks · CALL · DB2 · CICS · Abends · JSON

64 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & History
02 Installing GnuCOBOL
03 The Mainframe Dev Environment
04 Program Structure — Divisions
05 Coding Format

OUTPUT, INPUT & COMMENTS

06 DISPLAY
07 ACCEPT
08 Comments & Documentation

DATA DIVISION BASICS

09 DATA DIVISION Sections
10 Level Numbers & Group Items
11 PICTURE — Numeric
12 PICTURE — Alphanumeric & Edited
13 USAGE Clauses
14 VALUE & Figurative Constants
15 Condition Names (Level 88)
16 REDEFINES & RENAMES

DATA MANIPULATION

17 MOVE
18 INITIALIZE & SET
19 Editing Fields for Output
20 Data Conversion

ARITHMETIC

25 ADD & SUBTRACT
26 MULTIPLY & DIVIDE
27 COMPUTE
28 Intrinsic Math Functions

CONTROL FLOW

29 IF / ELSE
30 Conditions
31 EVALUATE
32 PERFORM
33 PERFORM Loops
34 GO TO, EXIT & Ending a Program

TABLES

35 OCCURS Basics
36 Indexes
37 SEARCH & SEARCH ALL
38 Variable-Length Tables & Loading

SEQUENTIAL FILES

39 File Concepts
40 SELECT & ASSIGN
41 FD & Record Definitions
42 OPEN, READ, WRITE & CLOSE
43 FILE STATUS Codes

INDEXED & RELATIVE FILES

44 Indexed Files (VSAM KSDS)
45 Random & Dynamic Access
46 Relative Files

SORTING & MERGING

47 The SORT Verb
48 MERGE & Sort Utilities

REPORTS & BATCH PATTERNS

49 Report Layout
50 Control Break Processing
51 Master / Transaction File Update
52 Data Validation & Error Handling

SUBPROGRAMS & COPYBOOKS

53 COPY & Copybooks
54 CALL & Subprograms
55 LINKAGE SECTION & Parameters
56 Nested Programs & User-Defined Functions

ENTERPRISE COBOL

57 Embedded SQL with DB2
58 CICS Basics
59 Compiler Options & Listings
60 Debugging & Abends
61 JSON, XML & Interop

CBL

Table of Contents

64 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 05

- 01 Introduction & History
- 02 Installing GnuCOBOL
- 03 The Mainframe Dev En…
- 04 Program Structure — D…
- 05 Coding Format

OUTPUT, INPUT & COMMENTS 06 – 08

- 06 DISPLAY
- 07 ACCEPT
- 08 Comments & Docume…

DATA DIVISION BASICS 09 – 16

- 09 DATA DIVISION Sections
- 10 Level Numbers & Grou…
- 11 PICTURE — Numeric
- 12 PICTURE — Alphanume…
- 13 USAGE Clauses
- 14 VALUE & Figurative Co…
- 15 Condition Names (Lev…
- 16 REDEFINES & RENAMES

DATA MANIPULATION 17 – 20

- 17 MOVE
- 18 INITIALIZE & SET
- 19 Editing Fields for Outp…
- 20 Data Conversion

STRINGS 21 – 24

- 21 STRING
- 22 UNSTRING
- 23 INSPECT
- 24 Reference Modificatio…

ARITHMETIC 25 – 28

- 25 ADD & SUBTRACT
- 26 MULTIPLY & DIVIDE
- 27 COMPUTE
- 28 Intrinsic Math Functions

CONTROL FLOW 29 – 34

- 29 IF / ELSE
- 30 Conditions
- 31 EVALUATE
- 32 PERFORM
- 33 PERFORM Loops
- 34 GO TO, EXIT & Ending a…

TABLES 35 – 38

- 35 OCCURS Basics
- 36 Indexes
- 37 SEARCH & SEARCH ALL
- 38 Variable-Length Table…

SEQUENTIAL FILES 39 – 43

- 39 File Concepts
- 40 SELECT & ASSIGN
- 41 FD & Record Definitions
- 42 OPEN, READ, WRITE & …
- 43 FILE STATUS Codes

INDEXED & RELATIVE FILES 44 – 46

- 44 Indexed Files (VSAM K…
- 45 Random & Dynamic Ac…
- 46 Relative Files

SORTING & MERGING 47 – 48

- 47 The SORT Verb
- 48 MERGE & Sort Utilities

REPORTS & BATCH PATTERNS 49 – 52

- 49 Report Layout
- 50 Control Break Processi…
- 51 Master / Transaction Fi…
- 52 Data Validation & Error…

SUBPROGRAMS & COPYBOOKS 53 – 56

- 53 COPY & Copybooks
- 54 CALL & Subprograms
- 55 LINKAGE SECTION & P…
- 56 Nested Programs & Us…

ENTERPRISE COBOL 57 – 62

- 57 Embedded SQL with D…
- 58 CICS Basics
- 59 Compiler Options & Li…
- 60 Debugging & Abends
- 61 JSON, XML & Interop
- 62 Testing & DevOps for …

QUICK REFERENCE 63 – 64

- 63 Quick Reference — Ver…
- 64 Quick Reference — PIC…

Introduction & History

COMMON BUSINESS-ORIENTED LANGUAGE

COBOL was designed in **1959** by the CODASYL committee, drawing on Grace Hopper's FLOW-MATIC, so business programs could be written in English-like statements and run on any vendor's machine. It still runs core systems at **banks, insurers, airlines and government agencies**, mostly as overnight **batch** jobs and **CICS** transactions on IBM Z mainframes.

KEY POINTS

- Source files: `.cbl`, `.cob`, `.cpy` (copybooks)
- Programs are split into four **divisions**
- Strong at fixed-format records and exact **decimal** math
- IBM Enterprise COBOL 6.x on z/OS; GnuCOBOL 3.x for local practice

HELLO, WORLD

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. HELLO.
      * The smallest useful COBOL program
       PROCEDURE DIVISION.
           DISPLAY "Hello, World".
           STOP RUN.

      *> GnuCOBOL:  cobc -x hello.cbl
      *>            ./hello
      *> Output:    Hello, World
```

CODE EXPLAINED

- → `IDENTIFICATION DIVISION` and `PROGRAM-ID` are the only required lines
- → Code starts in column 8; column 7 holds `*` for comments
- → `DISPLAY` writes a line to standard output (SYSOUT on z/OS)
- → `STOP RUN` ends the whole run unit; subprograms use `GOBACK`

MODERN EQUIVALENT

- ↔ **Python** `print("Hello, World")`
- ↔ **Java** `System.out.println("Hello, World");` inside `main`
- ↔ **JavaScript** `console.log("Hello, World");`

PREFER / AVOID

- ⇄ "COBOL is dead" → about 800 billion lines are estimated in production
- ⇄ learning old COBOL-74 style → COBOL-85 scope terminators (`END-IF`)

TIMELINE

YEAR	MILESTONE
1959	CODASYL designs COBOL
1968	First ANSI standard
1985	COBOL-85: `END-IF`, inline `PERFORM`
2002	OO, free format, `*>` comments
2014	`ROUNDED MODE`, table `SORT`
2023	Current ISO/IEC 1989 standard

WHERE IT RUNS

PLATFORM	COMPILER
`IBM z/OS`	Enterprise COBOL 6.x
`IBM i`	ILE COBOL
`Linux / macOS / Windows`	GnuCOBOL 3.x
`Linux / Windows`	Micro Focus (Rocket) Visual COBOL
`Cloud mainframe`	IBM Z Xplore, Wazi as a Service

⚠ COMMON MISTAKES

- Treating COBOL as a toy language: it handles exact decimal money math better than most languages
- Starting code in column 1 in fixed format (columns 1–7 are reserved)
- Using `STOP RUN` in a called subprogram, which ends the caller too

✓ TIPS & BEST PRACTICES

- Practice locally with GnuCOBOL, then move to IBM Z Xplore for real z/OS
- Read existing programs first: maintenance is most COBOL work
- Learn JCL alongside COBOL (see the JCL & z/OS set)

QUICK SUMMARY

- `DIVISION` — four-part program structure
- `DISPLAY` — print a line
- `STOP RUN` — end the program
- `cobc -x` — compile with GnuCOBOL

Installing GnuCOBOL

brew install gnucobol · apt install gnucobol · cobc -x · cobc -free · cobcrun · VS Code extensions

A FREE COBOL COMPILER

GnuCOBOL translates COBOL to C and compiles it with your system C compiler, so it runs on macOS, Linux and Windows. It supports most of COBOL-85, much of 2002/2014, and IBM dialect options, which makes it the easiest way to practice before touching a mainframe.

KEY POINTS

- `cobc` is the compiler; `cobcrun` runs modules
- `-x` builds an executable; without it you get a module
- `-std=ibm` mimics Enterprise COBOL behavior
- Error messages give file, line and severity

INSTALL, COMPILE & RUN

```
# macOS (Homebrew)
brew install gnucobol
# Debian / Ubuntu
sudo apt install gnucobol

cobc --version                  # cobc (GnuCOBOL) 3.2.0

cobc -x hello.cbl               # build ./hello
./hello                         # Hello, World

cobc -x -free hello.cbl         # free-format source
cobc -x -std=ibm payroll.cbl    # IBM dialect rules
cobc -x -I copy/ main.cbl       # copybook folder
cobc -m util.cbl                # module util.so / .dylib
cobcrun util                    # run a module
cobc -x -debug -Wall prog.cbl   # runtime checks + warnings
```

CODE EXPLAINED

→ Default source format is **fixed**; add `-free` for free format

→ `-I` adds a search path for `COPY` members

→ `-debug` turns on subscript and numeric checks at run time

→ `-Wall` shows warnings such as truncation risks

MODERN EQUIVALENT

↔ **Python** `brew install python` · `python3 app.py`

↔ **Java** `brew install openjdk` · `javac App.java && java App`

↔ **JavaScript** `brew install node` · `node app.js`

PREFER / AVOID

⇄ compiling with no flags → `-Wall -debug` while learning

⇄ tabs in fixed-format source → spaces (tabs shift columns)

COBC OPTIONS

FLAG	DOES
`-x`	Build executable
`-m`	Build loadable module
`-free / -fixed`	Source format
`-std=ibm / mf / cobol2014`	Dialect
`-I dir`	Copybook path
`-o name`	Output name
`-debug`	Runtime checks
`-ftrace / -ftraceall`	Trace paragraphs / statements

EDITORS

TOOL	NOTES
`VS Code + COBOL (bitlang)`	Syntax, columns
`IBM Z Open Editor`	Enterprise COBOL language server
`Zowe Explorer`	Mainframe datasets & jobs
`OpenCobolIDE`	Small GnuCOBOL IDE

⚠ COMMON MISTAKES

- Forgetting `-x` and wondering where the executable is
- Saving source with tabs, which pushes code past column 72
- Expecting IBM-only syntax to compile without `-std=ibm`

✓ TIPS & BEST PRACTICES

- Set editor rulers at columns 7, 8, 12 and 72
- Use `-ftrace` to print each paragraph as it runs
- Keep copybooks in a `copy/` folder and pass `-I copy`

QUICK SUMMARY

- `cobc -x` — compile to an executable
- `-free` — free-format source
- `-std=ibm` — IBM dialect
- `-debug -Wall` — safer learning builds

Sheet 03

The Mainframe Dev Environment

z/OS · TSO / ISPF · datasets & members · Zowe Explorer · IBM Z Open Editor · IBM Z Xplore

WHERE PRODUCTION COBOL LIVES

On **z/OS**, source code is stored as **members** of partitioned datasets (PDS), not files in folders. You edit with **ISPF** or VS Code with **Zowe Explorer**, compile and run by submitting **JCL** jobs, and read the output in **SDSF** or the Zowe job view.

KEY POINTS

- Typical libraries: `USER.COBOL` (source), `USER.COPYLIB`, `USER.LOADLIB`
- Dataset names are uppercase, dot-separated qualifiers
- Compile, link and run are each steps in a JCL job
- IBM Z Xplore gives free z/OS access for learning

COMPILE & RUN A MEMBER (JCL)

```
//HELLOJOB JOB (ACCT),'COMPILE HELLO',CLASS=A,
//         MSGCLASS=X,NOTIFY=&SYSUID
//* Compile, link and run USER.COBOL(HELLO)
//CLG      EXEC IGYWCLG
//COBOL.SYSIN  DD DSN=USER.COBOL(HELLO),DISP=SHR
//COBOL.SYSLIB DD DSN=USER.COPYLIB,DISP=SHR
//GO.SYSOUT    DD SYSOUT=*
```

CODE EXPLAINED

- → `IGYWCLG` is IBM's compile-link-go procedure
- → `COBOL.SYSIN` overrides the compile step's source input
- → `SYSLIB` is where `COPY` members are found
- → `DISPLAY` output lands in `GO.SYSOUT` in the job log

MODERN EQUIVALENT

- ↔ **Python** Python script run by `python job.py`; no compile step
- ↔ **Java** Maven build (`mvn package`) then `java -jar app.jar`
- ↔ **JavaScript** `npm run build && node dist/app.js` in a CI pipeline

PREFER / AVOID

- ⇄ editing in ISPF only → Zowe Explorer + Git for version control
- ⇄ copying JCL from coworkers blindly → read each DD (see the JCL set)

LOCAL VS MAINFRAME

TASK	GNUCOBOL / Z/OS
Edit	Any editor / ISPF, Zowe
Store	Files / PDS members
Compile	`cobc` / IGYCRCTL step
Link	automatic / binder (IEWL)
Run	`./prog` / EXEC PGM=
Output	terminal / SYSOUT in JES

ZOWE CLI

COMMAND	DOES
`zowe files ls ds "USER.*"`	List datasets
`zowe files dl am "USER.COBOL"`	Download members
`zowe files ul ftds f.cbl "USER.COBOL(F)"`	Upload member
`zowe jobs submit ds "USER.JCL(CLG)"`	Submit job
`zowe jobs view sfbi JOB01234 2`	View spool file

⚠ COMMON MISTAKES

- Lowercase dataset names in JCL (they must be uppercase)
- Forgetting `SYSLIB`, so every `COPY` fails to compile
- Looking for output on the terminal instead of the job's SYSOUT

✓ TIPS & BEST PRACTICES

- Sign up for IBM Z Xplore to practice real z/OS for free
- Keep a working JCL member per program for quick recompiles
- Use Zowe Explorer to edit members in VS Code

QUICK SUMMARY

- `PDS(member)` — where source lives
- `IGYWCLG` — compile, link, go
- `SYSOUT` — DISPLAY output
- Zowe — VS Code access to z/OS

Sheet 04

Program Structure — Divisions

IDENTIFICATION DIVISION · ENVIRONMENT DIVISION · DATA DIVISION · PROCEDURE DIVISION · sections & paragraphs · the period

FOUR DIVISIONS, ALWAYS IN ORDER

Every program has up to four **divisions**: IDENTIFICATION names it, ENVIRONMENT connects it to files, DATA declares every variable and record, and PROCEDURE holds the logic. The PROCEDURE DIVISION is organized into **paragraphs** (named blocks) that you call with PERFORM.

KEY POINTS

- Hierarchy: division → section → paragraph → sentence → statement
- A **sentence** ends with a period; a statement does not
- All variables are declared up front in the DATA DIVISION
- Paragraph names start in Area A (column 8)

A COMPLETE SKELETON

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. PAYCALC.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT EMP-FILE ASSIGN TO EMPIN.
       DATA DIVISION.
       FILE SECTION.
       FD  EMP-FILE.
       01  EMP-REC            PIC X(80).
       WORKING-STORAGE SECTION.
       01  WS-TOTAL           PIC 9(7)V99 VALUE 0.
       PROCEDURE DIVISION.
       0000-MAIN.
           PERFORM 1000-INIT
           PERFORM 9000-END
           STOP RUN.
       1000-INIT.
           DISPLAY "STARTING".
       9000-END.
           DISPLAY "TOTAL " WS-TOTAL.
```

CODE EXPLAINED

- → `0000-MAIN` drives the program by calling other paragraphs
- → Numbered paragraph names (1000-, 9000-) are a common shop standard
- → Only the last statement in a paragraph needs the period
- → `SELECT … ASSIGN TO EMPIN` links the file to DD name `EMPIN`

MODERN EQUIVALENT

- ↔ **Python** Module with imports, globals, then functions and `if __name__ == "__main__":`
- ↔ **Java** Class with fields, then methods, then `public static void main`
- ↔ **JavaScript** Module: `import`s, top-level `const`s, functions, then a `main()` call

PREFER / AVOID

- ⇄ periods after every statement → one period per paragraph
- ⇄ one giant paragraph → small numbered paragraphs called by `PERFORM`

DIVISIONS

DIVISION	HOLDS
IDENTIFICATION	PROGRAM-ID, docs
ENVIRONMENT	CONFIGURATION, INPUT-OUTPUT
DATA	Files, variables, parameters
PROCEDURE	Executable logic

BUILDING BLOCKS

UNIT	EXAMPLE
Section	FILE SECTION.
Paragraph	1000-INIT.
Sentence	statements ending in .
Statement	MOVE A TO B
Clause	PIC 9(5)

⚠ COMMON MISTAKES

- Putting divisions out of order, which is a compile error
- A stray period inside an `IF`, which ends the `IF` early
- Declaring variables in the PROCEDURE DIVISION (not allowed)

✓ TIPS & BEST PRACTICES

- Keep a skeleton template and start every program from it
- Number paragraphs so their order in the source is obvious
- Put one paragraph per task: read, validate, write

QUICK SUMMARY

- `IDENTIFICATION` — program name
- `ENVIRONMENT` — files
- `DATA` — all variables
- `PROCEDURE` — paragraphs of logic

Coding Format

columns 1–6 sequence · column 7 indicator · Area A (8–11) · Area B (12–72) · >>SOURCE FORMAT FREE · continuation lines

FIXED COLUMNS FROM THE PUNCH-CARD ERA

Classic COBOL is **fixed format**: every 80-column line is divided into areas. Division, section, paragraph and `01`/`77` level names start in **Area A**; statements and other levels go in **Area B**. Free format (2002+) drops the columns, but most production code is still fixed.

KEY POINTS

- Columns 73–80 are ignored (old card sequence numbers)
- Column 7: `*` comment, `-` continuation, `D` debug line, `/` page eject
- COBOL is case-insensitive; shops usually write uppercase
- Words are separated by spaces; hyphens join words: `WS-TOTAL`

UNDER THE HOOD

- Enterprise COBOL stores source as 80-byte **FB** records (`LRECL=80`)
- The compiler ignores columns 73–80, so text there silently disappears
- GnuCOBOL defaults to fixed; `-free` or `>>SOURCE FORMAT FREE` switches

COLUMN LAYOUT

```
      *A---B---+----2----+----3----+----4----+----5----+
       IDENTIFICATION DIVISION.
       PROGRAM-ID. COLS.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01  WS-MSG    PIC X(60) VALUE "THIS LITERAL IS LONG
      -        "ER THAN ONE LINE".
       PROCEDURE DIVISION.
       MAIN-PARA.
           DISPLAY WS-MSG
      D    DISPLAY "ONLY WITH DEBUGGING MODE"
           STOP RUN.

      *> Free format (2002+): no columns
      *> >>SOURCE FORMAT FREE
      *> display "anywhere on the line"
```

CODE EXPLAINED

- → Ruler: `*` is column 7, `A` marks column 8 (Area A), `B` marks column 12 (Area B)
- → Continuation: the open literal runs to column 72, trailing spaces included; the next line has `-` in column 7 and reopens the quote
- → `D` lines compile only with `WITH DEBUGGING MODE`
- → `MAIN-PARA.` sits in Area A, statements in Area B (col 12+)

MODERN EQUIVALENT

- ↔ **Python** Indentation is syntax; no column limits (PEP 8 suggests 79)
- ↔ **Java** Free format; braces and semicolons, any column
- ↔ **JavaScript** Free format; whitespace ignored, Prettier sets 80 columns

PREFER / AVOID

- ⇄ continuing long literals → shorter literals joined with `STRING`
- ⇄ mixed case → follow the shop standard (usually uppercase)

COLUMN AREAS

COLUMNS	USE
1–6	Sequence number (optional)
7	Indicator
8–11	Area A: divisions, paragraphs, 01, 77, FD
12–72	Area B: statements, levels 02–49
73–80	Identification (ignored)

COLUMN 7 INDICATORS

CHAR	MEANING
*	Comment line
/	Comment + page eject in listing
-	Continuation
D	Debugging line
(space)	Normal code

⚠ COMMON MISTAKES

- Code running past column 72 is silently cut off, causing odd errors
- Paragraph names starting in Area B (column 12) are treated as statements
- Using tabs, which editors and compilers count differently

✓ TIPS & BEST PRACTICES

- Turn on editor rulers at 7, 8, 12 and 72
- Use `*>` inline comments only if your compiler supports 2002 syntax
- Avoid literal continuation; split text into two literals

QUICK SUMMARY

- `8-11` — Area A for names
- `12-72` — Area B for code
- `*` in col 7 — comment
- `>>SOURCE FORMAT FREE` — free format

DISPLAY

WRITING LINES OF OUTPUT

DISPLAY writes one or more literals and data items as a single line. On z/OS the line goes to **SYSOUT** (the job log); under GnuCOBOL it goes to the terminal. It is meant for messages and debugging; reports go through file WRITE statements.

KEY POINTS

- Items are joined with no separator; add `" "` yourself
- Numeric `DISPLAY` fields print all digits, including leading zeros
- Signed fields show an overpunched last digit unless edited
- Each `DISPLAY` ends the line unless `WITH NO ADVANCING`

DISPLAY VARIATIONS

```
       WORKING-STORAGE SECTION.
       01  WS-NAME      PIC X(10)     VALUE "ADA".
       01  WS-QTY       PIC 9(5)      VALUE 42.
       01  WS-AMT       PIC S9(5)V99 VALUE -12.50.
       01  WS-AMT-ED    PIC -ZZ,ZZ9.99.
       PROCEDURE DIVISION.
           DISPLAY "HELLO"                 *> HELLO
           DISPLAY "NAME: " WS-NAME "|"    *> NAME: ADA       |
           DISPLAY WS-QTY                  *> 00042
           DISPLAY WS-AMT                  *> 0001250- (GnuCOBOL)
           MOVE WS-AMT TO WS-AMT-ED
           DISPLAY WS-AMT-ED               *>     -12.50
           DISPLAY "NO NEWLINE " WITH NO ADVANCING
           DISPLAY "SAME LINE"             *> NO NEWLINE SAME LINE
           DISPLAY "TO OPERATOR" UPON CONSOLE
           DISPLAY ALL "-"                 *> repeat "-" (1 char)
           STOP RUN.
```

CODE EXPLAINED

- → `PIC X(10)` pads with spaces, so `ADA` prints with trailing blanks
- → Move numbers to an **edited** field before displaying them
- → Signed `DISPLAY` output differs by compiler; never parse it
- → `UPON CONSOLE` sends to the z/OS operator console: avoid in batch

MODERN EQUIVALENT

- ↔ **Python** `print("NAME:", name)` adds spaces; `end=""` skips the newline
- ↔ **Java** `System.out.print(...)` vs `println(...)`
- ↔ **JavaScript** `console.log("NAME:", name)`; `process.stdout.write()` for no newline

PREFER / AVOID

- ⇄ displaying raw numeric fields → move to an edited `PIC` first
- ⇄ leaving debug `DISPLAY`s in production → `D` lines or a debug flag

COMMON FORMS

FORM	RESULT
`DISPLAY lit`	Text line
`DISPLAY a b c`	Concatenated, no spaces
`WITH NO ADVANCING`	No line break
`UPON SYSOUT`	Job log (default)
`UPON CONSOLE`	Operator console
`UPON SYSERR`	stderr (GnuCOBOL)

WHAT PRINTS

FIELD	SHOWS
`PIC 9(5) VALUE 42`	00042
`PIC X(5) VALUE "AB"`	AB + 3 spaces
`PIC ZZZZ9 VALUE 42`	42 with leading spaces
`PIC 9V99 VALUE 1.5`	150 (no point)
`COMP-3 field`	converted to digits

⚠ COMMON MISTAKES

- Expecting a decimal point from `PIC 9V99` (the V is implied, not printed)
- Displaying `COMP-3` data with a group move first, which prints garbage bytes
- Relying on `DISPLAY` for report output instead of a print file

✓ TIPS & BEST PRACTICES

- Wrap values in `"|"` delimiters while debugging to see padding
- Create edited fields like `PIC Z,ZZ9.99` for any number a person reads
- Use `UPON SYSERR` in GnuCOBOL for error messages

QUICK SUMMARY

- `DISPLAY a b` — one line, no separators
- `WITH NO ADVANCING` — stay on the line
- edited PIC — readable numbers
- `UPON CONSOLE` — operator, rarely used

ACCEPT

READING INPUT AND SYSTEM VALUES

ACCEPT reads a line of input into a data item, or copies a system value such as the date or time. In batch, plain ACCEPT reads from **SYSIN** (an in-stream card in the JCL); interactive programs on GnuCOBOL read from the keyboard.

KEY POINTS

- Input is moved into the field like an alphanumeric `MOVE`
- Each `ACCEPT` reads the next SYSIN record
- `FROM DATE` without `YYYYMMDD` gives a 2-digit year
- `FUNCTION CURRENT-DATE` returns 21 characters with time zone

ACCEPT FORMS

```
       WORKING-STORAGE SECTION.
       01  WS-PARM         PIC X(20).
       01  WS-TODAY        PIC 9(8).
       01  WS-NOW          PIC 9(8).
       01  WS-DOW          PIC 9.
       01  WS-STAMP.
           05 WS-YYYY      PIC 9(4).
           05 WS-MM        PIC 99.
           05 WS-DD        PIC 99.
           05 FILLER       PIC X(13).
       PROCEDURE DIVISION.
           ACCEPT WS-PARM                *> next SYSIN line
           ACCEPT WS-TODAY FROM DATE YYYYMMDD  *> 20261112
           ACCEPT WS-NOW   FROM TIME     *> HHMMSShh 14302599
           ACCEPT WS-DOW   FROM DAY-OF-WEEK    *> 1=Mon..7=Sun
           MOVE FUNCTION CURRENT-DATE TO WS-STAMP
           DISPLAY WS-YYYY "-" WS-MM "-" WS-DD *> 2026-11-12
           STOP RUN.

      *> JCL:  //SYSIN DD *
      *>       BRANCH042
      *>       /*
```

CODE EXPLAINED

- → `ACCEPT WS-PARM` reads `BRANCH042` from the JCL's in-stream SYSIN
- → `TIME` is hours, minutes, seconds and hundredths
- → Lay out `CURRENT-DATE` as a group to pick out parts
- → For startup parameters, many shops prefer `PARM=` + LINKAGE

MODERN EQUIVALENT

- ↔ **Python** `input()` ; `date.today().strftime("%Y%m%d")`
- ↔ **Java** `scanner.nextLine()` ; `LocalDate.now().format(BASIC_ISO_DATE)`
- ↔ **JavaScript** `readline` for input; `new Date().toISOString().slice(0,10)`

PREFER / AVOID

- ⇄ `ACCEPT … FROM DATE` (2-digit year) → `DATE YYYYMMDD`
- ⇄ hard-coded run dates → read them from SYSIN so reruns can override

ACCEPT FROM

SOURCE	FORMAT
DATE	YYMMDD
DATE YYYYMMDD	YYYYMMDD
DAY	YYDDD (Julian)
DAY YYYYDDD	YYYYDDD
DAY-OF-WEEK	1 (Mon) – 7 (Sun)
TIME	HHMMSShh

CURRENT-DATE (21 CHARS)

POSITIONS	VALUE
1–4	Year
5–8	Month, day
9–14	Hours, minutes, seconds
15–16	Hundredths
17–21	UTC offset, e.g. -0600

⚠ COMMON MISTAKES

- Using 2-digit `FROM DATE` (the Y2K bug in miniature)
- No SYSIN DD in the JCL, so `ACCEPT` gets end-of-file or abends
- Accepting into a numeric field and getting non-numeric data

✓ TIPS & BEST PRACTICES

- Accept into `PIC X` , validate with `IS NUMERIC` , then move
- Use `FUNCTION CURRENT-DATE` when you also need the time zone
- Let SYSIN override the run date for testing and reruns

QUICK SUMMARY

- `ACCEPT x` — read SYSIN or keyboard
- `FROM DATE YYYYMMDD` — 4-digit date
- `FROM TIME` — HHMMSShh
- `CURRENT-DATE` — date, time, offset

Sheet 08

Comments & Documentation

∗ in column 7 · ∗> inline comments · / page eject · program header blocks · change logs · AUTHOR (obsolete)

DOCUMENTING LONG-LIVED CODE

COBOL programs often outlive their authors by decades, so comments carry history. A ∗ in **column 7** makes the whole line a comment; COBOL 2002 added ∗> for comments anywhere on a line. Most shops require a **header block** with purpose, inputs, outputs and a change log.

KEY POINTS

- `/` in column 7 is a comment that also starts a new listing page
- `*>` works in free format and modern fixed format
- `AUTHOR`, `DATE-WRITTEN` and `REMARKS` are obsolete paragraphs
- Comment the **why**: business rules and ticket numbers

HEADER BLOCK & COMMENT STYLES

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. INTCALC.
      *================================================*
      * PROGRAM : INTCALC                              *
      * PURPOSE : Monthly interest on savings accounts *
      * INPUT   : ACCTIN  (account master, 200 bytes)  *
      * OUTPUT  : ACCTOUT (updated master), RPTOUT     *
      *------------------------------------------------*
      * DATE       WHO  CHANGE                         *
      * 2019-03-04 RJB  Initial version                *
      * 2026-11-12 RJB  CHG0042 tiered rates           *
      *================================================*
       PROCEDURE DIVISION.
      /
      * New listing page starts here
           COMPUTE WS-INT ROUNDED = WS-BAL * WS-RATE
      *    Rate is annual / 12; see policy FIN-17
           ADD WS-INT TO WS-BAL            *> inline (2002+)
           STOP RUN.
```

CODE EXPLAINED

→ The header block is just `*` comment lines in column 7

→ Change-log lines tie code changes to tickets (CHG0042)

→ `/` forces a page break in the compiler listing

→ `*>` needs a compiler that supports 2002 syntax

MODERN EQUIVALENT

↔ **Python** `# comment` and `"""docstring"""` at module top

↔ **Java** `// comment`, `/* */`, `/** Javadoc */` above classes

↔ **JavaScript** `// comment`, `/* */`, `/** JSDoc */` above functions

PREFER / AVOID

⇄ commented-out dead code → delete it; version control remembers

⇄ change log in comments only → also commit messages in Git

COMMENT SYNTAX

FORM	WHERE
`* in col 7`	Whole line
`/ in col 7`	Whole line + page eject
`*>`	Rest of line (2002+)
`D in col 7`	Debug-only code

HEADER FIELDS

FIELD	CONTENT
`PROGRAM`	Name = load module
`PURPOSE`	Business function
`INPUT / OUTPUT`	DD names + layouts
`CALLS`	Subprograms used
`CHANGE LOG`	Date, who, ticket

⚠ COMMON MISTAKES

- Putting `*` in column 1 instead of 7, which makes it code
- Inline `*>` on a compiler that only supports COBOL-85
- Comments that restate the code instead of the business rule

✓ TIPS & BEST PRACTICES

- List every DD name the program uses in the header
- Put copybook and layout names next to file definitions
- Remove obsolete `AUTHOR` paragraphs only if the shop agrees

QUICK SUMMARY

- `*` col 7 — line comment
- `*>` — inline comment
- `/` — comment + page eject
- header block — purpose, I/O, changes

DATA DIVISION Sections

FILE SECTION · WORKING-STORAGE SECTION · LOCAL-STORAGE SECTION · LINKAGE SECTION · section order · storage lifetime

WHERE EVERY VARIABLE LIVES

All data is declared in the DATA DIVISION, in fixed sections. **FILE SECTION** describes record buffers, **WORKING-STORAGE** holds variables that keep their values between calls, **LOCAL-STORAGE** is fresh on every call, and **LINKAGE** describes data passed in by a caller.

KEY POINTS

- Order: FILE, WORKING-STORAGE, LOCAL-STORAGE, LINKAGE
- WORKING-STORAGE is initialized once, when the program loads
- LOCAL-STORAGE is re-initialized on each `CALL`
- LINKAGE items have no storage of their own

ALL FOUR SECTIONS

```
       DATA DIVISION.
       FILE SECTION.
       FD  CUST-FILE.
       01  CUST-REC.
           05 CUST-ID        PIC 9(6).
           05 CUST-NAME      PIC X(30).
       WORKING-STORAGE SECTION.
       01  WS-CALL-COUNT     PIC 9(5) VALUE 0.
       01  WS-EOF            PIC X    VALUE "N".
           88 END-OF-FILE             VALUE "Y".
       LOCAL-STORAGE SECTION.
       01  LS-TEMP           PIC 9(5) VALUE 0.
       LINKAGE SECTION.
       01  LK-PARM.
           05 LK-LEN         PIC S9(4) COMP.
           05 LK-DATA        PIC X(100).
       PROCEDURE DIVISION USING LK-PARM.
           ADD 1 TO WS-CALL-COUNT   *> grows each call
           ADD 1 TO LS-TEMP         *> always 1
           GOBACK.
```

CODE EXPLAINED

- → `CUST-REC` is the buffer that each `READ` fills
- → `WS-CALL-COUNT` keeps counting across calls
- → `LS-TEMP` resets to 0 on each call
- → `LK-PARM` maps the caller's storage (or the JCL `PARM=`)

MODERN EQUIVALENT

- ↔ **Python** Module globals vs function locals vs function parameters
- ↔ **Java** Static fields vs local variables vs method parameters
- ↔ **JavaScript** Module-scope `let` vs function locals vs function arguments

PREFER / AVOID

- ⇄ counters in LOCAL-STORAGE → WORKING-STORAGE when they must persist
- ⇄ recursive programs with WORKING-STORAGE → LOCAL-STORAGE + `RECURSIVE`

SECTIONS

SECTION	LIFETIME
FILE	Record buffer per file
WORKING-STORAGE	Whole run unit
LOCAL-STORAGE	One call
LINKAGE	Caller owns it
REPORT	Report Writer (rare)
SCREEN	Screen I/O (GnuCOBOL)

⚠ COMMON MISTAKES

- Reading a FILE SECTION record after end-of-file (contents are undefined)
- Expecting LOCAL-STORAGE values to survive between calls
- Writing to a LINKAGE item that the caller did not pass

✓ TIPS & BEST PRACTICES

- Use `WS-`, `LS-` and `LK-` prefixes to show each item's section
- Copy file records into WORKING-STORAGE with `READ … INTO`
- Keep LINKAGE layouts in a copybook shared with callers

QUICK SUMMARY

- `FILE SECTION` — record buffers
- `WORKING-STORAGE` — persistent
- `LOCAL-STORAGE` — per call
- `LINKAGE` — passed-in data

Sheet 10

Level Numbers & Group Items

01 records · 05–49 levels · elementary vs group · FILLER · 77 items · group = alphanumeric

BUILDING RECORDS FROM LEVELS

COBOL data is a **hierarchy**: an `01` record contains numbered sub-levels, and a smaller-numbered item contains every larger-numbered item below it. Items with a `PIC` are **elementary**; items without one are **groups**, which are just the concatenated bytes of their children.

KEY POINTS

- Levels 01–49 build hierarchies; any increasing numbers work
- Common style: 01, 05, 10, 15 so you can insert levels later
- `FILLER` reserves unnamed bytes (spacing, unused fields)
- `77` is a standalone elementary item (old style)

A CUSTOMER RECORD

```
WORKING-STORAGE SECTION.
01  CUSTOMER-REC.
    05 CUST-ID          PIC 9(6).
    05 CUST-NAME.
       10 FIRST-NAME    PIC X(15).
       10 LAST-NAME     PIC X(20).
    05 CUST-ADDR.
       10 STREET        PIC X(30).
       10 CITY          PIC X(20).
       10 STATE         PIC XX.
       10 ZIP           PIC 9(5).
    05 FILLER           PIC X(2).
    05 BALANCE          PIC S9(7)V99.
77  WS-COUNT           PIC 9(4) VALUE 0.

    MOVE "ADA"      TO FIRST-NAME
    MOVE SPACES     TO CUST-ADDR      *> clears 4 fields
    DISPLAY LENGTH OF CUSTOMER-REC    *> 109
```

CODE EXPLAINED

→ `CUST-NAME` is a group: 35 bytes made of two fields

→ Moving `SPACES` to a group fills all its bytes, numeric fields too

→ `LENGTH OF` gives the record length in bytes

→ `FILLER` keeps the layout aligned with the file format

MODERN EQUIVALENT

↔ **Python** `@dataclass` with nested dataclasses, or a dict of dicts

↔ **Java** Class with nested record fields (`record Address(...)`)

↔ **JavaScript** Nested object literal `{ name: { first, last }, addr: {...} }`

PREFER / AVOID

⇄ `77` items → `01` items (same effect, groups later)

⇄ levels 01, 02, 03 → 01, 05, 10 (room to insert)

LEVEL NUMBERS

LEVEL	MEANING
01	Record (top level)
02–49	Subordinate items
66	RENAMES
77	Standalone item
88	Condition name

GROUP VS ELEMENTARY

ITEM	RULE
Group	No PIC; alphanumeric; its children's bytes
Elementary	Has PIC; has a type
MOVE to group	Byte copy, no conversion
MOVE to elementary	Converted by type

⚠ COMMON MISTAKES

- Giving a group item a `PIC` clause (compile error)
- Moving `SPACES` to a group with numeric children, then doing math on them (SOC7)
- Changing a record layout without matching the file's record length

✓ TIPS & BEST PRACTICES

- Indent each level 3 spaces so the tree is visible
- Put shared layouts in copybooks so every program agrees
- Check record lengths with `LENGTH OF` while testing

QUICK SUMMARY

- `01` — record
- `05` – `49` — children
- no PIC — group (bytes)
- `FILLER` — unnamed bytes

PICTURE — Numeric

DECLARING EXACT-DECIMAL NUMBERS

A numeric PICTURE says how many digits a field holds and where the decimal point is. 9 is a digit, S keeps a sign, and V marks an **implied** decimal point that takes no storage. The result is a fixed-point decimal, ideal for money.

KEY POINTS

- `9(5)` is shorthand for `99999`
- Without `S`, a field is unsigned and drops negative signs
- Max 18 digits by default; 31 with `ARITH(EXTEND)`
- `P` scales without storing digits (rare)

UNDER THE HOOD

- `PIC S9(5)V99` in DISPLAY usage = 7 bytes, sign in the last byte's zone
- The V is never stored: 123.45 is the digits 12345
- The same picture in `COMP-3` takes only 4 bytes

NUMERIC PICTURES

```
       01  WS-COUNT      PIC 9(4)       VALUE 42.
      *>   stored: 0042                  range 0 to 9999
       01  WS-PRICE      PIC 9(5)V99    VALUE 123.45.
      *>   stored: 0012345               range 0 to 99999.99
       01  WS-BALANCE    PIC S9(7)V99   VALUE -500.
      *>   stored: 000050000 + sign      range +/-9999999.99
       01  WS-RATE       PIC V999       VALUE .075.
      *>   stored: 075                   range 0 to .999
       01  WS-MILLIONS   PIC 9(3)PPP    VALUE 12000.
      *>   stored: 012 (scaled by 1000)

           MOVE 12345.678 TO WS-PRICE    *> 12345.67 (truncated)
           MOVE 1234567   TO WS-PRICE    *> 34567.00 (high digits lost)
           MOVE -10       TO WS-COUNT    *> 10 (sign dropped)
           COMPUTE WS-BALANCE = WS-BALANCE * 1.05
```

CODE EXPLAINED

- → Extra decimal digits are **truncated** unless you use `ROUNDED`
- → Too many integer digits are cut from the **left**, silently
- → Unsigned fields store the absolute value
- → Decimal math is exact: no floating-point drift

MODERN EQUIVALENT

- ↔ **Python** `Decimal("123.45")`; `quantize(Decimal("0.01"))` to fix places
- ↔ **Java** `new BigDecimal("123.45").setScale(2, RoundingMode.DOWN)`
- ↔ **JavaScript** No decimal type: store cents as integers or use decimal.js

PREFER / AVOID

- ⇄ money in `COMP-1` / `COMP-2` floats → `S9(n)V99` fixed decimal
- ⇄ unsigned balances → always `S` for anything that can go negative

SYMBOLS

SYMBOL	MEANING
9	Digit 0–9
S	Operational sign
V	Implied decimal point
P	Assumed zero (scaling)
(n)	Repeat count

EXAMPLES

PIC	HOLDS
9(3)	0 to 999
S9(3)	-999 to 999
9(5)V99	0.00 to 99999.99
S9(13)V99	Big money amounts
V99	.00 to .99

⚠ COMMON MISTAKES

- Leaving off `S`, so refunds become positive amounts
- Field too small: high-order digits vanish with no error unless `ON SIZE ERROR`
- Expecting `V` to print a decimal point (use an edited picture)

✓ TIPS & BEST PRACTICES

- Size fields for the largest real value plus headroom
- Use `ON SIZE ERROR` on arithmetic into tight fields
- Match pictures to the copybook exactly when reading files

QUICK SUMMARY

- `9` — digit
- `S` — signed
- `V` — implied decimal
- truncation — left and right, silent

PICTURE — Alphanumeric & Edited

TEXT FIELDS AND DISPLAY FORMATS

PIC X holds any characters and PIC A holds letters and spaces. **Edited** pictures format numbers for people: suppress leading zeros, insert commas and a real decimal point, add currency and signs. You MOVE a number into an edited field; you cannot do math on it.

KEY POINTS

- Alphanumeric moves are left-justified and space-padded
- Edited fields are for output only
- Floating `$$$` or `---` moves the symbol next to the first digit
- `.` in an edited picture is a real, stored period

EDITING IN ACTION

```
01  WS-AMT        PIC S9(5)V99   VALUE -1234.5.
01  WS-CODE       PIC X(6)       VALUE "AB".
01  E1             PIC ZZ,ZZ9.99.
01  E2             PIC $$$,$$9.99.
01  E3             PIC ***,**9.99.
01  E4             PIC ZZ,ZZ9.99CR.
01  E5             PIC -ZZ,ZZ9.99.
01  E6             PIC 99/99/9999.
01  E7             PIC 999B99B9999.

    MOVE WS-AMT TO E1    *>  1,234.50  (sign lost)
    MOVE WS-AMT TO E2    *>  $1,234.50
    MOVE WS-AMT TO E3    *> **1,234.50
    MOVE WS-AMT TO E4    *>  1,234.50CR
    MOVE WS-AMT TO E5    *> - 1,234.50
    MOVE 11122026  TO E6 *> 11/12/2026
    MOVE 123456789 TO E7 *> 123 45 6789
    DISPLAY "[" WS-CODE "]"   *> [AB    ]
```

CODE EXPLAINED

- → `Z` replaces leading zeros with spaces; `*` with asterisks
- → `$$$,$$9` floats the `$` to sit beside the first digit
- → Without `CR`, `DB`, `+` or `-`, the sign is lost
- → `B` inserts a space and `/` a slash, useful for IDs and dates

MODERN EQUIVALENT

- ↔ **Python** `f"{amt:,.2f}"` → `-1,234.50`; `str.ljust(6)` pads text
- ↔ **Java** `String.format("%,.2f", amt)` or `DecimalFormat("#,##0.00")`
- ↔ **JavaScript** `amt.toLocaleString("en-US", {minimumFractionDigits: 2})`

PREFER / AVOID

- ⇄ math on edited fields → keep a numeric field, edit only for output
- ⇄ hand-built date strings → edited `99/99/9999` picture

EDITING SYMBOLS

SYMBOL	EFFECT
`Z`	Leading zero → space
`*`	Leading zero → *
`$`	Currency (floats if repeated)
`, .`	Inserted comma, real point
`CR / DB`	Shown when negative
`+ / -`	Sign (floats if repeated)
`B 0 /`	Insert space, zero, slash

VALUE 0.00 THROUGH

PIC	SHOWS
`ZZZ9.99`	0.00
`ZZZZ.ZZ`	(all spaces)
`$$$9.99`	$0.00
`***9.99`	***0.00

ALPHANUMERIC

PIC	ALLOWS
`X(n)`	Any characters
`A(n)`	Letters + space
`JUSTIFIED RIGHT`	Right-align text

⚠ COMMON MISTAKES

- Using `-ZZ9` but moving a large value, so the leading digit is cut off
- Forgetting a sign symbol, so negatives print as positives
- Doing `ADD` on an edited field (compile error)

✓ TIPS & BEST PRACTICES

- Leave one more position than the largest value for the sign or `$`
- Use `CR` on statements and `-` on internal reports
- Use `BLANK WHEN ZERO` for columns that should be empty when 0

QUICK SUMMARY

- `X` — any text
- `Z` — suppress zeros
- `$$$,$$9.99` — floating currency
- `CR` / `-` — show negatives

USAGE Clauses

HOW A NUMBER IS STORED

USAGE decides the bytes behind a picture. **DISPLAY** stores one character per digit, **COMP-3** packs two digits per byte (the standard for money on mainframes), and **COMP** stores binary integers. The picture still limits the digits; USAGE only changes storage and speed.

KEY POINTS

- Default USAGE is DISPLAY
- COMP-3 is fast for decimal math on IBM Z hardware
- COMP is best for subscripts and counters
- Floats (COMP-1/2) are approximate: never use for money

UNDER THE HOOD

- `PIC S9(5)V99 COMP-3` = 4 bytes: `X'0012345C'` (C = +, D = -)
- COMP `PIC S9(4)` = halfword (2 bytes), `S9(9)` = fullword (4 bytes)
- With `TRUNC(STD)`, COMP is truncated to its PIC digits; `COMP-5` never is

SAME VALUE, DIFFERENT USAGE

```
01  WS-ZONED   PIC S9(5)V99.          *> 7 bytes
01  WS-PACKED  PIC S9(5)V99 COMP-3.   *> 4 bytes
01  WS-BIN     PIC S9(4)    COMP.     *> 2 bytes
01  WS-NATIVE  PIC S9(4)    COMP-5.   *> 2 bytes
01  WS-FLOAT   COMP-2.                *> 8 bytes
01  WS-IDX     USAGE INDEX.
01  WS-PTR     USAGE POINTER.

    MOVE -123.45 TO WS-ZONED     *> F0F0F1F2F3F4D5
    MOVE -123.45 TO WS-PACKED    *> 0012345D
    MOVE 1234    TO WS-BIN       *> 04D2
    MOVE 32000   TO WS-NATIVE    *> fits: 2-byte max 32767
    COMPUTE WS-FLOAT = 1 / 3     *> 0.333333...
    MOVE WS-PACKED TO WS-ZONED   *> converted correctly
```

CODE EXPLAINED

- → Zoned: each digit is a byte, the sign is in the last byte's high nibble
- → Packed: two digits per byte, sign nibble last
- → Moving between USAGEs converts values; group moves copy raw bytes
- → `COMP-5` holds the full binary range regardless of PIC digits

MODERN EQUIVALENT

- ↔ **Python** `int` (arbitrary size), `float`, `Decimal`; `struct.pack` for raw bytes
- ↔ **Java** `int`, `long`, `double`, `BigDecimal`; `ByteBuffer` for raw bytes
- ↔ **JavaScript** `Number` (double), `BigInt`; `DataView` for binary layouts

PREFER / AVOID

- ⇄ DISPLAY for heavy arithmetic → COMP-3 (or COMP for counters)
- ⇄ even digit counts in COMP-3 → odd (`S9(7)`) to use every nibble

USAGE TYPES

USAGE	STORAGE
DISPLAY	1 byte per digit
COMP-3 / PACKED-DECIMAL	(digits + 1) / 2 bytes
COMP / BINARY / COMP-4	2, 4 or 8 bytes
COMP-5	Native binary, full range
COMP-1	4-byte float
COMP-2	8-byte float

BINARY SIZES

PIC DIGITS	BYTES
1–4	2 (halfword)
5–9	4 (fullword)
10–18	8 (doubleword)

⚠ COMMON MISTAKES

- Reading packed data with the wrong PIC length, which shifts every field
- Displaying a COMP-3 field via a group or `X` redefine (shows garbage)
- Using COMP-2 for currency and getting rounding errors

✓ TIPS & BEST PRACTICES

- Use odd digit counts for COMP-3 fields
- Check the copybook for USAGE when mapping a file layout
- Prefer `COMP-5` for values passed to C or Java

QUICK SUMMARY

- `DISPLAY` — readable digits
- `COMP-3` — packed decimal
- `COMP` / `COMP-5` — binary
- `COMP-1/2` — floats, not for money

VALUE & Figurative Constants

VALUE clause · ZERO / ZEROS · SPACE / SPACES · HIGH-VALUES / LOW-VALUES · QUOTE · ALL "literal"

STARTING VALUES AND BUILT-IN CONSTANTS

The VALUE clause sets an item's initial value when the program loads. **Figurative constants** are reserved words that fill a field of any size: ZEROS, SPACES, HIGH-VALUES (all bits on) and LOW-VALUES (all bits off). They work in VALUE, MOVE and comparisons.

KEY POINTS

- WORKING-STORAGE without VALUE starts as whatever was in memory
- Singular and plural forms are identical: `ZERO` = `ZEROS` = `ZEROES`
- `HIGH-VALUES` sorts after everything: a classic end-of-file key
- `VALUE` on a group sets the whole group as alphanumeric

UNDER THE HOOD

- `HIGH-VALUES` = `X'FF'` bytes; `LOW-VALUES` = `X'00'`
- `SPACES` = `X'40'` in EBCDIC, `X'20'` in ASCII
- `ZEROS` moved to a COMP-3 field gives `X'000C'`, a valid packed 0

VALUE AND CONSTANTS

```
01  WS-TOTAL      PIC 9(7)V99   VALUE ZERO.
01  WS-NAME       PIC X(20)     VALUE SPACES.
01  WS-TITLE      PIC X(20)     VALUE "MONTHLY REPORT".
01  WS-LINE       PIC X(40)     VALUE ALL "-".
01  WS-KEY        PIC X(10)     VALUE LOW-VALUES.
01  WS-PI         PIC 9V9(4)    VALUE 3.1416.
01  WS-HEX        PIC X(2)      VALUE X"0D25".
01  WS-HDR.
    05 FILLER     PIC X(6)      VALUE "PAGE: ".
    05 HDR-PAGE   PIC ZZ9.

    MOVE HIGH-VALUES TO WS-KEY      *> end-of-file marker
    MOVE QUOTE TO WS-NAME(1:1)      *> "
    IF WS-NAME = SPACES
       DISPLAY "NAME IS BLANK"
    END-IF
    MOVE ALL "AB" TO WS-NAME        *> ABABABABAB...
```

CODE EXPLAINED

- → `ALL "-"` repeats the literal to fill all 40 bytes
- → `FILLER` with `VALUE` builds constant report text
- → `X"0D25"` is a hexadecimal literal
- → Comparing to `SPACES` works for any length

MODERN EQUIVALENT

- ↔ **Python** `" " * 20`, `"-" * 40`, `0`, `None`; `b"\xff" * 10`
- ↔ **Java** `" ".repeat(20)`, `"-".repeat(40)`, `0`; `Arrays.fill`
- ↔ **JavaScript** `" ".repeat(20)`, `"-".repeat(40)`, `0`; `Buffer.alloc(10, 0xff)`

PREFER / AVOID

- ⇄ no VALUE on counters → `VALUE ZERO` (or `INITIALIZE` at start)
- ⇄ magic `"9999999999"` keys → `HIGH-VALUES`

FIGURATIVE CONSTANTS

CONSTANT	FILLS WITH
ZERO / ZEROS	0
SPACE / SPACES	Blanks
HIGH-VALUE(S)	Highest byte X'FF'
LOW-VALUE(S)	Lowest byte X'00'
QUOTE(S)	" character
ALL lit	Repeated literal
NULL(S)	Null pointer

⚠ COMMON MISTAKES

- Doing arithmetic on numeric fields with no VALUE (S0C7 data exception)
- Moving `HIGH-VALUES` to a numeric field, which is not a valid number
- Expecting a `VALUE` to be reapplied on each call of a subprogram

✓ TIPS & BEST PRACTICES

- Give every counter and accumulator `VALUE ZERO`
- Build fixed report headings with `FILLER VALUE` items
- Use `INITIALIZE` to reset records between iterations

QUICK SUMMARY

- `VALUE` — initial value at load
- `ZEROS` / `SPACES` — fill
- `HIGH-VALUES` — sorts last
- `ALL "x"` — repeat a literal

Condition Names (Level 88)

NAMING THE VALUES THAT MATTER

A level-**88** item gives a name to one or more values of the field above it. You test the name like a boolean (`IF END-OF-FILE`) and turn it on with `SET … TO TRUE`. This replaces magic values like `"Y"` and `"A"` scattered through the logic.

KEY POINTS

- 88s take no storage; they describe the parent field
- One 88 can list several values or ranges
- Several 88s on one field act like an enum
- `SET name TO FALSE` needs a `WHEN SET TO FALSE` value

FLAGS AND ENUMS

```
       01  WS-EOF-FLAG       PIC X      VALUE "N".
           88 END-OF-FILE               VALUE "Y".
           88 MORE-RECORDS              VALUE "N".
       01  WS-STATUS         PIC X.
           88 ACTIVE                    VALUE "A".
           88 CLOSED                    VALUE "C" "X".
           88 VALID-STATUS              VALUE "A" "C" "X".
       01  WS-AGE            PIC 999.
           88 MINOR                     VALUE 0 THRU 17.
           88 SENIOR                    VALUE 65 THRU 120.
       01  WS-DEBUG          PIC X      VALUE "N".
           88 DEBUG-ON                  VALUE "Y"
                                WHEN SET TO FALSE "N".

           PERFORM UNTIL END-OF-FILE
              READ IN-FILE AT END SET END-OF-FILE TO TRUE
              END-READ
           END-PERFORM
           IF NOT VALID-STATUS
              DISPLAY "BAD STATUS " WS-STATUS
           END-IF
           SET DEBUG-ON TO FALSE           *> moves "N"
```

CODE EXPLAINED

- → `SET END-OF-FILE TO TRUE` moves `"Y"` to the parent field
- → `CLOSED` is true for either `"C"` or `"X"`
- → `THRU` defines a range, inclusive at both ends
- → `WHEN SET TO FALSE` tells `SET … FALSE` what to move

MODERN EQUIVALENT

- ↔ **Python** `Enum` class or constants; `status in {"C", "X"}`
- ↔ **Java** `enum Status { ACTIVE, CLOSED }`; `boolean eof`
- ↔ **JavaScript** Object of constants or a TypeScript `enum`; `Set.has()`

PREFER / AVOID

- ⇄ `IF WS-STATUS = "A"` → `IF ACTIVE`
- ⇄ `MOVE "Y" TO WS-EOF-FLAG` → `SET END-OF-FILE TO TRUE`

88 FORMS

FORM	EXAMPLE
`Single value`	VALUE "Y"
`List`	VALUE "C" "X"
`Range`	VALUE 0 THRU 17
`Mixed`	VALUE 1 3 5 THRU 9
`False value`	WHEN SET TO FALSE "N"

USING 88S

STATEMENT	EFFECT
`IF ACTIVE`	Test
`IF NOT ACTIVE`	Negated test
`SET ACTIVE TO TRUE`	Moves "A"
`PERFORM UNTIL END-OF-FILE`	Loop control
`EVALUATE TRUE / WHEN ACTIVE`	Branch

⚠ COMMON MISTAKES

- `SET … TO TRUE` on an 88 with several values moves only the first one
- Testing the parent field with literals and the 88 elsewhere (inconsistent)
- Using `SET … TO FALSE` without a `WHEN SET TO FALSE` clause

✓ TIPS & BEST PRACTICES

- Name 88s as yes/no questions: `IS-VALID`, `END-OF-FILE`
- Add a `VALID-…` 88 listing every legal code for validation
- Define 88s in the copybook so every program shares them

QUICK SUMMARY

- `88` — named value(s)
- `IF name` — test
- `SET name TO TRUE` — assign
- `THRU` — inclusive range

REDEFINES & RENAMES

REDEFINES · overlaying storage · alternate record layouts · numeric / text views · 66 RENAMES · data exceptions

TWO NAMES FOR THE SAME BYTES

REDEFINES lays a second description over the same storage, so one area can be read as different layouts. It is how COBOL handles files with several record types and how it splits a date into parts. 66 RENAMES gives a new name to a range of adjacent fields.

KEY POINTS

- The redefining item must follow the original at the same level
- No new storage: both names share the bytes
- Only the first description may have a `VALUE` (except 88s)
- The redefining item must not be longer than the original (at 01 it can be)

UNDER THE HOOD

- A packed field viewed through `PIC X` shows raw nibbles, not digits
- Arithmetic on a numeric view holding spaces raises **SOC7**
- REDEFINES generates no code: it only changes how offsets are named

RECORD TYPES AND DATE PARTS

```
01  TRAN-REC.
    05 TRAN-TYPE        PIC X.
       88 HEADER-REC              VALUE "H".
       88 DETAIL-REC              VALUE "D".
    05 TRAN-DATA        PIC X(79).
    05 HDR-DATA REDEFINES TRAN-DATA.
       10 HDR-BATCH     PIC 9(6).
       10 HDR-DATE      PIC 9(8).
       10 FILLER        PIC X(65).
    05 DTL-DATA REDEFINES TRAN-DATA.
       10 DTL-ACCT      PIC 9(10).
       10 DTL-AMT       PIC S9(7)V99.
       10 FILLER        PIC X(60).
01  WS-DATE            PIC 9(8).
01  WS-DATE-X REDEFINES WS-DATE.
    05 WS-YYYY          PIC 9(4).
    05 WS-MM            PIC 99.
    05 WS-DD            PIC 99.

    IF DETAIL-REC
       ADD DTL-AMT TO WS-TOTAL
    END-IF
    MOVE 20261112 TO WS-DATE
    DISPLAY WS-MM "/" WS-DD "/" WS-YYYY  *> 11/12/2026
```

CODE EXPLAINED

→ `TRAN-TYPE` decides which view of `TRAN-DATA` is valid

→ Each record type keeps the same total length (79 bytes)

→ `WS-DATE-X` splits the date with no `MOVE` s

→ Always check the type field before using a view

MODERN EQUIVALENT

↔ **Python** Tagged union: `match rec.type:` then a dataclass per type

↔ **Java** Sealed interface with records per type; `ByteBuffer` views

↔ **JavaScript** Discriminated union: `switch (rec.type)` ; `DataView` over a buffer

PREFER / AVOID

⇄ reading a view without checking the type → test the 88 first

⇄ REDEFINES for unrelated scratch fields → separate fields

REDEFINES RULES

RULE	DETAIL
Placement	Directly after the original
Level	Same level number
Length	Not longer (except 01)
VALUE	Not on the redefining item
OCCURS	Original may not have OCCURS

66 RENAMES

FORM	MEANING
66 A RENAMES X	Alias of X
66 A RENAMES X THRU Z	Bytes from X to Z
Placement	After the 01 record
Use	Rare in new code

⚠ COMMON MISTAKES

- Using the detail view on a header record, which feeds text into arithmetic (SOC7)
- Changing one layout's length so the views no longer line up
- Putting `VALUE` on the redefining item

✓ TIPS & BEST PRACTICES

- Pair every REDEFINES view with an 88 on the type code
- Keep all views the same length, padding with `FILLER`
- Prefer REDEFINES over reference modification for fixed sub-fields

QUICK SUMMARY

- `REDEFINES` — second layout, same bytes
- type code + 88 — choose the view
- `66 RENAMES` — alias a field range
- wrong view — SOC7 risk

MOVE

MOVE … TO · padding & truncation · decimal-point alignment · group moves · MOVE CORRESPONDING · multiple receivers

COPYING DATA BETWEEN FIELDS

MOVE copies a value into one or more fields, converting it to fit each receiver. **Alphanumeric** receivers are filled left to right and padded with spaces; **numeric** receivers line up on the decimal point and pad with zeros. Anything that does not fit is cut off silently.

KEY POINTS

- The source is unchanged; the receiver is overwritten
- Text: truncated on the right
- Numbers: integer digits cut on the left, decimals on the right
- Group moves are raw byte copies with no conversion

UNDER THE HOOD

- A group `MOVE` compiles to a single MVC instruction (byte copy)
- Numeric to numeric moves may PACK/UNPK to convert USAGE
- Moving alphanumeric to numeric copies bytes: non-digits cause later SOC7

MOVE RULES IN ACTION

```
01  WS-NAME      PIC X(5).
01  WS-NUM       PIC 9(3)V99.
01  WS-SMALL     PIC 99.
01  WS-OUT       PIC 9(5).
01  A-REC.
    05 CUST-ID   PIC 9(4).
    05 AMOUNT    PIC 9(5)V99.
01  B-REC.
    05 AMOUNT    PIC 9(5)V99.
    05 CUST-ID   PIC 9(4).

    MOVE "AB" TO WS-NAME          *> "AB   "
    MOVE "ABCDEFG" TO WS-NAME     *> "ABCDE"
    MOVE 12.5 TO WS-NUM           *> 012.50
    MOVE 1234.567 TO WS-NUM       *> 234.56
    MOVE 1234 TO WS-SMALL         *> 34
    MOVE WS-NUM TO WS-OUT         *> 00234 (decimals dropped)
    MOVE ZERO TO WS-NUM WS-OUT    *> two receivers
    MOVE CORRESPONDING A-REC TO B-REC
    MOVE CUST-ID OF A-REC TO WS-OUT  *> qualified name
```

CODE EXPLAINED

- → Short text is padded with spaces on the right
- → Numbers align on the decimal point; extra digits fall off both ends
- → `MOVE CORRESPONDING` moves fields with matching names, in any order
- → `OF` (or `IN`) qualifies a name used in more than one record

MODERN EQUIVALENT

- ↔ **Python** `b = a`; slicing `s[:5].ljust(5)` mimics PIC X(5)
- ↔ **Java** `b = a`; fixed-width needs `String.format("%-5.5s", s)`
- ↔ **JavaScript** `b = a`; `s.slice(0, 5).padEnd(5)` mimics PIC X(5)

PREFER / AVOID

- ⇄ silent truncation → size receivers to the largest value
- ⇄ `MOVE CORRESPONDING` on big records → explicit moves (clearer)

MOVE RULES

FROM → TO	RESULT
X → X	Left-justify, pad spaces, cut right
9 → 9	Align point, pad zeros, cut both ends
9 → X	Digits as text (unsigned integer)
X → 9	Bytes copied; must be digits
9 → edited	Formatted for output
group → any	Byte copy, no conversion

LEGAL MOVES

SOURCE	ALLOWED TO
Alphanumeric	X, edited, group
Numeric integer	All types
Numeric with decimals	Numeric, edited (not X)
Edited	X, or numeric (de-edit)
Figurative	Any field

⚠ COMMON MISTAKES

- Moving a 7-digit total into a `PIC 9(5)` field and losing the high digits
- Moving text into a numeric field and doing math on it later (SOC7)
- Expecting a group move to convert COMP-3 to readable digits

✓ TIPS & BEST PRACTICES

- Compile with warnings to catch truncating moves
- Use `READ … INTO` and `WRITE … FROM` instead of extra moves
- Qualify duplicate names with `OF` to avoid ambiguity errors

QUICK SUMMARY

- `MOVE a TO b c` — copy to receivers
- text — pad/cut right
- numbers — align on the point
- group — raw bytes

INITIALIZE & SET

RESETTING DATA AND SETTING FLAGS

INITIALIZE resets every field in a record by type: numeric fields to zero, alphanumeric fields to spaces, and leaves FILLER alone. SET assigns special items: condition names, indexes, pointers and addresses.

KEY POINTS

- INITIALIZE skips `FILLER` and `REDEFINES` items
- `WITH FILLER` (2002+) includes FILLER too
- `TO VALUE` resets fields to their VALUE clauses
- `SET` is the only way to change an index

INITIALIZE AND SET

```
01  ORDER-REC.
    05 ORD-ID       PIC 9(6).
    05 ORD-CUST     PIC X(20).
    05 ORD-AMT      PIC S9(7)V99 COMP-3.
    05 FILLER       PIC X(5)  VALUE "*END*".
    05 ORD-STATUS   PIC X     VALUE "N".
       88 ORD-NEW             VALUE "N".
       88 ORD-SHIPPED         VALUE "S".
01  WS-TABLE.
    05 WS-ITEM      PIC X(10) OCCURS 50 INDEXED BY IX.
01  WS-PTR          USAGE POINTER.

    INITIALIZE ORDER-REC           *> 0, spaces, 0; FILLER kept
    INITIALIZE ORDER-REC
       REPLACING ALPHANUMERIC BY "?"
                 NUMERIC      BY 1
    INITIALIZE ORDER-REC TO VALUE  *> status back to "N"
    SET ORD-SHIPPED TO TRUE        *> ORD-STATUS = "S"
    SET IX TO 1
    SET IX UP BY 2                 *> IX now 3
    SET WS-PTR TO ADDRESS OF ORDER-REC
```

CODE EXPLAINED

- → Plain `INITIALIZE` gives zeros and spaces, not VALUE clauses
- → `REPLACING` chooses a different fill per category
- → `SET … TO TRUE` moves the 88's value into its parent
- → Indexes hold offsets, so only `SET` and `SEARCH` change them

MODERN EQUIVALENT

- ↔ **Python** `rec = Order()` (new dataclass with defaults)
- ↔ **Java** `order = new Order();` or a `reset()` method
- ↔ **JavaScript** `order = { ...defaultOrder }`

PREFER / AVOID

- ⇄ `MOVE SPACES TO record` → `INITIALIZE` (keeps numerics valid)
- ⇄ `MOVE "S" TO ORD-STATUS` → `SET ORD-SHIPPED TO TRUE`

INITIALIZE FILLS

CATEGORY	DEFAULT
Numeric, numeric-edited	Zero
Alphanumeric, alpha-edited	Spaces
FILLER	Unchanged
Index, pointer	Unchanged
TO VALUE	VALUE clause

SET TARGETS

STATEMENT	EFFECT
SET cond TO TRUE	Assign 88 value
SET idx TO n	Index to occurrence n
SET idx UP / DOWN BY n	Move index
SET ptr TO ADDRESS OF x	Pointer
SET ADDRESS OF lk TO ptr	Map LINKAGE item

⚠ COMMON MISTAKES

- `MOVE SPACES` to a record with numeric fields, then adding to them (SOC7)
- Expecting INITIALIZE to restore `VALUE` clauses without `TO VALUE`
- Using `MOVE` or `ADD` on an index

✓ TIPS & BEST PRACTICES

- INITIALIZE output records before building each one
- Use `TO VALUE` to reset report headings and defaults
- Use `SET` with 88s everywhere flags change

QUICK SUMMARY

- `INITIALIZE` — zeros & spaces
- `REPLACING` — custom fill
- `SET cond TO TRUE` — flag
- `SET idx` — indexes

Editing Fields for Output

edited PICs in action · BLANK WHEN ZERO · JUSTIFIED RIGHT · SIGN LEADING SEPARATE · currency & dates · column-aligned output

MAKING NUMBERS READABLE

Report lines are built from **edited** fields and fixed text, then written as one record. Clauses like `BLANK WHEN ZERO`, `JUSTIFIED RIGHT` and `SIGN … SEPARATE` control how values sit in their columns, so a printed report lines up without any string math.

KEY POINTS

- Build each print line as an `01` group of fields and FILLER
- Widths of the edited fields define the columns
- `SIGN LEADING SEPARATE` puts a real +/- character first
- Dates can use `99/99/9999` or `9999-99-99` style pictures

A DETAIL LINE

```
01  DETAIL-LINE.
    05 DL-ACCT      PIC 9(6).
    05 FILLER       PIC X(2)   VALUE SPACES.
    05 DL-NAME      PIC X(20).
    05 DL-CITY      PIC X(12)  JUSTIFIED RIGHT.
    05 FILLER       PIC X(2)   VALUE SPACES.
    05 DL-BAL       PIC $$$,$$9.99-.
    05 FILLER       PIC X      VALUE SPACE.
    05 DL-FEES      PIC ZZ9.99 BLANK WHEN ZERO.
    05 FILLER       PIC X      VALUE SPACE.
    05 DL-OPENED    PIC 9999/99/99.
01  WS-EXPORT-AMT   PIC S9(5)V99
                    SIGN LEADING SEPARATE.

    MOVE CUST-ACCT  TO DL-ACCT
    MOVE CUST-NAME  TO DL-NAME
    MOVE CUST-CITY  TO DL-CITY       *> right-aligned
    MOVE CUST-BAL   TO DL-BAL        *> $1,234.50-
    MOVE ZERO       TO DL-FEES       *> blanks
    MOVE 20190304   TO DL-OPENED     *> 2019/03/04
    WRITE PRINT-REC FROM DETAIL-LINE
        AFTER ADVANCING 1 LINE
```

CODE EXPLAINED

- → Each `MOVE` fills one column of the print line
- → `JUSTIFIED RIGHT` pads text on the left instead of the right
- → A trailing `-` shows the sign after the amount
- → `SIGN LEADING SEPARATE` is common for files sent to other systems

MODERN EQUIVALENT

- ↔ **Python** f-strings with widths: `f"{acct:06d} {name:<20} {bal:>12,.2f}"`
- ↔ **Java** `String.format("%06d %-20s%,12.2f", …)`
- ↔ **JavaScript** Template string with `padStart` / `padEnd` / `toLocaleString`

PREFER / AVOID

- ⇄ building lines with `STRING` → a group of edited fields
- ⇄ hard-coded column positions → named FILLER items

CLAUSES

CLAUSE	EFFECT
`BLANK WHEN ZERO`	All spaces if value is 0
`JUSTIFIED RIGHT`	Right-align text
`SIGN LEADING SEPARATE`	Sign as first character
`SIGN TRAILING SEPARATE`	Sign as last character
`SIGN LEADING`	Sign in first digit's zone

DATE PICTURES

PIC	FROM 20261112
`9999/99/99`	2026/11/12
`9999B99B99`	2026 11 12
`99/99/9999`	needs MMDDYYYY: 11/12/2026

⚠ COMMON MISTAKES

- Line group longer than the print record, so the end is cut
- Moving a YYYYMMDD date into a `99/99/9999` picture and getting 20/26/1112
- Forgetting a sign position, so credits look like debits

✓ TIPS & BEST PRACTICES

- Total the widths of each line group against the record length
- Rearrange dates with REDEFINES before editing them
- Keep heading and detail columns in matching widths

QUICK SUMMARY

- line group — edited fields + FILLER
- `BLANK WHEN ZERO` — empty zeros
- `JUSTIFIED RIGHT` — align text
- `SIGN … SEPARATE` — real sign char

Data Conversion

MOVE between USAGEs · FUNCTION NUMVAL / NUMVAL-C · de-editing · zoned ↔ packed · text → number pitfalls · NUMERIC class test

TURNING TEXT INTO NUMBERS SAFELY

COBOL converts between numeric USAGEs automatically on MOVE and in arithmetic. Text is different: a PIC X field holding "12.50" is not a number. Use FUNCTION NUMVAL (or NUMVAL-C for currency) to parse it, and test IS NUMERIC before trusting any input.

KEY POINTS

- DISPLAY ↔ COMP-3 ↔ COMP conversions are automatic
- `NUMVAL` accepts spaces, a sign and a decimal point
- `NUMVAL-C` also accepts currency signs and commas
- Moving an edited field to a numeric field **de-edits** it

UNDER THE HOOD

- `IS NUMERIC` on a zoned field checks every byte is X'F0'–X'F9' (sign allowed last)
- Packed fields are NUMERIC only if every nibble is a digit and the sign is C, D or F
- Bad packed data is the classic SOC7 cause

PARSING AND CONVERTING

```
       01  IN-AMOUNT      PIC X(12)        VALUE "  -1,234.50".
       01  IN-QTY         PIC X(5)         VALUE "00042".
       01  WS-AMT         PIC S9(7)V99     COMP-3.
       01  WS-QTY         PIC 9(5).
       01  WS-EDITED      PIC ZZ,ZZ9.99-.
       01  WS-BACK        PIC S9(5)V99.

           COMPUTE WS-AMT = FUNCTION NUMVAL-C(IN-AMOUNT)
      *>   WS-AMT = -1234.50
           IF IN-QTY IS NUMERIC
              MOVE IN-QTY TO WS-QTY            *> 42
           ELSE
              DISPLAY "BAD QTY: " IN-QTY
           END-IF
           IF FUNCTION TEST-NUMVAL(IN-AMOUNT) = 0
              DISPLAY "VALID NUMBER TEXT"
           END-IF
           MOVE WS-AMT TO WS-EDITED            *>  1,234.50-
           MOVE WS-EDITED TO WS-BACK           *> -1234.50 (de-edit)
```

CODE EXPLAINED

→ `NUMVAL-C` strips commas and handles the leading minus

→ `IS NUMERIC` guards the plain `MOVE` from text into a number

→ `TEST-NUMVAL` returns 0 when the text is a valid number

→ De-editing reverses an edited picture back to a value

MODERN EQUIVALENT

↔ **Python** `Decimal(s.replace(",", ""))` inside `try/except InvalidOperation`

↔ **Java** `new BigDecimal(s.replace(",", "").trim())` with `NumberFormatException`

↔ **JavaScript** `Number(s.replace(/,/g, ""))` then `Number.isNaN()` check

PREFER / AVOID

⇄ moving input straight to numeric → `IS NUMERIC` or `TEST-NUMVAL` first

⇄ manual digit-by-digit parsing → `FUNCTION NUMVAL`

CONVERSION TOOLS

TOOL	USE
`MOVE`	Between numeric USAGEs
`NUMVAL(x)`	Text "-12.5" → number
`NUMVAL-C(x)`	Text "$1,234.50" → number
`TEST-NUMVAL(x)`	0 if valid, else bad position
`IS NUMERIC`	Class test on a field
`edited → numeric MOVE`	De-edit

NUMVAL ACCEPTS

INPUT	RESULT
`" 42"`	42
`"-12.50"`	-12.5
`"12.50-"`	-12.5
`"12.50 CR"`	-12.5
`"$1,234" (NUMVAL-C)`	1234

⚠ COMMON MISTAKES

- Moving `"12.50"` from a PIC X field to PIC 9V99 (the period is not a digit)
- Skipping validation on file input, then abending with SOC7 at 2 a.m.
- Using NUMVAL on text with commas (use NUMVAL-C)

✓ TIPS & BEST PRACTICES

- Validate every external numeric field before arithmetic
- Write rejects to an error file instead of abending
- Use `TEST-NUMVAL` to report the position of the bad character

QUICK SUMMARY

- MOVE — converts numeric USAGEs
- `NUMVAL` / `NUMVAL-C` — parse text
- `IS NUMERIC` — validate
- edited → numeric — de-edit

STRING

JOINING TEXT TOGETHER

STRING concatenates pieces into one receiving field. Each piece is copied up to its **delimiter**: SIZE takes the whole field, while SPACE or a literal stops at the first match. This is how COBOL trims trailing blanks while building names, CSV lines and messages.

KEY POINTS

- The receiver is **not** cleared first: move `SPACES` to it
- Copying stops when the receiver is full (overflow)
- `WITH POINTER` tracks the next position to write
- Literals and figurative constants can be pieces

BUILDING A FULL NAME AND CSV LINE

```
       01  FIRST-NAME   PIC X(15)  VALUE "ADA".
       01  LAST-NAME    PIC X(20)  VALUE "LOVELACE".
       01  CITY         PIC X(15)  VALUE "LONDON".
       01  WS-FULL      PIC X(40).
       01  WS-CSV       PIC X(60).
       01  WS-PTR       PIC 99.

           MOVE SPACES TO WS-FULL
           STRING FIRST-NAME DELIMITED BY SPACE
                  " "        DELIMITED BY SIZE
                  LAST-NAME  DELIMITED BY SPACE
             INTO WS-FULL
           END-STRING                    *> "ADA LOVELACE"
           MOVE SPACES TO WS-CSV
           MOVE 1 TO WS-PTR
           STRING LAST-NAME DELIMITED BY "  "
                  ","       DELIMITED BY SIZE
                  CITY      DELIMITED BY "  "
             INTO WS-CSV WITH POINTER WS-PTR
             ON OVERFLOW DISPLAY "CSV TOO LONG"
           END-STRING                    *> "LOVELACE,LONDON"
           DISPLAY WS-PTR                *> 16 (next free byte)
```

CODE EXPLAINED

- → `DELIMITED BY SPACE` stops at the first blank, trimming the field
- → `DELIMITED BY " "` (two spaces) keeps single spaces inside names
- → `WITH POINTER` starts at 1 and ends one past the last byte written
- → `ON OVERFLOW` runs if the receiver fills up

MODERN EQUIVALENT

- ↔ **Python** `" ".join([first, last])` or `f"{first} {last}"`
- ↔ **Java** `String.join(" ", first, last)` or `first + " " + last`
- ↔ **JavaScript** `[first, last].join(" ")` or `` `${first} ${last}` ``

PREFER / AVOID

- ⇄ `DELIMITED BY SPACE` on "MARY ANN" → two-space delimiter or `FUNCTION TRIM`
- ⇄ forgetting to clear the receiver → `MOVE SPACES` before `STRING`

CLAUSES

CLAUSE	MEANING
`DELIMITED BY SIZE`	Whole piece
`DELIMITED BY SPACE`	Up to first blank
`DELIMITED BY lit`	Up to that text
`WITH POINTER p`	Start/next position
`ON OVERFLOW`	Receiver full or bad pointer
`NOT ON OVERFLOW`	Success branch

⚠ COMMON MISTAKES

- Leftover text from the last use because the receiver was not cleared
- Names with spaces ("VAN DYKE") cut at the first space
- Pointer starting at 0, which triggers overflow immediately

✓ TIPS & BEST PRACTICES

- Initialize the pointer to 1 before every STRING
- Use a pointer to append across several STRING statements
- Check `ON OVERFLOW` whenever the input lengths vary

QUICK SUMMARY

- `STRING … INTO` — concatenate
- `DELIMITED BY SPACE` — trim
- `WITH POINTER` — append position
- `ON OVERFLOW` — too long

UNSTRING

SPLITTING TEXT INTO FIELDS

UNSTRING breaks a field apart at delimiters and moves each piece into a receiving field, like `split`. Use it to parse CSV lines, names and free-form input. Optional phrases record each piece's length, which delimiter matched and how many fields were filled.

KEY POINTS

- `ALL SPACES` treats a run of blanks as one delimiter
- List several delimiters with `OR`
- Extra pieces are ignored unless you check `ON OVERFLOW`
- Each receiver is filled using normal MOVE rules

PARSING A CSV LINE

```
       01  IN-LINE      PIC X(60) VALUE "1042,ADA,LOVELACE,LONDON".
       01  WS-ID        PIC X(6).
       01  WS-FIRST     PIC X(15).
       01  WS-LAST      PIC X(20).
       01  WS-CITY      PIC X(15).
       01  WS-LEN-LAST  PIC 99.
       01  WS-FIELDS    PIC 99 VALUE 0.
       01  WS-DELIM     PIC X.

           UNSTRING IN-LINE DELIMITED BY "," OR ";"
               INTO WS-ID
                    WS-FIRST
                    WS-LAST  DELIMITER IN WS-DELIM
                             COUNT IN WS-LEN-LAST
                    WS-CITY
               TALLYING IN WS-FIELDS
               ON OVERFLOW DISPLAY "EXTRA FIELDS"
           END-UNSTRING
      *>   WS-ID "1042"  WS-LAST "LOVELACE"  WS-LEN-LAST 8
      *>   WS-DELIM ","  WS-FIELDS 4

           UNSTRING "ADA    BYRON  KING" DELIMITED BY ALL SPACES
               INTO WS-FIRST WS-LAST WS-CITY
           END-UNSTRING
```

CODE EXPLAINED

- → Each comma-separated piece goes to the next receiver
- → `COUNT IN` gets the length of the piece before padding
- → `TALLYING IN` adds the number of receivers filled (start it at 0)
- → `ALL SPACES` collapses runs of blanks between words

MODERN EQUIVALENT

- ↔ **Python** `line.split(",")` ; `re.split(r"[,;]", line)` ; `s.split()` for spaces
- ↔ **Java** `line.split(",")` ; `line.split("[,;]")` ; `s.trim().split("\\s+")`
- ↔ **JavaScript** `line.split(",")` ; `line.split(/[,;]/)` ; `s.trim().split(/\s+/)`

PREFER / AVOID

- ⇄ quoted CSV with embedded commas → a real parser or `JSON PARSE`
- ⇄ tallying without resetting → `MOVE 0` to the counter first

PHRASES

PHRASE	GIVES
`DELIMITED BY x OR y`	Split points
`ALL lit`	Runs count as one
`DELIMITER IN d`	Which delimiter ended the piece
`COUNT IN n`	Piece length
`WITH POINTER p`	Start position
`TALLYING IN t`	Receivers filled
`ON OVERFLOW`	More data than receivers

⚠ COMMON MISTAKES

- Not resetting the `TALLYING` counter, so counts accumulate
- Receivers too short, silently cutting long values
- Using it on quoted CSV with commas inside quotes

✓ TIPS & BEST PRACTICES

- Unstring into PIC X fields, then validate and convert numbers
- Use `COUNT IN` to detect empty fields (length 0)
- Use `ON OVERFLOW` to catch lines with extra columns

QUICK SUMMARY

- `UNSTRING … INTO` — split
- `ALL SPACES` — collapse blanks
- `COUNT IN` — piece length
- `TALLYING IN` — field count

INSPECT

COUNTING AND REPLACING CHARACTERS

INSPECT scans a field in place. **TALLYING** counts occurrences, **REPLACING** swaps characters or strings of the same length, and **CONVERTING** maps one set of characters to another. BEFORE and AFTER INITIAL limit the scan to part of the field.

KEY POINTS

- Replacement text must be the same length as the original
- Tally counters are added to, not reset
- `LEADING` only matches at the start of the field
- `CONVERTING` is a character-for-character translation

COUNT, REPLACE, CONVERT

```
       01  WS-TEXT      PIC X(30) VALUE "000123-45-6789  MAIN ST".
       01  WS-NAME      PIC X(20) VALUE "ada lovelace".
       01  WS-COUNT     PIC 99.
       01  WS-ZEROS     PIC 99.

           MOVE 0 TO WS-COUNT WS-ZEROS
           INSPECT WS-TEXT TALLYING WS-COUNT FOR ALL "-"
      *>   WS-COUNT = 2
           INSPECT WS-TEXT TALLYING WS-ZEROS FOR LEADING "0"
      *>   WS-ZEROS = 3
           INSPECT WS-TEXT REPLACING ALL "-" BY " "
      *>   "000123 45 6789  MAIN ST"
           INSPECT WS-TEXT REPLACING LEADING "0" BY " "
      *>   "   123 45 6789  MAIN ST"
           INSPECT WS-TEXT REPLACING ALL " " BY "_"
                   AFTER INITIAL "ST"
           INSPECT WS-NAME CONVERTING
                   "abcdefghijklmnopqrstuvwxyz"
                TO "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
      *>   "ADA LOVELACE"
           MOVE 0 TO WS-COUNT
           INSPECT WS-NAME TALLYING WS-COUNT
                   FOR CHARACTERS BEFORE INITIAL " "
      *>   WS-COUNT = 3 ("ADA")
```

CODE EXPLAINED

- → `FOR ALL "-"` counts every dash
- → `FOR LEADING "0"` stops at the first non-zero
- → `AFTER INITIAL "ST"` only touches bytes after the first `ST`
- → `CHARACTERS BEFORE INITIAL " "` counts the bytes before the first blank

MODERN EQUIVALENT

- ↔ **Python** `s.count("-")`, `s.replace("-", " ")`, `s.upper()`, `s.translate(...)`
- ↔ **Java** `s.chars().filter(c -> c == '-').count()`, `s.replace(...)`, `s.toUpperCase()`
- ↔ **JavaScript** `s.split("-").length - 1`, `s.replaceAll("-", " ")`, `s.toUpperCase()`

PREFER / AVOID

- ⇄ `CONVERTING` for case → `FUNCTION UPPER-CASE` (clearer)
- ⇄ REPLACING with different lengths → `STRING` / `UNSTRING` or a new field

INSPECT FORMS

FORM	DOES
`TALLYING n FOR ALL x`	Count x
`TALLYING n FOR LEADING x`	Count leading x
`TALLYING n FOR CHARACTERS`	Count all bytes
`REPLACING ALL x BY y`	Swap every x
`REPLACING FIRST x BY y`	Swap the first
`CONVERTING abc TO xyz`	Translate chars
`BEFORE / AFTER INITIAL z`	Limit range

⚠ COMMON MISTAKES

- Not zeroing the tally field, so each run adds to the last count
- Replacing `"ST"` with `"STREET"` (lengths must match)
- Expecting LEADING to skip spaces before the zeros

✓ TIPS & BEST PRACTICES

- Count trailing spaces: reverse with `FUNCTION REVERSE`, then tally LEADING spaces
- Use CONVERTING to strip unwanted characters to spaces
- Combine TALLYING and REPLACING in one statement to save a pass

QUICK SUMMARY

- `TALLYING` — count
- `REPLACING` — same-length swap
- `CONVERTING` — translate
- `INITIAL` — limit range

Sheet 24

Reference Modification & String Functions

field(start:length) · FUNCTION UPPER-CASE / LOWER-CASE · FUNCTION REVERSE · FUNCTION LENGTH · FUNCTION TRIM · FUNCTION ORD / CHAR

SUBSTRINGS AND INTRINSIC FUNCTIONS

Reference modification picks bytes by position: `NAME(1:3)` is the first three characters. **Intrinsic functions** return values you can use in `MOVE`, `COMPUTE` and conditions: case conversion, reversal, length and trimming.

KEY POINTS

- Positions start at **1**; length is optional (to the end)
- Start and length can be variables or expressions
- `FUNCTION TRIM` is in GnuCOBOL; check your Enterprise COBOL level
- Out-of-range positions are caught only with `SSRANGE`

SUBSTRINGS AND FUNCTIONS

```
       01  WS-NAME      PIC X(20)  VALUE "  Ada Lovelace".
       01  WS-PHONE     PIC X(10)  VALUE "3195551234".
       01  WS-OUT       PIC X(20).
       01  WS-LEN       PIC 99.
       01  WS-POS       PIC 99     VALUE 4.

           DISPLAY WS-PHONE(1:3)              *> 319
           DISPLAY WS-PHONE(WS-POS:3)         *> 555
           DISPLAY WS-PHONE(7:)               *> 1234
           MOVE "X" TO WS-PHONE(10:1)         *> 319555123X
           MOVE FUNCTION UPPER-CASE(WS-NAME) TO WS-OUT
      *>   "  ADA LOVELACE"
           MOVE FUNCTION TRIM(WS-NAME) TO WS-OUT
      *>   "Ada Lovelace"
           MOVE FUNCTION TRIM(WS-NAME TRAILING) TO WS-OUT
           COMPUTE WS-LEN =
              FUNCTION LENGTH(FUNCTION TRIM(WS-NAME))  *> 12
           MOVE FUNCTION REVERSE(WS-PHONE) TO WS-OUT
           DISPLAY FUNCTION ORD("A")      *> 66 ASCII / 194 EBCDIC
           DISPLAY FUNCTION CHAR(66)      *> A (ASCII)
```

CODE EXPLAINED

- → `(7:)` takes position 7 through the end
- → Reference modification also works on the receiving side of a MOVE
- → `TRIM` removes leading and trailing spaces; `LEADING` / `TRAILING` pick one side
- → `ORD` returns the ordinal position (code + 1), not the raw code

MODERN EQUIVALENT

- ↔ **Python** `s[0:3]`, `s.upper()`, `s.strip()`, `len(s)`, `s[::-1]`, `ord("A")`
- ↔ **Java** `s.substring(0, 3)`, `toUpperCase()`, `strip()`, `length()`, `StringBuilder.reverse()`
- ↔ **JavaScript** `s.slice(0, 3)`, `toUpperCase()`, `trim()`, `length`, `[...s].reverse().join("")`

PREFER / AVOID

- ⇄ magic positions `(14:2)` everywhere → REDEFINES with named sub-fields
- ⇄ loops to find the last non-blank → `FUNCTION TRIM` / `LENGTH`

STRING FUNCTIONS

FUNCTION	RETURNS
`UPPER-CASE(x)`	Uppercase copy
`LOWER-CASE(x)`	Lowercase copy
`TRIM(x [LEADING|TRAILING])`	Spaces removed
`LENGTH(x)`	Length in bytes
`REVERSE(x)`	Reversed text
`ORD(c) / CHAR(n)`	Ordinal ↔ character
`CONCAT(a b …)`	Joined text (2014; check compiler)

REF-MOD FORMS

FORM	MEANS
`x(s:l)`	l bytes from s
`x(s:)`	s to end
`x(i + 1:2)`	Expression start
`tbl(i)(1:3)`	Table entry substring

⚠ COMMON MISTAKES

- Starting at 0 instead of 1
- Length running past the end of the field (no error without `SSRANGE`)
- Assuming `ORD("A")` returns the character code (it is code + 1)

✓ TIPS & BEST PRACTICES

- Compile with `SSRANGE` in test to catch bad positions
- Prefer named REDEFINES fields for fixed layouts
- Check your compiler level before using `TRIM` or `CONCAT`

QUICK SUMMARY

- `x(s:l)` — substring
- `UPPER-CASE` / `LOWER-CASE`
- `TRIM` — strip spaces
- `LENGTH` / `REVERSE`

ADD & SUBTRACT

ENGLISH-STYLE ARITHMETIC

ADD and SUBTRACT change fields in place (TO, FROM) or store the result elsewhere (GIVING). ROUNDED rounds instead of truncating, and ON SIZE ERROR catches results too big for the receiver instead of silently losing digits.

KEY POINTS

- `ADD A TO B` changes B; `ADD A B GIVING C` leaves A and B alone
- Several receivers can be listed
- Without `ROUNDED`, extra decimals are truncated
- With `SIZE ERROR`, the receiver is left unchanged on overflow

ADD AND SUBTRACT FORMS

```
       01  WS-A         PIC 9(3)V99  VALUE 10.25.
       01  WS-B         PIC 9(3)V99  VALUE 5.50.
       01  WS-TOTAL     PIC 9(5)V99  VALUE 0.
       01  WS-SMALL     PIC 9(3)     VALUE 990.
       01  WS-WHOLE     PIC 9(3)     VALUE 0.

           ADD WS-A TO WS-TOTAL                *> 10.25
           ADD WS-A WS-B TO WS-TOTAL           *> 26.00
           ADD WS-A WS-B GIVING WS-TOTAL       *> 15.75
           ADD WS-A TO WS-WHOLE ROUNDED        *> 10 (10.25 rounds down)
           SUBTRACT WS-B FROM WS-TOTAL         *> 10.25
           SUBTRACT WS-B FROM WS-A GIVING WS-TOTAL  *> 4.75
           ADD 20 TO WS-SMALL
              ON SIZE ERROR
                 DISPLAY "OVERFLOW: STILL " WS-SMALL   *> 990
              NOT ON SIZE ERROR
                 DISPLAY "OK"
           END-ADD
           ADD CORRESPONDING MONTH-TOTALS TO YEAR-TOTALS
```

CODE EXPLAINED

- → `TO` adds into the existing value; `GIVING` replaces it
- → `ROUNDED` uses half-up rounding by default
- → 1010 does not fit `PIC 9(3)`, so the size error branch runs
- → `ADD CORRESPONDING` adds fields with matching names

MODERN EQUIVALENT

- ↔ **Python** `total += a`; `total = a + b`; `Decimal.quantize` to round
- ↔ **Java** `total = total.add(a)`; `setScale(2, RoundingMode.HALF_UP)`
- ↔ **JavaScript** `total += a`; for money use integer cents to avoid float drift

PREFER / AVOID

- ⇄ no SIZE ERROR on totals → `ON SIZE ERROR` with a logged message
- ⇄ chains of ADD/SUBTRACT → one `COMPUTE` when it reads better

FORMS

STATEMENT	RESULT IN
`ADD a TO b`	b = b + a
`ADD a b TO c`	c = c + a + b
`ADD a b GIVING c`	c = a + b
`SUBTRACT a FROM b`	b = b − a
`SUBTRACT a FROM b GIVING c`	c = b − a
`ADD CORR g1 TO g2`	Matching fields

⚠ COMMON MISTAKES

- Using `ADD A TO B` when you meant `GIVING`, which overwrites B's original value
- Truncated cents because `ROUNDED` was left off
- Silent overflow on totals with no `SIZE ERROR` check

✓ TIPS & BEST PRACTICES

- Use scope terminators (`END-ADD`) whenever you use SIZE ERROR
- Accumulate totals in fields larger than any single amount
- Keep signed (`S`) receivers for subtraction

QUICK SUMMARY

- `ADD … TO` — in place
- `GIVING` — new receiver
- `ROUNDED` — half-up
- `ON SIZE ERROR` — catch overflow

MULTIPLY & DIVIDE

MULTIPLICATION AND DIVISION

`MULTIPLY A BY B` stores the product in B; `DIVIDE A INTO B` divides B by A and stores it in B. The `GIVING` forms put the result somewhere else, and `REMAINDER` captures what is left over. Division by zero is a size error.

KEY POINTS

- `INTO` divides the second operand by the first
- `BY` (with GIVING) reads left to right: A / B
- `REMAINDER` needs `GIVING`
- Divide by zero without `SIZE ERROR` abends (SOCB)

MULTIPLY AND DIVIDE FORMS

```
01  WS-QTY      PIC 9(3)      VALUE 12.
01  WS-PRICE    PIC 9(3)V99   VALUE 2.50.
01  WS-COST     PIC 9(5)V99.
01  WS-TOTAL    PIC 9(5)V99   VALUE 100.
01  WS-AVG      PIC 9(3)V99.
01  WS-BOXES    PIC 9(3).
01  WS-LEFT     PIC 9(3).
01  WS-ZERO     PIC 9         VALUE 0.

    MULTIPLY WS-QTY BY WS-PRICE GIVING WS-COST    *> 30.00
    MULTIPLY 2 BY WS-QTY                          *> 24
    DIVIDE 3 INTO WS-TOTAL GIVING WS-AVG ROUNDED  *> 33.33
    DIVIDE WS-TOTAL BY 3 GIVING WS-AVG            *> 33.33
    DIVIDE WS-QTY BY 5 GIVING WS-BOXES
           REMAINDER WS-LEFT                      *> 4 r 4
    DIVIDE WS-TOTAL BY WS-ZERO GIVING WS-AVG
       ON SIZE ERROR
          DISPLAY "DIVIDE BY ZERO"
    END-DIVIDE
```

CODE EXPLAINED

- → `MULTIPLY 2 BY WS-QTY` changes WS-QTY in place
- → `DIVIDE 3 INTO WS-TOTAL` = WS-TOTAL / 3
- → 24 / 5 = 4 boxes with 4 left over
- → The size error branch catches the zero divisor

MODERN EQUIVALENT

- ↔ **Python** `cost = qty * price` ; `divmod(24, 5)` → `(4, 4)`
- ↔ **Java** `price.multiply(qty)` ; `24 / 5` and `24 % 5` ; `ArithmeticException` on /0
- ↔ **JavaScript** `qty * price` ; `Math.trunc(24 / 5)` and `24 % 5` ; /0 gives `Infinity`

PREFER / AVOID

- ⇄ mixing INTO and BY → `COMPUTE` for anything non-trivial
- ⇄ unchecked division → `ON SIZE ERROR` or test the divisor first

FORMS

STATEMENT	RESULT
`MULTIPLY a BY b`	b = a × b
`MULTIPLY a BY b GIVING c`	c = a × b
`DIVIDE a INTO b`	b = b / a
`DIVIDE a INTO b GIVING c`	c = b / a
`DIVIDE a BY b GIVING c`	c = a / b
`… REMAINDER r`	r = leftover

⚠ COMMON MISTAKES

- Swapping `INTO` and `BY` and dividing the wrong way
- Writing `MULTIPLY A BY 2` (a literal cannot receive the result)
- Dividing by a field that can be zero with no check (SOCB abend)

✓ TIPS & BEST PRACTICES

- Read `DIVIDE A INTO B` as "A goes into B"
- Use `ROUNDED` on averages and rates
- Use `REMAINDER` for packing, paging and batching counts

QUICK SUMMARY

- `MULTIPLY … BY` — product
- `DIVIDE … INTO` — b / a
- `DIVIDE … BY … GIVING` — a / b
- `REMAINDER` — leftover

COMPUTE

COMPUTE x = … · + − * / ** · precedence & parentheses · ROUNDED MODE (2014) · intermediate precision · ARITH(EXTEND)

FORMULAS IN ONE STATEMENT

COMPUTE evaluates an arithmetic expression and stores the result, like assignment in modern languages. It supports +, −, *, / and ** (power) with normal precedence, plus ROUNDED, ON SIZE ERROR and several receivers.

KEY POINTS

- Spaces are required around operators: `A - B`, not `A-B`
- `A-B` is read as a single name with a hyphen
- Intermediate results keep extra decimal places
- Multiple receivers can each have their own `ROUNDED`

UNDER THE HOOD

- Intermediate precision depends on operand sizes and the `ARITH` compiler option
- `ARITH(COMPAT)` allows 18 digits; `ARITH(EXTEND)` allows 31
- Exponentiation with non-integer powers uses floating point internally

FORMULAS

```
       01  WS-PRIN      PIC 9(7)V99    VALUE 10000.
       01  WS-RATE      PIC V9(4)      VALUE .0525.
       01  WS-YEARS     PIC 99         VALUE 3.
       01  WS-AMT       PIC 9(7)V99.
       01  WS-INT       PIC 9(7)V99.
       01  WS-PCT       PIC 999V9.
       01  WS-A         PIC S99        VALUE 7.
       01  WS-B         PIC S99        VALUE 2.

           COMPUTE WS-AMT ROUNDED =
              WS-PRIN * (1 + WS-RATE) ** WS-YEARS     *> 11659.13
           COMPUTE WS-INT = WS-AMT - WS-PRIN          *> 1659.13
           COMPUTE WS-PCT ROUNDED =
              WS-INT / WS-PRIN * 100                  *> 16.6
           COMPUTE WS-A = WS-A - WS-B * 2             *> 3
           COMPUTE WS-AMT ROUNDED MODE NEAREST-EVEN = 2.125
              ON SIZE ERROR DISPLAY "TOO BIG"
           END-COMPUTE                                *> 2.12
```

CODE EXPLAINED

- → `**` binds tighter than `*`, which binds tighter than `+`
- → Parentheses make the compound-interest formula clear
- → `ROUNDED MODE NEAREST-EVEN` is banker's rounding (2014 standard)
- → `END-COMPUTE` closes the SIZE ERROR scope

MODERN EQUIVALENT

- ↔ **Python** `amt = (prin * (1 + rate) ** years).quantize(Decimal("0.01"))`
- ↔ **Java** `prin.multiply(BigDecimal.ONE.add(rate).pow(years)).setScale(2, HALF_UP)`
- ↔ **JavaScript** `Math.round(prin * (1 + rate) ** years * 100) / 100` (float caveats)

PREFER / AVOID

- ⇄ `COMPUTE A=B-C` with no spaces → `COMPUTE A = B - C`
- ⇄ deeply nested formulas → intermediate named fields

OPERATORS

OP	MEANING
+ −	Add, subtract
* /	Multiply, divide
**	Power
( )	Grouping
unary −	Negate

ROUNDED MODE (2014)

MODE	ROUNDS
NEAREST-AWAY-FROM-ZERO	Half up (default)
NEAREST-EVEN	Banker's
TRUNCATION	Drop digits
AWAY-FROM-ZERO	Always up in size
TOWARD-GREATER	Ceiling

⚠ COMMON MISTAKES

- `COMPUTE X = A-B` becomes "the field named A-B"
- Expecting floating-point behavior from fixed decimals (or vice versa)
- Small receivers silently truncating the high digits

✓ TIPS & BEST PRACTICES

- Put spaces around every operator, always
- Use `ROUNDED` on any money result
- Check whether your compiler supports `ROUNDED MODE` before using it

QUICK SUMMARY

- `COMPUTE x = expr` — formula
- `**` — power
- spaces around `-` — required
- `ROUNDED MODE` — rounding rule

Intrinsic Math Functions

BUILT-IN NUMERIC FUNCTIONS

Intrinsic functions return a value you use inside COMPUTE, MOVE or a condition. COBOL has functions for remainders, rounding to integers, absolute value, square roots, statistics over a list or table, random numbers and even financial values like present value.

KEY POINTS

- Write `FUNCTION name(args)`; or omit `FUNCTION` with `REPOSITORY … ALL INTRINSIC`
- `ALL` passes every table entry: `SUM(TBL(ALL))`
- `MOD` follows the divisor's sign; `REM` follows the dividend's
- `INTEGER` rounds down; `INTEGER-PART` truncates toward zero

MATH FUNCTIONS

```
01  WS-N        PIC S9(5)    VALUE -17.
01  WS-X        PIC S9(5)V99.
01  WS-SCORES.
    05 WS-SCORE PIC 999 OCCURS 5 VALUE 0.
01  WS-SEED     PIC 9(8)     VALUE 12345.

    COMPUTE WS-X = FUNCTION MOD(WS-N, 5)          *> 3
    COMPUTE WS-X = FUNCTION REM(WS-N, 5)          *> -2
    COMPUTE WS-X = FUNCTION ABS(WS-N)             *> 17
    COMPUTE WS-X = FUNCTION INTEGER(-2.5)         *> -3
    COMPUTE WS-X = FUNCTION INTEGER-PART(-2.5)    *> -2
    COMPUTE WS-X = FUNCTION SQRT(2)               *> 1.41
    COMPUTE WS-X = FUNCTION MAX(4, 9, 2)          *> 9
    COMPUTE WS-X = FUNCTION SUM(WS-SCORE(ALL))
    COMPUTE WS-X = FUNCTION MEAN(WS-SCORE(ALL))
    COMPUTE WS-X = FUNCTION RANDOM(WS-SEED)       *> first call seeds
    COMPUTE WS-X = FUNCTION RANDOM * 100          *> 0 to 99.99
    COMPUTE WS-X = FUNCTION PRESENT-VALUE(.05, 100, 100)
```

CODE EXPLAINED

- → MOD(-17, 5) = 3 but REM(-17, 5) = -2
- → INTEGER(-2.5) = -3 (floor); INTEGER-PART(-2.5) = -2
- → `(ALL)` passes the whole table as arguments
- → Seed RANDOM once, then call it with no argument

MODERN EQUIVALENT

- ↔ **Python** `-17 % 5` → 3, `math.fmod(-17, 5)` → -2, `math.floor`, `statistics.mean`
- ↔ **Java** `Math.floorMod(-17, 5)` → 3, `-17 % 5` → -2, `Math.floor`, streams
- ↔ **JavaScript** `((-17 % 5) + 5) % 5` → 3, `-17 % 5` → -2, `Math.floor`, `reduce`

PREFER / AVOID

- ⇄ loops to sum a table → `FUNCTION SUM(TBL(ALL))`
- ⇄ MOD on negative numbers without checking → know MOD vs REM

FUNCTIONS

FUNCTION	RETURNS
`MOD(a, b)`	Modulo (sign of b)
`REM(a, b)`	Remainder (sign of a)
`INTEGER(x)`	Floor
`INTEGER-PART(x)`	Truncate
`ABS(x)`	Absolute value
`SQRT(x)`	Square root
`MAX / MIN(…)`	Largest / smallest
`SUM / MEAN / MEDIAN`	Statistics
`RANDOM[(seed)]`	0 ≤ r < 1
`PRESENT-VALUE / ANNUITY`	Finance

⚠ COMMON MISTAKES

- Using MOD when the business rule needs REM behavior for negatives
- Calling RANDOM with the same seed every time (same "random" number)
- Assuming INTEGER truncates negative numbers

✓ TIPS & BEST PRACTICES

- Use `MOD` for check digits and day-of-cycle math
- Pass tables with `(ALL)` instead of writing loops
- Store function results in fields with enough decimals

QUICK SUMMARY

- `MOD` / `REM` — remainders
- `INTEGER` / `INTEGER-PART`
- `SUM(t(ALL))` — whole table
- `RANDOM` — seed once

IF / ELSE

IF … ELSE … END-IF · scope terminators vs the period · nested IFs · CONTINUE vs NEXT SENTENCE · stray-period bug

MAKING DECISIONS

IF runs statements when a condition is true and ELSE runs others when it is false. Close every IF with END-IF. In old code the **period** ended the IF instead, and a misplaced period is still one of the most common COBOL bugs.

KEY POINTS

- A period ends **every** open IF at once
- `CONTINUE` is a no-op placeholder
- `NEXT SENTENCE` jumps past the next period (avoid)
- Indent nested IFs and match each with its END-IF

IF WITH SCOPE TERMINATORS

```
           IF WS-BALANCE < 0
              DISPLAY "OVERDRAWN"
              ADD 25 TO WS-FEES
           ELSE
              IF WS-BALANCE > 10000
                 SET PREMIUM-CUST TO TRUE
              ELSE
                 CONTINUE
              END-IF
           END-IF

      *>   The stray-period bug:
           IF WS-QTY = 0
              DISPLAY "NO STOCK".
              ADD 1 TO WS-REORDERS
      *>   ADD runs ALWAYS: the period closed the IF

           IF ACTIVE AND WS-BALANCE > 0
              PERFORM 2000-BILL
           END-IF
```

CODE EXPLAINED

- → `END-IF` ends exactly one IF, so nesting is safe
- → `CONTINUE` fills an empty branch
- → The period after "NO STOCK" ends the IF; the ADD is outside it
- → Conditions can mix 88-level names and comparisons

MODERN EQUIVALENT

- ↔ **Python** `if bal < 0: … elif bal > 10000: … else: pass`
- ↔ **Java** `if (bal < 0) { … } else if (bal > 10000) { … }`
- ↔ **JavaScript** `if (bal < 0) { … } else if (bal > 10000) { … }`

PREFER / AVOID

- ⇄ period-terminated IFs → `END-IF` on every IF
- ⇄ `NEXT SENTENCE` → `CONTINUE`

TERMINATORS

ITEM	EFFECT
END-IF	Closes one IF
. (period)	Closes all open statements
CONTINUE	Do nothing, fall through
NEXT SENTENCE	Jump past next period

NESTING TIPS

RULE	WHY
One period per paragraph	No accidental closes
Indent 3 per level	Readable branches
ELSE aligned with IF	Shows pairing
Max ~3 levels	Use EVALUATE beyond that

⚠ COMMON MISTAKES

- A period inside an IF that silently ends it early
- Mixing `NEXT SENTENCE` with `END-IF` (jumps further than expected)
- Deep nesting that should be an EVALUATE

✓ TIPS & BEST PRACTICES

- Search old programs for `.` inside IF blocks when debugging
- Use EVALUATE TRUE for three or more branches
- Put the period on its own line at paragraph end if your shop allows

QUICK SUMMARY

- `IF … END-IF` — always close
- period — closes everything
- `CONTINUE` — empty branch
- avoid `NEXT SENTENCE`

Conditions

relational = / > / NOT < · class tests NUMERIC / ALPHABETIC · sign tests POSITIVE / NEGATIVE / ZERO · condition names · AND / OR / NOT · abbreviated conditions

WRITING TESTS

COBOL conditions compare values (relational), check what kind of data a field holds (class), test the sign of a number (sign), or use level-88 condition names. They combine with AND, OR and NOT, and COBOL allows **abbreviated** forms that leave out a repeated subject.

KEY POINTS

- Relational words and symbols are interchangeable: `GREATER THAN` = `>`
- Precedence: NOT, then AND, then OR
- Alphanumeric comparisons pad the shorter side with spaces
- Use parentheses whenever AND and OR are mixed

KINDS OF CONDITIONS

```
           IF WS-AGE >= 18                        *> relational
           IF WS-AGE IS GREATER THAN OR EQUAL TO 18
           IF WS-NAME NOT = SPACES
           IF IN-QTY IS NUMERIC                   *> class
           IF WS-CODE IS ALPHABETIC               *> letters + space
           IF WS-CODE IS ALPHABETIC-UPPER
           IF WS-BALANCE IS NEGATIVE              *> sign
           IF WS-BALANCE IS ZERO
           IF END-OF-FILE                         *> condition name
           IF (WS-AGE > 64 OR DISABLED)
              AND NOT WS-STATE = "IA"             *> combined

       *>  Abbreviated: subject and operator carried forward
           IF WS-STATE = "IA" OR "MN" OR "WI"
           IF WS-AGE > 18 AND < 65
       *>  means WS-AGE > 18 AND WS-AGE < 65
```

CODE EXPLAINED

- → Class tests check content, not the PIC: use them on input
- → `ALPHABETIC` accepts letters and spaces
- → Sign tests work on any numeric field
- → Abbreviated conditions repeat the last subject (and operator)

MODERN EQUIVALENT

- ↔ **Python** `state in ("IA", "MN", "WI")` ; `18 < age < 65` ; `s.isdigit()`
- ↔ **Java** `Set.of("IA","MN","WI").contains(state)` ; `age > 18 && age < 65`
- ↔ **JavaScript** `["IA","MN","WI"].includes(state)` ; `age > 18 && age < 65`

PREFER / AVOID

- ⇄ long OR chains on literals → an 88 with a VALUE list
- ⇄ `NOT A = B OR C` (confusing) → parentheses or an 88

RELATIONAL

SYMBOL	WORDS
=	EQUAL TO
>	GREATER THAN
<	LESS THAN
>=	GREATER THAN OR EQUAL TO
<=	LESS THAN OR EQUAL TO
NOT =	NOT EQUAL TO

CLASS & SIGN

TEST	TRUE WHEN
NUMERIC	All digits (+sign)
ALPHABETIC	A–Z, a–z, space
ALPHABETIC-UPPER / -LOWER	One case + space
POSITIVE	> 0
NEGATIVE	< 0
ZERO	0

⚠ COMMON MISTAKES

- `IF A NOT = 1 OR 2`, which is always true
- Testing `IS NUMERIC` on a COMP field (meaningless for binary)
- Forgetting that `"ABC" = "ABC "` is true (space padding)

✓ TIPS & BEST PRACTICES

- Rewrite negative OR conditions as positive AND conditions
- Use 88-levels for code lists to keep conditions short
- Add parentheses even when precedence would work

QUICK SUMMARY

- relational — `=`, `>`, `NOT <`
- class — `NUMERIC`, `ALPHABETIC`
- sign — `POSITIVE`, `ZERO`
- abbreviated — `= "A" OR "B"`

EVALUATE

COBOL'S CASE STATEMENT

EVALUATE picks the first matching WHEN and runs its statements: there is no fall-through. EVALUATE TRUE tests a list of conditions; EVALUATE field matches values and ranges; ALSO evaluates two or more subjects at once, like a decision table.

KEY POINTS

- Only the first matching WHEN runs
- Stacked WHENs share one action (an OR)
- `WHEN OTHER` is the default branch
- `ANY` matches anything in an ALSO column

THREE EVALUATE STYLES

```
EVALUATE TRUE
   WHEN WS-SCORE >= 90   MOVE "A" TO WS-GRADE
   WHEN WS-SCORE >= 80   MOVE "B" TO WS-GRADE
   WHEN OTHER            MOVE "F" TO WS-GRADE
END-EVALUATE

EVALUATE WS-MONTH
   WHEN 1 THRU 3         MOVE 1 TO WS-QTR
   WHEN 4 THRU 6         MOVE 2 TO WS-QTR
   WHEN 7
   WHEN 8
   WHEN 9                MOVE 3 TO WS-QTR
   WHEN OTHER            MOVE 4 TO WS-QTR
END-EVALUATE

EVALUATE TRUE  ALSO WS-STATE
   WHEN MEMBER ALSO "IA"   MOVE .05 TO WS-DISC
   WHEN MEMBER ALSO ANY    MOVE .03 TO WS-DISC
   WHEN OTHER              MOVE 0   TO WS-DISC
END-EVALUATE
```

CODE EXPLAINED

→ Order matters: the 90 test must come before the 80 test

→ WHEN 7, 8, 9 stacked act as one branch

→ `ALSO` lines up two subjects: a condition and the state

→ `ANY` makes the second column match every state

MODERN EQUIVALENT

↔ **Python** `match month:` with `case 1|2|3:` ; `if/elif` for conditions

↔ **Java** `switch (month) { case 1, 2, 3 -> 1; … default -> 4; }`

↔ **JavaScript** `switch (true) { case score >= 90: … default: … }`

PREFER / AVOID

⇄ nested IF ladders → EVALUATE TRUE

⇄ no WHEN OTHER → always add one, even if it just logs

FORMS

FORM	MATCHES
EVALUATE TRUE	First true condition
EVALUATE x	Value of x
WHEN a THRU b	Range
WHEN a / WHEN b	Either value
EVALUATE x ALSO y	Two subjects
WHEN ANY	Wildcard
WHEN OTHER	Default

⚠ COMMON MISTAKES

- Putting a broad WHEN before a narrow one so the narrow one never runs
- Expecting C-style fall-through between WHENs
- Leaving out WHEN OTHER and silently skipping bad codes

✓ TIPS & BEST PRACTICES

- Use EVALUATE TRUE with 88-level names for readable rules
- Use ALSO to code a business decision table directly
- Keep each WHEN to one PERFORM when actions are long

QUICK SUMMARY

- `EVALUATE TRUE` — condition list
- `WHEN a THRU b` — ranges
- `ALSO` — decision tables
- `WHEN OTHER` — default

PERFORM

CALLING PARAGRAPHS

PERFORM runs a paragraph (or section) and comes back to the next statement, like calling a function with no parameters. Programs are built **top-down**: a main paragraph performs initialization, processing and termination paragraphs, which perform smaller ones.

KEY POINTS

- Paragraphs share all WORKING-STORAGE (no parameters, no locals)
- `THRU` runs a range of paragraphs in source order
- Falling into the next paragraph happens without PERFORM
- Recursion is not allowed unless the program is `RECURSIVE`

TOP-DOWN STRUCTURE

```
       PROCEDURE DIVISION.
       0000-MAIN.
           PERFORM 1000-INIT
           PERFORM 2000-PROCESS UNTIL END-OF-FILE
           PERFORM 9000-TERM
           STOP RUN.

       1000-INIT.
           OPEN INPUT IN-FILE OUTPUT OUT-FILE
           PERFORM 8000-READ.

       2000-PROCESS.
           PERFORM 2100-VALIDATE
           IF VALID-REC
              PERFORM 2200-WRITE
           END-IF
           PERFORM 8000-READ.

       8000-READ.
           READ IN-FILE AT END SET END-OF-FILE TO TRUE
           END-READ.

       9000-TERM.
           PERFORM 3 TIMES
              DISPLAY "*** END ***"
           END-PERFORM
           CLOSE IN-FILE OUT-FILE.
```

CODE EXPLAINED

- → `0000-MAIN` reads like an outline of the whole program
- → The priming read in `1000-INIT` comes before the loop
- → Each process pass ends by reading the next record
- → `STOP RUN` keeps control from falling into `1000-INIT`

MODERN EQUIVALENT

- ↔ **Python** Functions `init()`, `process()`, `term()` called from `main()`
- ↔ **Java** Private methods called from `main`, sharing fields
- ↔ **JavaScript** Functions called from `main()` sharing module-level state

PREFER / AVOID

- ⇄ `PERFORM A THRU A-EXIT` with GO TOs → plain PERFORM of small paragraphs
- ⇄ falling through paragraphs → `STOP RUN` / `GOBACK` at the end of main

PERFORM FORMS

FORM	RUNS
`PERFORM para`	Once
`PERFORM a THRU b`	a to b in order
`PERFORM para n TIMES`	n times
`PERFORM para UNTIL c`	Loop (sheet 33)
`PERFORM … END-PERFORM`	Inline block
`PERFORM section`	All paragraphs in it

PARAGRAPH NUMBERING

RANGE	TYPICAL USE
`0000`	Main control
`1000`	Initialization
`2000-7000`	Processing
`8000`	I/O routines
`9000`	Termination / errors

⚠ COMMON MISTAKES

- Missing `STOP RUN` in main, so execution falls into the next paragraph
- Performing a paragraph that itself performs back into its caller
- Using `THRU` and later inserting a paragraph inside the range

✓ TIPS & BEST PRACTICES

- Keep paragraphs short: one job each
- Number paragraphs in the order they appear
- Use the read-at-end pattern: priming read, then read last in the loop

QUICK SUMMARY

- `PERFORM para` — call and return
- `THRU` — range of paragraphs
- `n TIMES` — repeat
- top-down — main calls the rest

PERFORM Loops

LOOPING WITH PERFORM

COBOL loops are all PERFORM: UNTIL repeats while a condition is false, VARYING adds a counter like a `for` loop, and AFTER nests counters. Inline loops put the body between PERFORM and END-PERFORM instead of in a separate paragraph.

KEY POINTS

- `UNTIL` tests **before** each pass by default (like while)
- `WITH TEST AFTER` runs the body at least once (like do-while)
- The loop ends when the condition becomes **true**
- VARYING increments after each pass, then retests

LOOP FORMS

```
01  WS-I        PIC 99.
01  WS-R        PIC 99.
01  WS-C        PIC 99.
01  WS-SUM      PIC 9(5) VALUE 0.

    PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 10
       ADD WS-I TO WS-SUM
    END-PERFORM                    *> WS-SUM 55, WS-I 11

    PERFORM 2000-READ-AND-SUM
       UNTIL END-OF-FILE

    PERFORM WITH TEST AFTER
            UNTIL WS-ANSWER = "Y" OR "N"
       DISPLAY "CONTINUE? (Y/N)"
       ACCEPT WS-ANSWER
    END-PERFORM

    PERFORM VARYING WS-R FROM 1 BY 1 UNTIL WS-R > 3
              AFTER WS-C FROM 1 BY 1 UNTIL WS-C > 4
       MOVE ZERO TO GRID-CELL(WS-R, WS-C)
    END-PERFORM

    PERFORM VARYING WS-I FROM 10 BY -2 UNTIL WS-I < 1
       DISPLAY WS-I                *> 10 8 6 4 2
    END-PERFORM
```

CODE EXPLAINED

- → After the loop, WS-I is 11: one past the last value
- → `AFTER` is the inner loop: WS-C runs 1–4 for each WS-R
- → `BY -2` counts down
- → The test-after loop always asks at least once

MODERN EQUIVALENT

- ↔ **Python** `for i in range(1, 11):` ; `while not eof:` ; nested `for` loops
- ↔ **Java** `for (int i = 1; i <= 10; i++)` ; `do { } while (...)`
- ↔ **JavaScript** `for (let i = 1; i <= 10; i++)` ; `do { } while (...)`

PREFER / AVOID

- ⇄ counter `PIC 9` with `UNTIL > 9` → `PIC 99` (9 + 1 wraps to 0: infinite loop)
- ⇄ out-of-line paragraphs for tiny bodies → inline PERFORM

LOOP FORMS

COBOL	LIKE
`PERFORM UNTIL c`	while (!c)
`WITH TEST AFTER UNTIL c`	do … while (!c)
`VARYING i FROM 1 BY 1 UNTIL i > n`	for i = 1..n
`… AFTER j …`	Nested for
`PERFORM n TIMES`	repeat n
`EXIT PERFORM`	break (2002+)
`EXIT PERFORM CYCLE`	continue (2002+)

⚠ COMMON MISTAKES

- Counter too small for the limit + 1, so it wraps and loops forever
- Forgetting to read the next record inside an UNTIL END-OF-FILE loop
- Changing the VARYING counter inside the body

✓ TIPS & BEST PRACTICES

- Size counters one digit bigger than the limit needs
- Use `COMP` counters for speed in tight loops
- Use `EXIT PERFORM` instead of GO TO to leave a loop early

QUICK SUMMARY

- `UNTIL` — while not
- `WITH TEST AFTER` — do-while
- `VARYING … AFTER` — nested for
- inline — `PERFORM … END-PERFORM`

Sheet 34

GO TO, EXIT & Ending a Program

GO TO · GO TO … DEPENDING ON · EXIT PARAGRAPH / EXIT PERFORM · STOP RUN vs GOBACK · RETURN-CODE · structured style

JUMPS AND EXITS

GO TO jumps to a paragraph and never comes back, which makes old programs hard to follow. COBOL 2002 added EXIT PARAGRAPH and EXIT PERFORM for structured exits. Programs end with STOP RUN (whole run unit) or GOBACK (return to the caller), setting RETURN-CODE for the job step.

KEY POINTS

- `GOBACK` works in main programs and subprograms
- `RETURN-CODE` becomes the job step's condition code
- `GO TO … DEPENDING ON` jumps by index (1, 2, 3…)
- Many shops allow `GO TO` only to an exit paragraph

OLD STYLE VS STRUCTURED

```
      *>   Legacy: PERFORM THRU with GO TO exit
           PERFORM 2000-EDIT THRU 2000-EXIT.
       2000-EDIT.
           IF IN-QTY NOT NUMERIC
              MOVE "BAD QTY" TO ERR-MSG
              GO TO 2000-EXIT
           END-IF
           MOVE IN-QTY TO WS-QTY.
       2000-EXIT.
           EXIT.

      *>   Structured (2002+)
       2100-EDIT.
           IF IN-QTY NOT NUMERIC
              MOVE "BAD QTY" TO ERR-MSG
              EXIT PARAGRAPH
           END-IF
           MOVE IN-QTY TO WS-QTY.

       9000-TERM.
           IF WS-ERRORS > 0
              MOVE 8 TO RETURN-CODE
           END-IF
           GOBACK.
```

CODE EXPLAINED

- → `EXIT.` alone is a no-op paragraph used as a GO TO target
- → `EXIT PARAGRAPH` replaces the GO TO-exit pattern
- → RC 8 tells the JCL that this step had errors
- → `GOBACK` is safe whether or not the program was called

MODERN EQUIVALENT

- ↔ **Python** `return` from a function; `sys.exit(8)`
- ↔ **Java** `return;` / `break;` ; `System.exit(8)`
- ↔ **JavaScript** `return;` / `break;` ; `process.exitCode = 8`

PREFER / AVOID

- ⇄ `GO TO` between paragraphs → `PERFORM` + `EXIT PARAGRAPH`
- ⇄ `STOP RUN` everywhere → `GOBACK` (works when called too)

ENDING & EXITING

STATEMENT	EFFECT
STOP RUN	End the run unit
GOBACK	Return to caller / end
EXIT PROGRAM	Return from subprogram
EXIT PARAGRAPH	Leave the paragraph
EXIT PERFORM	Leave inline loop
GO TO p	Jump, no return

RETURN CODES

RC	CONVENTION
0	Success
4	Warning
8	Error
12	Severe error
16	Fatal

⚠ COMMON MISTAKES

- `GO TO` out of a PERFORMed paragraph, leaving the PERFORM unfinished
- `STOP RUN` in a subprogram, which ends the calling program too
- Never setting `RETURN-CODE`, so failed runs look successful

✓ TIPS & BEST PRACTICES

- Set RETURN-CODE so the JCL can skip later steps on errors
- Refactor GO TO only with good tests: legacy logic can depend on it
- End every program with GOBACK

QUICK SUMMARY

- `GO TO` — jump, avoid
- `EXIT PARAGRAPH` — structured exit
- `GOBACK` — return / end
- `RETURN-CODE` — step condition code

OCCURS Basics

TABLES ARE COBOL'S ARRAYS

The OCCURS clause repeats a field or group a fixed number of times, making a **table**. You reach an entry with a **subscript** in parentheses, starting at **1**. Nest OCCURS for two or three dimensions, and repeat whole groups to build tables of records.

KEY POINTS

- OCCURS cannot be used on an `01` or `77` level
- Subscripts can be literals, fields or `field + n`
- Size is fixed at compile time (see ODO, sheet 38)
- No bounds checking unless compiled with `SSRANGE`

UNDER THE HOOD

- Entry address = table start + (subscript − 1) × entry length
- Out-of-range subscripts read or overwrite neighboring storage
- `SSRANGE` adds a check on every reference; with `-debug` GnuCOBOL does the same

DEFINING AND USING TABLES

```
01  MONTH-TABLE.
    05 MONTH-SALES     PIC 9(7)V99 OCCURS 12 TIMES.
01  REGION-TABLE.
    05 REGION OCCURS 4 TIMES.
       10 REGION-NAME  PIC X(10).
       10 REGION-QTR   PIC 9(7)V99 OCCURS 4 TIMES.
01  WS-SUB             PIC 99 COMP.
01  WS-R               PIC 9  COMP.
01  WS-Q               PIC 9  COMP.
01  WS-YEAR            PIC 9(9)V99 VALUE 0.

    MOVE 1500.00 TO MONTH-SALES(1)
    MOVE 2 TO WS-SUB
    ADD 250 TO MONTH-SALES(WS-SUB)
    DISPLAY MONTH-SALES(WS-SUB + 1)
    MOVE "MIDWEST" TO REGION-NAME(3)
    MOVE 99.95 TO REGION-QTR(3, 2)     *> region 3, Q2
    PERFORM VARYING WS-SUB FROM 1 BY 1 UNTIL WS-SUB > 12
       ADD MONTH-SALES(WS-SUB) TO WS-YEAR
    END-PERFORM
    INITIALIZE REGION-TABLE
```

CODE EXPLAINED

- → `MONTH-SALES(1)` is the first entry, not `(0)`
- → Each REGION entry holds a name and 4 quarter amounts
- → `(3, 2)` lists outer subscript first, then inner
- → COMP subscripts are faster than DISPLAY ones

MODERN EQUIVALENT

- ↔ **Python** `sales = [Decimal(0)] * 12` ; `sales[0]` is the first
- ↔ **Java** `BigDecimal[] sales = new BigDecimal[12]` ; `sales[0]`
- ↔ **JavaScript** `const sales = new Array(12).fill(0)` ; `sales[0]`

PREFER / AVOID

- ⇄ off-by-one from 0-based habits → loops `FROM 1 … UNTIL > n`
- ⇄ testing without SSRANGE → compile tests with `SSRANGE`

TABLE FORMS

FORM	EXAMPLE
Simple	05 X PIC 9 OCCURS 10
Group	05 ENT OCCURS 5. 10 …
2-D	OCCURS inside OCCURS
Max levels	7 nested (standard)
Reference	X(i), X(i, j)
Relative	X(i + 1), X(i − 1)

⚠ COMMON MISTAKES

- Subscript 0, which points before the table and corrupts memory
- Using OCCURS on an 01 level (compile error)
- Loop limit higher than the OCCURS count

✓ TIPS & BEST PRACTICES

- Turn on SSRANGE in test builds, then measure before using it in production
- Use COMP for subscripts and counters
- Use `(ALL)` with SUM/MAX instead of loops when you only need a total

QUICK SUMMARY

- `OCCURS n` — fixed-size table
- `X(1)` — first entry
- `X(i, j)` — 2-D
- `SSRANGE` — bounds checks

Indexes

FASTER TABLE ACCESS

An **index** declared with INDEXED BY is tied to one table and holds a byte offset instead of an occurrence number, so table access needs no multiplication. Indexes are required by SEARCH, and you change them only with SET, SEARCH or PERFORM VARYING.

KEY POINTS

- You still write `TBL(IX)`, as with subscripts
- Index names are not declared elsewhere: `INDEXED BY` defines them
- `USAGE INDEX` fields store index values for later
- Mixing: `TBL(IX + 1)` is allowed (relative indexing)

UNDER THE HOOD

- Index value = (occurrence − 1) × entry length: occurrence 3 of 20-byte entries is 40
- Subscripts need a multiply on every access; indexes do not
- Each index is tied to its own table: using it on another table gives wrong offsets

INDEXES VS SUBSCRIPTS

```
       01  RATE-TABLE.
           05 RATE-ENTRY OCCURS 20 TIMES INDEXED BY RX.
              10 RATE-CODE     PIC XX.
              10 RATE-PCT      PIC V999.
       01  WS-SAVE-IX          USAGE INDEX.
       01  WS-COUNT            PIC 99 VALUE 0.

           SET RX TO 1
           MOVE "A1" TO RATE-CODE(RX)
           SET RX UP BY 1                    *> occurrence 2
           MOVE .045 TO RATE-PCT(RX)
           SET WS-SAVE-IX TO RX              *> remember position
           PERFORM VARYING RX FROM 1 BY 1 UNTIL RX > 20
              IF RATE-CODE(RX) NOT = SPACES
                 ADD 1 TO WS-COUNT
              END-IF
           END-PERFORM
           SET RX TO WS-SAVE-IX              *> back to entry 2
           DISPLAY RATE-CODE(RX + 1)         *> relative index
```

CODE EXPLAINED

- → `SET RX TO 1` points at the first occurrence
- → `SET … UP BY` moves the index by whole entries
- → `USAGE INDEX` saves and restores an index position
- → PERFORM VARYING works with indexes too

MODERN EQUIVALENT

- ↔ **Python** A position variable; Python lists index directly by number
- ↔ **Java** `int i` positions; JIT handles the offset math
- ↔ **JavaScript** `let i` positions; typed arrays for fixed layouts

PREFER / AVOID

- ⇄ subscripts in SEARCH loops → `INDEXED BY` (required by SEARCH)
- ⇄ `MOVE` to an index → `SET`

INDEX VS SUBSCRIPT

	INDEX	SUBSCRIPT
Declared by	INDEXED BY	Normal field
Holds	Byte offset	Occurrence #
Changed by	SET, SEARCH, VARYING	MOVE, ADD, …
Speed	Faster	Slower (unless COMP)
SEARCH	Required	Not allowed
Display	Not directly	Yes

⚠ COMMON MISTAKES

- `MOVE 1 TO RX` (indexes need SET)
- Displaying an index directly to debug (shows an offset or is not allowed)
- Using one table's index on another table with a different entry size

✓ TIPS & BEST PRACTICES

- Use indexes for tables you search
- To show the position, `SET` a numeric field to the index first
- Give each table its own index name

QUICK SUMMARY

- `INDEXED BY` — define index
- `SET ix TO / UP BY` — change it
- `USAGE INDEX` — save positions
- required by `SEARCH`

SEARCH & SEARCH ALL

serial SEARCH · AT END · WHEN · SEARCH ALL binary search · ASCENDING KEY / DESCENDING KEY · table must be sorted

LOOKING UP TABLE ENTRIES

SEARCH scans a table one entry at a time from the current index until a WHEN condition is true. SEARCH ALL does a **binary search** on a table declared with ASCENDING or DESCENDING KEY, which is much faster for large tables but requires the data to be in key order.

KEY POINTS

- Set the index to 1 before a serial SEARCH
- SEARCH ALL sets its own starting point
- SEARCH ALL allows only `=` tests joined by AND
- `AT END` runs when nothing matches

UNDER THE HOOD

- SEARCH ALL halves the range each step: about 10 compares for 1,000 entries
- Unsorted data does not fail loudly: SEARCH ALL just misses entries
- Unused trailing entries should hold HIGH-VALUES so the order holds

SERIAL AND BINARY SEARCH

```
01  STATE-TABLE.
    05 STATE-ENT OCCURS 50 TIMES
          ASCENDING KEY IS ST-CODE
          INDEXED BY SX.
       10 ST-CODE      PIC XX.
       10 ST-NAME      PIC X(14).
       10 ST-TAX       PIC V9999.

*>  Serial: start where you want, any condition
    SET SX TO 1
    SEARCH STATE-ENT
       AT END DISPLAY "NOT FOUND"
       WHEN ST-NAME(SX) = "IOWA"
          DISPLAY "CODE " ST-CODE(SX)       *> IA
    END-SEARCH

*>  Binary: table must be in ST-CODE order
    SEARCH ALL STATE-ENT
       AT END MOVE 0 TO WS-TAX
       WHEN ST-CODE(SX) = WS-STATE
          MOVE ST-TAX(SX) TO WS-TAX
    END-SEARCH
```

CODE EXPLAINED

- → Serial SEARCH can test any field with any condition
- → SEARCH ALL tests only the declared key with `=`
- → After a match, SX points at the found entry
- → `AT END` handles a missing code

MODERN EQUIVALENT

- ↔ **Python** `next((s for s in states if s.name == "IOWA"), None)`; `bisect` for sorted
- ↔ **Java** `list.stream().filter(...).findFirst()`; `Collections.binarySearch`
- ↔ **JavaScript** `states.find(s => s.name === "IOWA")`; a `Map` for fast lookups

PREFER / AVOID

- ⇄ serial SEARCH on large tables → SEARCH ALL on a sorted table
- ⇄ unsorted load into a keyed table → sort first or check order on load

SEARCH VS SEARCH ALL

	SEARCH	SEARCH ALL
Method	Serial	Binary
Start	Current index	Automatic
Conditions	Any	= on keys, AND
Order needed	No	Yes (KEY)
Speed (1,000)	Up to 1,000 compares	About 10

⚠ COMMON MISTAKES

- Forgetting `SET SX TO 1`, so SEARCH starts mid-table
- SEARCH ALL on unsorted data: it returns "not found" for entries that exist
- Using `>` or `OR` inside SEARCH ALL (not allowed)

✓ TIPS & BEST PRACTICES

- Fill unused entries with HIGH-VALUES before a SEARCH ALL
- Load reference tables from sorted files at startup
- Use a serial SEARCH when you need the first match by a non-key field

QUICK SUMMARY

- `SEARCH` — serial, any test
- `SEARCH ALL` — binary, key `=`
- `ASCENDING KEY` — needed for ALL
- `AT END` — not found

Sheet 38

Variable-Length Tables & Loading

OCCURS … DEPENDING ON · ODO object rules · loading a table from a file · hard-coded tables via REDEFINES · table SORT

TABLES WHOSE SIZE CHANGES

`OCCURS 1 TO n TIMES DEPENDING ON count` (ODO) declares a table whose **logical** size is held in a counter field. Storage is still reserved for the maximum. Reference tables are usually loaded from a file at startup, or hard-coded with a VALUE block redefined as a table.

KEY POINTS

- The ODO object (counter) must be set before using the table
- Group length and SEARCH use the current count
- Hard-coded tables: FILLER VALUEs, then REDEFINES … OCCURS
- Table `SORT` is COBOL 2014; check your compiler

ODO, FILE LOAD AND CONSTANT TABLE

```
01  WS-PROD-COUNT       PIC 9(4) COMP VALUE 0.
01  PROD-TABLE.
    05 PROD-ENT OCCURS 1 TO 500 TIMES
          DEPENDING ON WS-PROD-COUNT
          ASCENDING KEY IS PT-CODE INDEXED BY PX.
       10 PT-CODE        PIC X(6).
       10 PT-PRICE       PIC 9(5)V99.
01  DAY-NAMES-DATA.
    05 FILLER PIC X(9) VALUE "MONDAY".
    05 FILLER PIC X(9) VALUE "TUESDAY".
    05 FILLER PIC X(9) VALUE "WEDNESDAY".
01  DAY-NAMES REDEFINES DAY-NAMES-DATA.
    05 DAY-NAME PIC X(9) OCCURS 3 TIMES.

1000-LOAD-PRODUCTS.
    PERFORM UNTIL END-OF-PROD OR WS-PROD-COUNT = 500
       READ PROD-FILE
          AT END SET END-OF-PROD TO TRUE
          NOT AT END
             ADD 1 TO WS-PROD-COUNT
             MOVE PF-CODE  TO PT-CODE(WS-PROD-COUNT)
             MOVE PF-PRICE TO PT-PRICE(WS-PROD-COUNT)
       END-READ
    END-PERFORM.
```

CODE EXPLAINED

- → The counter grows as each record is loaded
- → The load stops at 500 so the table cannot overflow
- → SEARCH ALL on PROD-ENT searches only the loaded entries
- → `DAY-NAME(2)` is "TUESDAY" from the constant block

MODERN EQUIVALENT

- ↔ **Python** `products = []` then `products.append(...)`; `len(products)`
- ↔ **Java** `List<Product> products = new ArrayList<>()`; `size()`
- ↔ **JavaScript** `const products = []`; `products.push(...)`; `length`

PREFER / AVOID

- ⇄ loading without a limit check → stop at the OCCURS maximum and report
- ⇄ big hard-coded tables → a reference file loaded at startup

ODO RULES

RULE	DETAIL
Syntax	OCCURS min TO max DEPENDING ON n
Storage	Always the maximum
Counter	Set before use; 0 ≤ n ≤ max
Placement	Last item in the record (usually)
Moves	Group length uses current n

WAYS TO FILL A TABLE

METHOD	USE FOR
Read a file at start	Prices, rates, codes
VALUE + REDEFINES	Small constant lists
DB2 cursor FETCH	Tables from a database
INITIALIZE	Reset to zeros/spaces

⚠ COMMON MISTAKES

- Using ODO entries before setting the counter
- No limit check during the load, writing past the table end
- Reading a file into a keyed table that is not in key order, then using SEARCH ALL

✓ TIPS & BEST PRACTICES

- Log how many entries were loaded at startup
- Abend clearly (RC 16) if the reference file is bigger than the table
- Keep the max OCCURS in one copybook constant

QUICK SUMMARY

- ODO — logical size in a counter
- load loop — stop at max
- VALUE + REDEFINES — constant table
- SEARCH ALL — searches current count

File Concepts

HOW COBOL SEES FILES

COBOL processes files one **record** at a time: a fixed layout of fields described in the program. The program names a logical file; at run time a **DD name** (z/OS) or environment variable (GnuCOBOL) connects it to a real dataset. Organization decides how records can be reached.

KEY POINTS

- **Sequential**: read in order, append at the end
- **Indexed** (VSAM KSDS): read by key or in key order
- **Relative** (RRDS): read by record number
- Record formats: fixed (F/FB) or variable (V/VB)

THE FILE PATH THROUGH A PROGRAM

```
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT CUST-FILE ASSIGN TO CUSTIN      *> DD name
               ORGANIZATION IS SEQUENTIAL
               FILE STATUS IS WS-CUST-STATUS.
       DATA DIVISION.
       FILE SECTION.
       FD  CUST-FILE
           RECORDING MODE IS F.
       01  CUST-REC              PIC X(100).
       WORKING-STORAGE SECTION.
       01  WS-CUST-STATUS        PIC XX.
       COPY CUSTREC.                          *> 100-byte layout
       PROCEDURE DIVISION.
           OPEN INPUT CUST-FILE
           READ CUST-FILE INTO CUSTOMER-LAYOUT
           CLOSE CUST-FILE

      *>   z/OS JCL:   //CUSTIN DD DSN=PROD.CUSTOMER,DISP=SHR
      *>   GnuCOBOL:   export DD_CUSTIN=customer.dat
```

CODE EXPLAINED

- → `SELECT` names the file; `ASSIGN TO CUSTIN` names its DD
- → `FD` describes the record buffer
- → `READ … INTO` copies the record into a detailed layout
- → Without the DD in JCL, OPEN fails with status 35

MODERN EQUIVALENT

- ↔ **Python** `open("customer.dat")` ; path comes from config or `sys.argv`
- ↔ **Java** `Files.newBufferedReader(Path.of(...))`
- ↔ **JavaScript** `fs.createReadStream(path)` or `fs.readFileSync(path)`

PREFER / AVOID

- ⇄ hard-coded file paths → DD names (the JCL picks the dataset)
- ⇄ one FD per layout → one FD + `READ … INTO` copybook layouts

ORGANIZATIONS

TYPE	ACCESS
SEQUENTIAL	In order only
LINE SEQUENTIAL	Text lines (GnuCOBOL / PC)
INDEXED	By key or in key order
RELATIVE	By record number

RECORD FORMATS

RECFM	MEANING
F / FB	Fixed, blocked
V / VB	Variable, 4-byte RDW
U	Undefined (load modules)
FBA / VBA	With ASA print control

⚠ COMMON MISTAKES

- Assuming files are text lines: mainframe records have no newline characters
- FD record length not matching the dataset LRECL (status 39)
- Forgetting the DD statement in the JCL (status 35)

✓ TIPS & BEST PRACTICES

- Keep record layouts in copybooks shared by every program
- Always declare `FILE STATUS` for every file
- Use LINE SEQUENTIAL for text files when testing on GnuCOBOL

QUICK SUMMARY

- record — fixed layout
- DD name — runtime link
- SEQUENTIAL / INDEXED / RELATIVE
- FB / VB — record formats

SELECT & ASSIGN

FILE-CONTROL · SELECT … ASSIGN TO · ORGANIZATION IS · ACCESS MODE IS · FILE STATUS IS · GnuCOBOL vs z/OS assignment

DECLARING FILES

Each file gets a SELECT entry in the FILE-CONTROL paragraph. It gives the internal file name, the external name it is **assigned** to, its organization, how it will be accessed, and the two-byte **FILE STATUS** field that receives a result code after every I/O.

KEY POINTS

- On z/OS, the ASSIGN name is the DD name (max 8 characters)
- GnuCOBOL maps ASSIGN names to env vars `DD_name` or file names
- `ACCESS MODE` is SEQUENTIAL, RANDOM or DYNAMIC
- One SELECT per file, even input and output of the same layout

SELECT EXAMPLES

```
       FILE-CONTROL.
           SELECT TRAN-IN  ASSIGN TO TRANIN
               ORGANIZATION IS SEQUENTIAL
               ACCESS MODE  IS SEQUENTIAL
               FILE STATUS  IS WS-TRAN-FS.

           SELECT ACCT-MAST ASSIGN TO ACCTMAST
               ORGANIZATION IS INDEXED
               ACCESS MODE  IS DYNAMIC
               RECORD KEY   IS AM-ACCT-NO
               FILE STATUS  IS WS-ACCT-FS.

           SELECT RPT-OUT  ASSIGN TO RPTOUT
               FILE STATUS  IS WS-RPT-FS.

      *>   GnuCOBOL only: text file named by literal
           SELECT CSV-IN   ASSIGN TO "data/input.csv"
               ORGANIZATION IS LINE SEQUENTIAL
               FILE STATUS  IS WS-CSV-FS.
```

CODE EXPLAINED

→ SEQUENTIAL is the default organization and access mode

→ Indexed files need `RECORD KEY` (sheet 44)

→ DYNAMIC allows both random and sequential reads

→ A literal path is handy on GnuCOBOL; production uses DD names

MODERN EQUIVALENT

↔ **Python** `open(os.environ["TRANIN"])` style config-driven paths

↔ **Java** `Path.of(System.getenv("TRANIN"))`

↔ **JavaScript** `fs.createReadStream(process.env.TRANIN)`

PREFER / AVOID

⇄ no FILE STATUS → a status field on every SELECT

⇄ literal paths in production code → DD names / env vars

SELECT CLAUSES

CLAUSE	VALUES
`ASSIGN TO`	DD name or literal
`ORGANIZATION`	SEQUENTIAL, LINE SEQUENTIAL, INDEXED, RELATIVE
`ACCESS MODE`	SEQUENTIAL, RANDOM, DYNAMIC
`RECORD KEY`	Indexed primary key
`ALTERNATE RECORD KEY`	Secondary keys
`RELATIVE KEY`	Record number field
`FILE STATUS`	PIC XX result code

NAME MAPPING

ENVIRONMENT	ASSIGN TO TRANIN
`z/OS batch`	//TRANIN DD DSN=…
`GnuCOBOL`	env DD_TRANIN or dd_TRANIN, then TRANIN
`Micro Focus`	env var dd_TRANIN
`CICS`	Files defined in CICS, not SELECT

⚠ COMMON MISTAKES

- ASSIGN names longer than 8 characters on z/OS
- Same DD name used for two SELECTs
- Indexed file declared without `RECORD KEY`

✓ TIPS & BEST PRACTICES

- Name DD names after the file's role: `ACCTMAST`, `TRANIN`, `RPTOUT`
- Use one FILE STATUS field per file, not a shared one
- On GnuCOBOL, set `DD_name` variables in a run script

QUICK SUMMARY

- `SELECT … ASSIGN TO dd`
- `ORGANIZATION` — file type
- `ACCESS MODE` — how you read
- `FILE STATUS` — result code

FD & Record Definitions

DESCRIBING THE RECORD BUFFER

The **FD** (file description) in the FILE SECTION describes a file's records. The `01` levels under it are the I/O buffer: `READ` fills it and `WRITE` sends it. Several 01s under one FD redefine the same buffer, which is how files with multiple record types are described.

KEY POINTS

- `BLOCK CONTAINS 0` lets z/OS pick the best block size
- `RECORDING MODE V` for variable-length records
- Multiple 01s under an FD share storage (implicit REDEFINES)
- Buffer contents are undefined after end-of-file or CLOSE

UNDER THE HOOD

- RECFM=FB, LRECL=100 in JCL must match a 100-byte 01 record
- Variable records carry a 4-byte RDW the program never sees
- The FD buffer lives in system I/O areas: copy data out with `READ … INTO`

FD FOR FIXED AND VARIABLE FILES

```
       FILE SECTION.
       FD  CUST-FILE
           RECORDING MODE IS F
           BLOCK CONTAINS 0 RECORDS
           RECORD CONTAINS 100 CHARACTERS.
       01  CUST-REC              PIC X(100).

       FD  TRAN-FILE
           RECORDING MODE IS V
           RECORD IS VARYING IN SIZE FROM 20 TO 200
              DEPENDING ON WS-TRAN-LEN.
       01  TRAN-HEADER.
           05 TH-TYPE            PIC X.
           05 TH-BATCH           PIC 9(6).
           05 FILLER             PIC X(13).
       01  TRAN-DETAIL.
           05 TD-TYPE            PIC X.
           05 TD-ACCT            PIC 9(10).
           05 TD-AMT             PIC S9(7)V99 COMP-3.
           05 TD-MEMO            PIC X(184).

       WORKING-STORAGE SECTION.
       01  WS-TRAN-LEN           PIC 9(4) COMP.
```

CODE EXPLAINED

→ The JCL dataset for CUST-FILE should be RECFM=FB, LRECL=100

→ TRAN-HEADER and TRAN-DETAIL overlay the same buffer

→ On READ, `WS-TRAN-LEN` gets the actual record length

→ Check the type byte before using either layout

MODERN EQUIVALENT

↔ **Python** `struct.Struct("6s30s…").unpack(record)` for fixed layouts

↔ **Java** `ByteBuffer.wrap(record)` and `get` by offset

↔ **JavaScript** `Buffer` slices: `rec.subarray(0, 6).toString()`

PREFER / AVOID

⇄ detailed layouts inside the FD → `PIC X(n)` FD + `READ … INTO` a copybook

⇄ hard-coded `BLOCK CONTAINS 27920` → `BLOCK CONTAINS 0`

FD CLAUSES

CLAUSE	PURPOSE
`RECORDING MODE F / V / U`	Record format
`BLOCK CONTAINS 0`	System block size
`RECORD CONTAINS n`	Fixed length
`RECORD VARYING … DEPENDING ON`	Variable length
`LABEL RECORDS`	Obsolete; ignored

MATCH JCL TO FD

JCL DCB	FD / 01
`RECFM=FB`	RECORDING MODE F
`LRECL=100`	01 of 100 bytes
`RECFM=VB, LRECL=204`	max 200 + 4 RDW
`BLKSIZE=0`	BLOCK CONTAINS 0

⚠ COMMON MISTAKES

- 01 record length different from the dataset LRECL (status 39 on OPEN)
- Reading fields from the FD buffer after end-of-file
- Counting the 4-byte RDW in the COBOL record length

✓ TIPS & BEST PRACTICES

- Let the system choose block sizes with `BLOCK CONTAINS 0`
- Use `READ … INTO` so data survives the next READ
- Keep a type byte first in every multi-layout record

QUICK SUMMARY

- `FD` — file description
- 01s — buffer layouts (shared)
- `BLOCK CONTAINS 0` — system blocks
- V files — `DEPENDING ON` length

OPEN, READ, WRITE & CLOSE

THE SEQUENTIAL FILE CYCLE

Every file is **opened** in a mode, processed record by record, and **closed**. The standard batch loop uses a **priming read**: read the first record before the loop, process it inside the loop, and read the next record at the bottom, stopping when `AT END` sets the end-of-file flag.

KEY POINTS

- `OUTPUT` creates or empties the file; `EXTEND` appends
- `I-O` allows READ then REWRITE of the same record
- `READ` uses the file name; `WRITE` uses the record name
- Always CLOSE files, even on error paths

COPY AND FILTER A FILE

```
       PROCEDURE DIVISION.
       0000-MAIN.
           OPEN INPUT  CUST-IN
                OUTPUT CUST-OUT
           PERFORM 8000-READ-CUST
           PERFORM UNTIL END-OF-CUST
              IF CL-STATE = "IA"
                 WRITE CUST-OUT-REC FROM CUST-LAYOUT
                 ADD 1 TO WS-WRITTEN
              END-IF
              PERFORM 8000-READ-CUST
           END-PERFORM
           CLOSE CUST-IN CUST-OUT
           DISPLAY "WROTE " WS-WRITTEN
           GOBACK.

       8000-READ-CUST.
           READ CUST-IN INTO CUST-LAYOUT
              AT END     SET END-OF-CUST TO TRUE
              NOT AT END ADD 1 TO WS-READ
           END-READ.

      *>   Update in place:
      *>   OPEN I-O CUST-FILE / READ CUST-FILE
      *>   change fields / REWRITE CUST-REC
```

CODE EXPLAINED

→ The priming read means an empty file skips the loop cleanly

→ `READ … INTO` copies each record into a working layout

→ `WRITE … FROM` copies a layout into the output record and writes it

→ `REWRITE` replaces the record just read (I-O mode)

MODERN EQUIVALENT

↔ **Python** `for line in open(path):` (the loop reads for you)

↔ **Java** `try (var r = Files.newBufferedReader(p)) { while ((line = r.readLine()) != null) … }`

↔ **JavaScript** `for await (const line of rl)` with `readline.createInterface`

PREFER / AVOID

⇄ READ at the top of the loop → priming read + read at the bottom

⇄ MOVE then WRITE → `WRITE … FROM`

OPEN MODES

MODE	ALLOWS
`INPUT`	READ
`OUTPUT`	WRITE (new / emptied file)
`EXTEND`	WRITE at end
`I-O`	READ, REWRITE, DELETE (indexed)

STATEMENTS

STATEMENT	NAMES
`READ file [INTO ws]`	File name
`WRITE rec [FROM ws]`	Record name
`REWRITE rec [FROM ws]`	Record name
`CLOSE file …`	File names
`AT END / NOT AT END`	READ branches

⚠ COMMON MISTAKES

- `WRITE CUST-OUT` (file name) instead of the record name
- Processing the record in the AT END branch, using stale data
- OPEN OUTPUT on a file you meant to append to (data is lost)

✓ TIPS & BEST PRACTICES

- Count records read and written and DISPLAY them at the end
- Check FILE STATUS after OPEN before reading
- Put each READ in its own paragraph so it is written once

QUICK SUMMARY

- `OPEN` — mode per file
- `READ … AT END` — next record
- `WRITE … FROM` — output
- priming read — standard loop

FILE STATUS Codes

00 success · 10 end of file · 22 duplicate key · 23 record not found · 35 file not found · 39 attribute mismatch · check after every I/O

CHECKING EVERY I/O RESULT

After each OPEN, READ, WRITE, REWRITE, DELETE, START or CLOSE, COBOL puts a two-character result code in the file's **FILE STATUS** field. The first digit is the class: 0 success, 1 end of file, 2 key errors, 3 permanent errors, 4 logic errors, 9 implementer-specific.

KEY POINTS

- Declare `PIC XX` and add 88-levels for the codes you handle
- With a FILE STATUS field, many errors return instead of abending
- Check it after **every** statement, not just OPEN
- VSAM adds extra feedback in an optional second field

UNDER THE HOOD

- Status 35 on z/OS: the DD statement is missing or the dataset is not found
- Status 39: LRECL, RECFM or key length in the JCL/catalog does not match the FD
- Status 9x on VSAM: see the VSAM return/feedback codes (extended status)

STATUS CHECKS WITH 88S

```
01  WS-ACCT-FS           PIC XX.
    88 FS-OK                    VALUE "00" "02".
    88 FS-EOF                   VALUE "10".
    88 FS-DUP-KEY               VALUE "22".
    88 FS-NOT-FOUND             VALUE "23".

    OPEN I-O ACCT-MAST
    IF NOT FS-OK
       DISPLAY "OPEN ACCTMAST FAILED, STATUS " WS-ACCT-FS
       MOVE 16 TO RETURN-CODE
       GOBACK
    END-IF

    MOVE WS-ACCT-NO TO AM-ACCT-NO
    READ ACCT-MAST
    EVALUATE TRUE
       WHEN FS-OK          PERFORM 2100-UPDATE
       WHEN FS-NOT-FOUND   PERFORM 2200-ADD-NEW
       WHEN OTHER          PERFORM 9900-IO-ERROR
    END-EVALUATE
```

CODE EXPLAINED

→ `02` means OK with a duplicate alternate key, so it counts as success

→ A failed OPEN ends the program with RC 16

→ EVALUATE handles the expected codes; anything else is an error

→ `9900-IO-ERROR` should log the file, operation, key and status

MODERN EQUIVALENT

↔ **Python** `try: open(...) except FileNotFoundError:` ; `KeyError` for missing keys

↔ **Java** `try { … } catch (NoSuchFileException e)` ; `Optional` for lookups

↔ **JavaScript** `try { … } catch (err) { if (err.code === "ENOENT") … }`

PREFER / AVOID

⇄ checking status only after OPEN → after every I/O statement

⇄ raw `"23"` literals → 88-level names

COMMON CODES

CODE	MEANING
00	Success
02	OK, duplicate alternate key
04	Length mismatch on READ
10	End of file
21	Sequence error (key out of order)
22	Duplicate key on WRITE
23	Record not found
24	Out of space / key beyond limit

ERROR CODES

CODE	MEANING
30	Permanent I/O error
34	Boundary violation (full)
35	File not found on OPEN
37	Open mode not allowed
39	Attributes do not match FD
41 / 42	Already open / not open
46	READ after end of file
48 / 49	WRITE / REWRITE in wrong mode

⚠ COMMON MISTAKES

- No FILE STATUS clause, so errors abend with less information
- Treating status 02 as a failure
- Continuing after a 3x or 4x status as if nothing happened

✓ TIPS & BEST PRACTICES

- Write one common I/O error paragraph that logs and sets RC 16
- Include the key value in error messages for keyed files
- Learn 00, 10, 22, 23, 35, 39 by heart: they cover most problems

QUICK SUMMARY

- `00` — OK
- `10` — end of file
- `22` / `23` — duplicate / not found
- `35` / `39` — missing / mismatched file

Indexed Files (VSAM KSDS)

FILES YOU CAN READ BY KEY

An **indexed** file keeps records in key order and lets you read any record directly by its key. On z/OS this is a **VSAM KSDS** (key-sequenced data set). You can also define **alternate keys**, like an index on a second column, which may allow duplicates.

KEY POINTS

- The primary key must be unique
- The key field must be inside the record, at the position VSAM expects
- Alternate keys need an alternate index (AIX) and PATH in VSAM
- The cluster is created with IDCAMS before the program runs

UNDER THE HOOD

- KSDS = data component + index component, defined together as a cluster
- The KEYS(len offset) in IDCAMS must match the RECORD KEY in the FD
- Inserts split control intervals (CI splits); FREESPACE leaves room for them

INDEXED FILE DEFINITION

```
       FILE-CONTROL.
           SELECT ACCT-MAST ASSIGN TO ACCTMAST
               ORGANIZATION IS INDEXED
               ACCESS MODE  IS DYNAMIC
               RECORD KEY   IS AM-ACCT-NO
               ALTERNATE RECORD KEY IS AM-SSN
                  WITH DUPLICATES
               FILE STATUS  IS WS-ACCT-FS.

       FILE SECTION.
       FD  ACCT-MAST.
       01  ACCT-REC.
           05 AM-ACCT-NO        PIC 9(10).
           05 AM-SSN            PIC 9(9).
           05 AM-NAME           PIC X(30).
           05 AM-BALANCE        PIC S9(9)V99 COMP-3.
           05 FILLER            PIC X(45).

      *>   IDCAMS (see the JCL set):
      *>   DEFINE CLUSTER (NAME(PROD.ACCT.MAST) INDEXED -
      *>          KEYS(10 0) RECORDSIZE(100 100))
```

CODE EXPLAINED

- → `KEYS(10 0)` : a 10-byte key at offset 0 = AM-ACCT-NO
- → DYNAMIC access allows random reads and browsing
- → `WITH DUPLICATES` lets several accounts share an SSN
- → The record is 100 bytes, matching `RECORDSIZE(100 100)`

MODERN EQUIVALENT

- ↔ **Python** `sqlite3` table with a PRIMARY KEY, or a `dict` keyed by account
- ↔ **Java** A database table via JDBC, or a `TreeMap<Long, Account>`
- ↔ **JavaScript** A database table, or a `Map` keyed by account number

PREFER / AVOID

- ⇄ sequential master files + sort/merge → KSDS for random lookups
- ⇄ key fields mid-record by accident → put the primary key first

ACCESS MODES

MODE	ALLOWS
SEQUENTIAL	Key order only
RANDOM	By key only
DYNAMIC	Both; READ NEXT after START

VSAM TYPES

TYPE	COBOL ORGANIZATION
KSDS	INDEXED
ESDS	SEQUENTIAL
RRDS	RELATIVE
AIX + PATH	ALTERNATE RECORD KEY
LDS	Not used from COBOL

⚠ COMMON MISTAKES

- FD key position or length not matching the IDCAMS KEYS (status 39)
- Opening a KSDS that was never loaded in INPUT mode (empty cluster issues)
- Forgetting the PATH DD for alternate keys

✓ TIPS & BEST PRACTICES

- Put the primary key first in the record
- Use DYNAMIC access when one program both looks up and browses
- Keep the IDCAMS DEFINE in source control with the program

QUICK SUMMARY

- `INDEXED` — VSAM KSDS
- `RECORD KEY` — unique key
- `ALTERNATE RECORD KEY` — second index
- `DYNAMIC` — random + browse

Random & Dynamic Access

READING AND UPDATING BY KEY

With RANDOM or DYNAMIC access, move a key value into the key field and READ to get that record. START positions at the first record matching a key condition, and READ NEXT then browses forward. INVALID KEY runs when the key is missing or duplicated.

KEY POINTS

- Set the key field, then READ: the key is not a READ argument
- `INVALID KEY` or FILE STATUS 23 means not found
- `WRITE` adds; status 22 means the key already exists
- REWRITE and DELETE need the file open I-O

LOOKUP, UPDATE AND BROWSE

```
           OPEN I-O ACCT-MAST

       *>  Random read and update
           MOVE 1000004217 TO AM-ACCT-NO
           READ ACCT-MAST
              INVALID KEY     DISPLAY "NO SUCH ACCOUNT"
              NOT INVALID KEY
                 ADD WS-DEPOSIT TO AM-BALANCE
                 REWRITE ACCT-REC
           END-READ

       *>  Add a new record
           WRITE ACCT-REC FROM WS-NEW-ACCT
              INVALID KEY DISPLAY "DUPLICATE " AM-ACCT-NO
           END-WRITE

       *>  Browse all accounts for one SSN (alternate key)
           MOVE WS-SSN TO AM-SSN
           START ACCT-MAST KEY IS = AM-SSN
              INVALID KEY SET END-OF-BROWSE TO TRUE
           END-START
           PERFORM UNTIL END-OF-BROWSE
              READ ACCT-MAST NEXT RECORD
                 AT END SET END-OF-BROWSE TO TRUE
              END-READ
              IF NOT END-OF-BROWSE AND AM-SSN NOT = WS-SSN
                 SET END-OF-BROWSE TO TRUE
              END-IF
           END-PERFORM
```

CODE EXPLAINED

- → The key value in AM-ACCT-NO picks the record
- → REWRITE replaces the record just read; the key must not change
- → `START KEY IS = AM-SSN` switches the browse to the alternate key
- → The browse stops when the SSN changes

MODERN EQUIVALENT

- ↔ **Python** `row = db.execute("SELECT … WHERE acct=?", (n,)).fetchone()`
- ↔ **Java** `ps.setLong(1, n); ResultSet rs = ps.executeQuery();`
- ↔ **JavaScript** `await db.get("SELECT … WHERE acct = ?", n)`

PREFER / AVOID

- ⇄ READ then check a field for "not found" → `INVALID KEY` / status 23
- ⇄ changing the primary key with REWRITE → DELETE + WRITE

KEYED STATEMENTS

STATEMENT	NEEDS
`READ f [KEY IS k]`	Key value set
`READ f NEXT`	Position from START or READ
`START f KEY IS >= k`	Position for browse
`WRITE rec`	New unique key
`REWRITE rec`	Prior READ, I-O
`DELETE f`	Prior READ (sequential) or key

START CONDITIONS

CONDITION	POSITIONS AT
`KEY IS = k`	Exact match
`KEY IS >= k`	First ≥ k
`KEY IS > k`	First > k
`KEY IS <= k (2002)`	Last ≤ k

⚠ COMMON MISTAKES

- Changing the key field before REWRITE (status 21 or wrong record)
- READ NEXT without a successful START or READ first
- Browsing an alternate key without checking when the value changes

✓ TIPS & BEST PRACTICES

- Use a generic START (`>=`) with a partial key to browse a range
- Read for update immediately before REWRITE to keep data current
- Log the key on every INVALID KEY branch

QUICK SUMMARY

- key field + `READ` — lookup
- `INVALID KEY` — not found / dup
- `START` + `READ NEXT` — browse
- `REWRITE` / `DELETE` — I-O mode

Relative Files

RECORDS BY SLOT NUMBER

A **relative** file is a row of numbered slots, and you read or write a record by its slot number held in the RELATIVE KEY field. It is the COBOL equivalent of an array on disk, mapped to a VSAM **RRDS**. It suits data keyed by a small dense number, like day of year or store number.

KEY POINTS

- Slot numbers start at 1
- The RELATIVE KEY field is in WORKING-STORAGE, not the record
- Empty slots return INVALID KEY / status 23 on READ
- Sequential READ skips empty slots

STORE TOTALS BY STORE NUMBER

```
       FILE-CONTROL.
           SELECT STORE-FILE ASSIGN TO STORES
               ORGANIZATION IS RELATIVE
               ACCESS MODE  IS RANDOM
               RELATIVE KEY IS WS-STORE-NO
               FILE STATUS  IS WS-STORE-FS.

       FILE SECTION.
       FD  STORE-FILE.
       01  STORE-REC.
           05 SR-NAME          PIC X(20).
           05 SR-SALES         PIC S9(9)V99 COMP-3.

       WORKING-STORAGE SECTION.
       01  WS-STORE-NO         PIC 9(4).

           OPEN I-O STORE-FILE
           MOVE 117 TO WS-STORE-NO
           READ STORE-FILE
              INVALID KEY
                 MOVE "NEW STORE" TO SR-NAME
                 MOVE ZERO TO SR-SALES
                 ADD WS-AMT TO SR-SALES
                 WRITE STORE-REC
              NOT INVALID KEY
                 ADD WS-AMT TO SR-SALES
                 REWRITE STORE-REC
           END-READ
```

CODE EXPLAINED

→ Store 117 lives in slot 117

→ An empty slot gives INVALID KEY, so the code creates it

→ WRITE puts the record in the slot named by WS-STORE-NO

→ The RRDS is defined with IDCAMS `NUMBERED`

MODERN EQUIVALENT

↔ **Python** A list indexed by number, or a fixed-size binary file with `seek(n * size)`

↔ **Java** `RandomAccessFile.seek((n - 1) * size)`

↔ **JavaScript** `fs.read(fd, buf, 0, size, (n - 1) * size)`

PREFER / AVOID

⇄ relative files for sparse keys (SSNs) → an indexed file

⇄ record-number math in the program → RELATIVE KEY

RELATIVE VS INDEXED

	RELATIVE	INDEXED
Key	Slot number	Field in record
Key location	WORKING-STORAGE	Record
VSAM	RRDS	KSDS
Good for	Small dense numbers	Any unique key
Alternate keys	No	Yes

⚠ COMMON MISTAKES

- Putting the RELATIVE KEY inside the record (it must be outside)
- Slot 0 (numbers start at 1)
- Using relative files for large, sparse number ranges

✓ TIPS & BEST PRACTICES

- Pre-load empty slots if programs expect every slot to exist
- Use DYNAMIC access plus START to browse from a slot
- Prefer KSDS for anything that may grow unpredictably

QUICK SUMMARY

- `RELATIVE` — slots by number
- `RELATIVE KEY` — slot field
- RRDS — VSAM type
- empty slot — status 23

The SORT Verb

SD sort file · SORT … ON ASCENDING KEY · USING / GIVING · INPUT PROCEDURE / OUTPUT PROCEDURE · RELEASE / RETURN · SORT-RETURN

SORTING FILES INSIDE A PROGRAM

SORT orders records by one or more keys using the system sort (DFSORT on z/OS). The simple form sorts one file USING into another GIVING. Input and output **procedures** let you filter or change records on the way in (RELEASE) and process them on the way out (RETURN).

KEY POINTS

- The sort work file is declared with `SD`, not FD
- USING/GIVING files must not be open
- Keys are fields in the SD record
- `SORT-RETURN` is 0 on success

UNDER THE HOOD

- COBOL SORT calls DFSORT on z/OS; sort work space comes from SORTWKnn DDs or dynamic allocation
- GnuCOBOL implements SORT itself with temporary files
- Filtering in the input procedure means fewer records to sort

USING/GIVING AND PROCEDURES

```
       FILE-CONTROL.
           SELECT SORT-WORK ASSIGN TO SORTWK1.
       DATA DIVISION.
       FILE SECTION.
       SD  SORT-WORK.
       01  SW-REC.
           05 SW-STATE          PIC XX.
           05 SW-CITY           PIC X(20).
           05 SW-SALES          PIC S9(7)V99 COMP-3.
           05 FILLER            PIC X(53).

       PROCEDURE DIVISION.
           SORT SORT-WORK
                ON ASCENDING  KEY SW-STATE SW-CITY
                ON DESCENDING KEY SW-SALES
                USING  SALES-IN
                GIVING SALES-SORTED
           IF SORT-RETURN NOT = 0
              DISPLAY "SORT FAILED " SORT-RETURN
              MOVE 16 TO RETURN-CODE
           END-IF

           SORT SORT-WORK ON ASCENDING KEY SW-STATE
                INPUT PROCEDURE  2000-SELECT-IA-MN
                GIVING SALES-SORTED.
       2000-SELECT-IA-MN.
           PERFORM UNTIL END-OF-SALES
              READ SALES-IN INTO SW-REC
                 AT END SET END-OF-SALES TO TRUE
                 NOT AT END
                    IF SW-STATE = "IA" OR "MN"
                       RELEASE SW-REC
                    END-IF
              END-READ
           END-PERFORM.
```

CODE EXPLAINED

- → Sort by state and city ascending, then sales descending
- → USING/GIVING opens and closes the files for you
- → The input procedure opens SALES-IN itself (not shown) and RELEASEs only IA and MN
- → An output procedure would `RETURN` sorted records one at a time

MODERN EQUIVALENT

- ↔ **Python** `sorted(rows, key=lambda r: (r.state, r.city, -r.sales))`
- ↔ **Java** `list.sort(comparing(R::state).thenComparing(R::city).thenComparing(R::sales, reverseOrder()))`
- ↔ **JavaScript** `rows.sort((a, b) => a.state.localeCompare(b.state) || b.sales - a.sales)`

PREFER / AVOID

- ⇄ sorting in a COBOL table by hand → SORT verb or a DFSORT step
- ⇄ SORT with no SORT-RETURN check → test it and set RETURN-CODE

SORT PIECES

PIECE	ROLE
SD	Sort work file
ON ASCENDING / DESCENDING KEY	Sort order
USING file	Input as-is
INPUT PROCEDURE	Filter + RELEASE
GIVING file	Output as-is
OUTPUT PROCEDURE	RETURN + process
WITH DUPLICATES IN ORDER	Stable for equal keys

⚠ COMMON MISTAKES

- Opening the USING or GIVING file yourself (they must be closed)
- WRITE instead of RELEASE in an input procedure
- No check of SORT-RETURN, so a failed sort looks successful

✓ TIPS & BEST PRACTICES

- Filter early in the input procedure to sort less data
- Add `WITH DUPLICATES IN ORDER` when equal keys must keep input order
- For plain sorts with no logic, a DFSORT JCL step is simpler

QUICK SUMMARY

- `SD` — sort file
- `SORT … KEY` — order
- `USING` / `GIVING` — whole files
- `RELEASE` / `RETURN` — procedures

MERGE & Sort Utilities

COMBINING SORTED FILES

MERGE combines two or more files that are **already sorted** on the same keys into one sorted file. Many shops skip COBOL for plain sorting and merging and use a **DFSORT** (or Syncsort) step in the JCL instead, keeping COBOL for the business logic.

KEY POINTS

- All MERGE inputs must be in the merge key order
- MERGE allows an OUTPUT PROCEDURE but no INPUT PROCEDURE
- DFSORT can also filter, reformat and sum in one step
- `SORT-CONTROL` names a DD with extra sort control statements

MERGE IN COBOL

```
       FILE-CONTROL.
           SELECT MERGE-WORK ASSIGN TO MRGWK.
           SELECT EAST-TRANS ASSIGN TO EASTIN.
           SELECT WEST-TRANS ASSIGN TO WESTIN.
           SELECT ALL-TRANS  ASSIGN TO ALLOUT.

       SD  MERGE-WORK.
       01  MW-REC.
           05 MW-ACCT          PIC 9(10).
           05 MW-DATE          PIC 9(8).
           05 FILLER           PIC X(62).

           MERGE MERGE-WORK
                 ON ASCENDING KEY MW-ACCT MW-DATE
                 USING  EAST-TRANS WEST-TRANS
                 GIVING ALL-TRANS
```

CODE EXPLAINED

→ East and west files are each sorted by account and date

→ MERGE interleaves them into one sorted output

→ The DFSORT table below does the same job as a JCL step

→ Use COBOL only when per-record logic is needed

MODERN EQUIVALENT

↔ **Python** `heapq.merge(east, west, key=lambda r: (r.acct, r.date))`

↔ **Java** Stream concat + sort, or a manual two-pointer merge

↔ **JavaScript** Concat + sort, or a two-pointer merge for big sorted inputs

PREFER / AVOID

⇄ COBOL program that only sorts → DFSORT step

⇄ merging unsorted files → sort each first

DFSORT EQUIVALENT

STATEMENT	DOES
`SORT FIELDS=(1,10,CH,A)`	Sort on bytes 1–10
`MERGE FIELDS=(1,10,CH,A,11,8,CH,A)`	Merge two keys
`INCLUDE COND=(1,2,CH,EQ,C'IA')`	Keep IA only
`SUM FIELDS=NONE`	Drop duplicate keys
`OUTREC BUILD=(1,30)`	Reformat output

COBOL OR UTILITY?

NEED	USE
`Plain sort / merge`	DFSORT step
`Filter by simple field test`	DFSORT INCLUDE
`Complex per-record rules`	COBOL SORT + procedures
`Lookups during sort`	COBOL input procedure

⚠ COMMON MISTAKES

- Merging files that are not sorted on the merge keys (wrong order, no error)
- Different record layouts in the inputs
- Writing a COBOL program for a job one DFSORT statement could do

✓ TIPS & BEST PRACTICES

- Keep DFSORT control cards in a PDS member so they are versioned
- Check sort order with a quick DFSORT `SORT … CKFIELDS` or a COBOL sequence check
- See the JCL set for SORT step details

QUICK SUMMARY

- `MERGE` — combine sorted files
- DFSORT — sort in JCL
- `SORT FIELDS=` — utility keys
- COBOL — for per-record logic

Report Layout

PRINTING PAGED REPORTS

A COBOL report is a print file written line by line. Each line type is a WORKING-STORAGE group: headings, details and totals. `WRITE … AFTER ADVANCING` controls spacing, and the program counts lines so it can start a new page with fresh headings when the page fills.

KEY POINTS

- `AFTER ADVANCING PAGE` starts a new page
- `AFTER ADVANCING 2 LINES` double-spaces
- Count lines yourself and break at a page limit
- Print datasets often use RECFM=FBA (ASA control character first)

HEADINGS, PAGING AND DETAILS

```
01  WS-LINE-CT       PIC 99   VALUE 99.
01  WS-PAGE-CT       PIC 999  VALUE 0.
01  WS-MAX-LINES     PIC 99   VALUE 55.
01  HDG-1.
    05 FILLER        PIC X(30) VALUE "ACME BANK".
    05 FILLER        PIC X(30) VALUE "ACCOUNT BALANCES".
    05 FILLER        PIC X(6)  VALUE "PAGE ".
    05 HDG-PAGE      PIC ZZ9.
01  HDG-2            PIC X(66) VALUE
    "ACCOUNT     NAME                          BALANCE".

2000-PRINT-DETAIL.
    IF WS-LINE-CT >= WS-MAX-LINES
       PERFORM 2100-HEADINGS
    END-IF
    WRITE PRINT-REC FROM DETAIL-LINE
        AFTER ADVANCING 1 LINE
    ADD 1 TO WS-LINE-CT.

2100-HEADINGS.
    ADD 1 TO WS-PAGE-CT
    MOVE WS-PAGE-CT TO HDG-PAGE
    WRITE PRINT-REC FROM HDG-1 AFTER ADVANCING PAGE
    WRITE PRINT-REC FROM HDG-2 AFTER ADVANCING 2 LINES
    MOVE 3 TO WS-LINE-CT.
```

CODE EXPLAINED

- → Starting WS-LINE-CT at 99 forces headings before the first detail
- → Headings reset the line count to the lines they used
- → HDG-2 is one literal laid out as column titles
- → Detail lines come from an edited group (sheet 19)

MODERN EQUIVALENT

- ↔ **Python** Write lines to a text file; reportlab or HTML templates for paged output
- ↔ **Java** Jasper Reports or `PrintWriter` with manual paging
- ↔ **JavaScript** HTML/CSS print styles or a PDF library like pdfkit

PREFER / AVOID

- ⇄ hard-coded 60-line pages → a `WS-MAX-LINES` constant
- ⇄ headings written once at start → headings on every page break

ADVANCING OPTIONS

PHRASE	EFFECT
`AFTER ADVANCING 1 LINE`	Single space
`AFTER ADVANCING 2 LINES`	Double space
`AFTER ADVANCING PAGE`	New page
`BEFORE ADVANCING n`	Print, then move
`AFTER ADVANCING ws-n`	Variable spacing

ASA CONTROL CHARS

CHAR	BEFORE PRINTING
space	Single space
0	Double space
-	Triple space
1	New page
+	No space (overprint)

⚠ COMMON MISTAKES

- Line counter starting at 0, so the first page has no headings
- Forgetting to reset the line count after headings
- Mixing BEFORE and AFTER ADVANCING in one file

✓ TIPS & BEST PRACTICES

- Put report title and run date in the first heading line
- Keep page size in one field so it can change easily
- Print a "*** END OF REPORT ***" line so readers know it is complete

QUICK SUMMARY

- line groups — heading / detail / total
- `AFTER ADVANCING PAGE` — new page
- line counter — page breaks
- FBA — ASA control byte

Control Break Processing

sorted input · holding the previous key · single-level break · multi-level breaks · subtotals & grand totals · end-of-file final break

SUBTOTALS WHEN A KEY CHANGES

A **control break** happens when the key of the current record differs from the previous one in a sorted file. At each break the program prints a subtotal, resets the accumulator and saves the new key. At end of file it forces a final break, then prints the grand total.

KEY POINTS

- Input must be sorted on the break keys
- Save the key of the group you are totalling
- Higher-level breaks trigger all lower-level breaks first
- Roll each subtotal into the next level's total

SINGLE-LEVEL BREAK BY STATE

```
01  WS-PREV-STATE     PIC XX      VALUE SPACES.
01  WS-STATE-TOT      PIC S9(9)V99 VALUE 0.
01  WS-GRAND-TOT      PIC S9(11)V99 VALUE 0.

0000-MAIN.
    OPEN INPUT SALES-IN OUTPUT RPT-OUT
    PERFORM 8000-READ
    IF NOT END-OF-SALES
       MOVE SL-STATE TO WS-PREV-STATE
    END-IF
    PERFORM UNTIL END-OF-SALES
       IF SL-STATE NOT = WS-PREV-STATE
          PERFORM 3000-STATE-BREAK
       END-IF
       ADD SL-AMT TO WS-STATE-TOT
       PERFORM 2000-PRINT-DETAIL
       PERFORM 8000-READ
    END-PERFORM
    PERFORM 3000-STATE-BREAK            *> final break
    PERFORM 4000-GRAND-TOTAL
    CLOSE SALES-IN RPT-OUT
    GOBACK.

3000-STATE-BREAK.
    MOVE WS-PREV-STATE TO ST-LINE-STATE
    MOVE WS-STATE-TOT  TO ST-LINE-AMT
    WRITE PRINT-REC FROM STATE-TOTAL-LINE
        AFTER ADVANCING 2 LINES
    ADD WS-STATE-TOT TO WS-GRAND-TOT
    MOVE ZERO TO WS-STATE-TOT
    MOVE SL-STATE TO WS-PREV-STATE.
```

CODE EXPLAINED

- → The first key is saved before the loop
- → A changed key prints the previous state's total
- → The final break after the loop prints the last state
- → Each state total rolls into the grand total, then resets

MODERN EQUIVALENT

- ↔ **Python** `itertools.groupby(rows, key=lambda r: r.state)` then `sum`
- ↔ **Java** `Collectors.groupingBy(R::state, summingDouble(R::amt))`
- ↔ **JavaScript** `Object.groupBy(rows, r => r.state)` then reduce each group

PREFER / AVOID

- ⇄ forgetting the final break → always break once more after the loop
- ⇄ unsorted input → sort step before the report

MULTI-LEVEL ORDER

KEY CHANGE	RUN BREAKS
City only	City
State	City, then State
Region	City, State, then Region
End of file	All levels, then grand total

BREAK PARAGRAPH STEPS

STEP	ACTION
1	Print subtotal line
2	Add subtotal to next level
3	Reset subtotal to zero
4	Save new key

⚠ COMMON MISTAKES

- Missing the final break, so the last group has no total
- Saving the new key before printing, so totals show the wrong label
- Running a break report on unsorted data

✓ TIPS & BEST PRACTICES

- Write one break paragraph per level and call lower levels from higher ones
- Check that grand total = sum of all detail amounts
- Test with an empty file and a one-record file

QUICK SUMMARY

- break — key changed
- save previous key — label totals
- final break — after the loop
- roll up — subtotal → total

Sheet 51

Master / Transaction File Update

balanced-line algorithm · matching keys · adds / changes / deletes · HIGH-VALUES sentinel · old master → new master

CLASSIC SEQUENTIAL UPDATE

The **balanced-line** algorithm updates a sorted master file with a sorted transaction file. Both files are read in key order; compare the keys and act: transaction key lower means an **add**, equal means a **change or delete**, master key lower means copy the master unchanged. Output is a new master file.

KEY POINTS

- Both files sorted on the same key
- At end of file, move HIGH-VALUES to that file's key
- Loop until both keys are HIGH-VALUES
- Unmatched changes and duplicate adds go to an error report

BALANCED-LINE LOOP

```
0000-MAIN.
    OPEN INPUT OLD-MAST TRANS OUTPUT NEW-MAST ERR-RPT
    PERFORM 8100-READ-MAST
    PERFORM 8200-READ-TRAN
    PERFORM UNTIL WS-MAST-KEY = HIGH-VALUES
              AND WS-TRAN-KEY = HIGH-VALUES
       EVALUATE TRUE
          WHEN WS-TRAN-KEY < WS-MAST-KEY
             PERFORM 2000-ADD           *> new account
             PERFORM 8200-READ-TRAN
          WHEN WS-TRAN-KEY = WS-MAST-KEY
             PERFORM 3000-CHANGE-OR-DELETE
             PERFORM 8200-READ-TRAN
          WHEN OTHER
             WRITE NEW-REC FROM OM-REC  *> unchanged
             PERFORM 8100-READ-MAST
       END-EVALUATE
    END-PERFORM
    CLOSE OLD-MAST TRANS NEW-MAST ERR-RPT
    GOBACK.

8100-READ-MAST.
    READ OLD-MAST
       AT END     MOVE HIGH-VALUES TO WS-MAST-KEY
       NOT AT END MOVE OM-ACCT     TO WS-MAST-KEY
    END-READ.
```

CODE EXPLAINED

- → HIGH-VALUES sort after every real key, so the other file drains first
- → 3000 applies a change to the master in memory, or flags a delete
- → Multiple transactions for one account are applied in turn
- → The new master is written only when the master record moves on (not shown)

MODERN EQUIVALENT

- ↔ **Python** A database `UPDATE` / `INSERT` / `DELETE` per transaction, or a merge-join of two sorted iterators
- ↔ **Java** JDBC batch upserts, or a two-pointer merge over sorted streams
- ↔ **JavaScript** SQL upserts, or a two-pointer merge over sorted arrays

PREFER / AVOID

- ⇄ in-place REWRITE of a tape-era master → keep old/new masters for restart
- ⇄ magic `999999` end keys → `HIGH-VALUES`

KEY COMPARISON

CONDITION	ACTION
TRAN < MAST	Add (or error if not an add)
TRAN = MAST	Change / delete (or error if add)
TRAN > MAST	Copy master to new master
Both HIGH-VALUES	Done

TRANSACTION CODES

CODE	VALID WHEN
A (add)	No matching master
C (change)	Master exists
D (delete)	Master exists
Invalid	Write to error report

⚠ COMMON MISTAKES

- Unsorted transactions, which produce adds for existing accounts
- Using numeric keys with HIGH-VALUES (use alphanumeric keys or 9s)
- Writing the master once per transaction instead of once per account

✓ TIPS & BEST PRACTICES

- Sort transactions by key, then by transaction type
- Keep old and new masters so a failed run can rerun from the old one
- Print control totals: masters in/out, adds, changes, deletes, errors

QUICK SUMMARY

- sorted master + sorted trans
- compare keys — add / change / copy
- `HIGH-VALUES` — end sentinel
- new master — restartable output

Sheet 52

Data Validation & Error Handling

NUMERIC checks · range checks with 88s · error / reject files · RETURN-CODE special register · ON EXCEPTION · control totals

REJECTING BAD DATA WITHOUT ABENDING

Batch programs validate every input field before using it. Bad records go to a **reject file** with a reason code instead of stopping the run. At the end, **control totals** prove nothing was lost, and RETURN-CODE tells the JCL whether the step was clean, had warnings or failed.

KEY POINTS

- Check NUMERIC before any arithmetic on input fields
- Use 88-levels for valid code lists and ranges
- Write rejects with the original record plus a reason
- Set RETURN-CODE: 0 clean, 4 rejects, 16 fatal

VALIDATE, REJECT, REPORT

```
       01  WS-REASON            PIC X(30).
       01  WS-COUNTS.
           05 WS-READ            PIC 9(7) VALUE 0.
           05 WS-GOOD            PIC 9(7) VALUE 0.
           05 WS-BAD             PIC 9(7) VALUE 0.

       2100-VALIDATE.
           MOVE SPACES TO WS-REASON
           EVALUATE TRUE
              WHEN IN-ACCT NOT NUMERIC
                 MOVE "ACCOUNT NOT NUMERIC" TO WS-REASON
              WHEN IN-AMT NOT NUMERIC
                 MOVE "AMOUNT NOT NUMERIC"  TO WS-REASON
              WHEN NOT VALID-TRAN-CODE
                 MOVE "BAD TRAN CODE"       TO WS-REASON
              WHEN IN-AMT > 50000
                 MOVE "AMOUNT OVER LIMIT"   TO WS-REASON
           END-EVALUATE
           IF WS-REASON = SPACES
              ADD 1 TO WS-GOOD
           ELSE
              ADD 1 TO WS-BAD
              STRING WS-REASON DELIMITED BY SIZE
                     IN-REC    DELIMITED BY SIZE
                INTO REJECT-REC
              WRITE REJECT-REC
           END-IF.

       9000-TERM.
           DISPLAY "READ " WS-READ " GOOD " WS-GOOD " BAD " WS-BAD
           EVALUATE TRUE
              WHEN WS-READ NOT = WS-GOOD + WS-BAD
                 MOVE 16 TO RETURN-CODE
              WHEN WS-BAD > 0
                 MOVE 4 TO RETURN-CODE
           END-EVALUATE.
```

CODE EXPLAINED

- → EVALUATE stops at the first failing check: one reason per record
- → The reject record keeps the reason and the original data
- → Read = good + bad proves every record was handled
- → RC 4 lets the next step run but flags the rejects

MODERN EQUIVALENT

- ↔ **Python** Pydantic or manual checks; append bad rows to a rejects list or file
- ↔ **Java** Bean Validation (`@NotNull` , `@Max`) and a reject writer
- ↔ **JavaScript** Zod or Joi schemas; `safeParse` and log failures

PREFER / AVOID

- ⇄ abending on the first bad record → reject file + RC 4
- ⇄ no control totals → print counts and check they balance

CHECKS

CHECK	COBOL
Is a number	IS NUMERIC
In a code list	88 VALUE "A" "C" "D"
In a range	88 VALUE 1 THRU 12
Not blank	NOT = SPACES
Valid date	FUNCTION TEST-DATE-YYYYMMDD
Cross-field	IF END-DT < START-DT

RETURN CODES

RC	MEANING
0	All records good
4	Some rejects
8	Many rejects / threshold
16	Totals off or fatal error

⚠ COMMON MISTAKES

- Doing arithmetic before validation and abending with SOC7
- Rejects with no reason, so nobody can fix the data
- Never comparing record counts at the end

✓ TIPS & BEST PRACTICES

- Use `FUNCTION TEST-DATE-YYYYMMDD` (returns 0 if valid) for dates
- Add a reject threshold that fails the run if too many records are bad
- Display control totals so they show in the job log

QUICK SUMMARY

- validate before use
- reject file — record + reason
- control totals — read = good + bad
- `RETURN-CODE` — 0 / 4 / 16

COPY & Copybooks

SHARING CODE BETWEEN PROGRAMS

A **copybook** is a source member that the compiler pastes in wherever COPY appears. Shops keep every record layout, common working-storage and standard paragraphs in copybooks so all programs agree on file formats. REPLACING swaps text as it is copied, such as a field prefix.

KEY POINTS

- Copybooks are found in SYSLIB (z/OS) or `-I` paths (GnuCOBOL)
- A COPY statement ends with a period
- `==text==` delimits pseudo-text for REPLACING
- Changing a copybook means recompiling every program that uses it

A COPYBOOK AND TWO USES

```
      *> Member CUSTREC (in USER.COPYLIB or copy/CUSTREC.cpy)
       01  :PFX:-CUSTOMER.
           05 :PFX:-ID          PIC 9(6).
           05 :PFX:-NAME        PIC X(30).
           05 :PFX:-STATE       PIC XX.
           05 :PFX:-BALANCE     PIC S9(7)V99 COMP-3.

      *> In the program:
       WORKING-STORAGE SECTION.
           COPY CUSTREC REPLACING ==:PFX:== BY ==IN==.
           COPY CUSTREC REPLACING ==:PFX:== BY ==OUT==.
      *>   gives IN-CUSTOMER, IN-ID … and OUT-CUSTOMER, OUT-ID …

       PROCEDURE DIVISION.
           MOVE IN-ID TO OUT-ID
           COPY ERRPARA.                    *> shared paragraphs

      *> GnuCOBOL:  cobc -x -I copy main.cbl
      *> z/OS:      //COBOL.SYSLIB DD DSN=USER.COPYLIB,DISP=SHR
```

CODE EXPLAINED

- → `:PFX:` is a placeholder tag, replaced on each COPY
- → One layout serves input and output with different prefixes
- → Paragraph copybooks share standard error handling
- → The compile listing shows copied lines, often marked with a C

MODERN EQUIVALENT

- ↔ **Python** `import` a module with a shared `@dataclass`
- ↔ **Java** A shared class or record in a common library JAR
- ↔ **JavaScript** `import` a shared type or schema module

PREFER / AVOID

- ⇄ layouts typed into each program → one copybook per record
- ⇄ copybooks with logic that varies by program → parameters via REPLACING or CALL

COPY FORMS

FORM	EFFECT
`COPY member.`	Paste member
`COPY member OF lib.`	From a named library
`REPLACING ==a== BY ==b==`	Swap pseudo-text
`REPLACING a BY b`	Swap whole words
`REPLACE ==a== BY ==b==.`	Applies to following source
`REPLACE OFF.`	Stop replacing

WHERE COPYBOOKS LIVE

ENVIRONMENT	LOCATION
`z/OS`	PDS in SYSLIB DD
`GnuCOBOL`	-I dir; COB_COPY_DIR
`Extensions`	.cpy, .cbl, .CPY
`Git repos`	copy/ or copybook/ folder

⚠ COMMON MISTAKES

- Changing a copybook and recompiling only some programs (file layouts disagree)
- Missing period after COPY
- Placeholder text that also appears in normal names, so REPLACING changes too much

✓ TIPS & BEST PRACTICES

- Use a distinctive tag like `:PFX:` for prefixes
- Search the compile listing to see the final expanded source
- Track which programs use each copybook (impact analysis)

QUICK SUMMARY

- `COPY m.` — paste source
- `REPLACING` — swap text
- SYSLIB / `-I` — search path
- one layout per copybook

CALL & Subprograms

CALL 'PROG' USING · BY REFERENCE / BY CONTENT / BY VALUE · static vs dynamic calls · CANCEL · ON EXCEPTION · DYNAM option

CALLING OTHER PROGRAMS

CALL runs another compiled program and passes data with USING. **BY REFERENCE** (the default) shares the caller's storage, **BY CONTENT** passes a copy, and **BY VALUE** passes the value itself (for C-style APIs). Calls are **static** (linked in) or **dynamic** (loaded at run time).

KEY POINTS

- A literal name with `NODYNAM` is a static call
- A data name (`CALL WS-PGM`) is always dynamic
- The subprogram stays in memory, keeping WORKING-STORAGE, until `CANCEL`
- `ON EXCEPTION` runs if the program cannot be found

UNDER THE HOOD

- Static calls link the subprogram into the caller's load module (binder resolves it)
- Dynamic calls load the module from STEPLIB/JOBLIB at the first CALL
- Missing module on a dynamic call without ON EXCEPTION = S806 abend

CALLER AND SUBPROGRAM

```
      *> Caller
       01  WS-PGM            PIC X(8) VALUE "CALCTAX".
       01  WS-TAX-PARMS.
           05 TP-AMOUNT      PIC S9(7)V99 COMP-3.
           05 TP-STATE       PIC XX.
           05 TP-TAX         PIC S9(7)V99 COMP-3.
           05 TP-RC          PIC 99.

           MOVE 1250.00 TO TP-AMOUNT
           MOVE "IA"    TO TP-STATE
           CALL WS-PGM USING BY REFERENCE WS-TAX-PARMS
              ON EXCEPTION
                 DISPLAY "CALCTAX NOT FOUND"
           END-CALL
           IF TP-RC NOT = 0 DISPLAY "TAX ERROR " TP-RC END-IF
           CALL "LOGIT" USING BY CONTENT "TAX DONE"
           CANCEL WS-PGM                    *> fresh state next call

      *> Subprogram CALCTAX
       LINKAGE SECTION.
           COPY TAXPARMS.                   *> same layout
       PROCEDURE DIVISION USING TAX-PARMS.
           COMPUTE TP-TAX ROUNDED = TP-AMOUNT * .06
           MOVE 0 TO TP-RC
           GOBACK.
```

CODE EXPLAINED

- → Calling through WS-PGM makes the call dynamic
- → BY REFERENCE: CALCTAX writes TP-TAX straight into the caller's field
- → BY CONTENT: LOGIT gets a copy and cannot change the caller's data
- → Caller and callee share the TAXPARMS copybook so layouts match

MODERN EQUIVALENT

- ↔ **Python** `tax = calc_tax(amount, state)` ; mutable objects act like BY REFERENCE
- ↔ **Java** `calcTax(params)` ; objects are passed by reference to the value
- ↔ **JavaScript** `calcTax(params)` ; objects shared, primitives copied

PREFER / AVOID

- ⇄ many loose parameters → one parameter group in a copybook
- ⇄ STOP RUN in subprograms → `GOBACK`

PASSING MODES

MODE	CALLEE SEES
BY REFERENCE	Caller's storage (default)
BY CONTENT	A copy
BY VALUE	The value (C interop)
RETURNING x	Function-style result

STATIC VS DYNAMIC

	STATIC	DYNAMIC
Written as	CALL "LIT" + NODYNAM	CALL ws-name / DYNAM
Resolved	Link time	Run time
Change sub	Relink caller	Replace module only
Missing module	Link error	S806 / ON EXCEPTION

⚠ COMMON MISTAKES

- Caller and callee layouts out of sync, shifting every field
- Passing fewer parameters than the callee's USING expects (storage overlay)
- Expecting a subprogram's WORKING-STORAGE to reset each call without CANCEL

✓ TIPS & BEST PRACTICES

- Put every parameter in one group defined in a shared copybook
- Add a return-code field to every parameter area
- Use dynamic calls when the subprogram changes often

QUICK SUMMARY

- `CALL … USING` — run subprogram
- BY REFERENCE / CONTENT / VALUE
- static vs dynamic
- `CANCEL` — reset state

Sheet 55

LINKAGE SECTION & Parameters

LINKAGE SECTION · PROCEDURE DIVISION USING · RETURNING · GOBACK · matching caller & callee layouts · ENTRY points

RECEIVING PARAMETERS

The **LINKAGE SECTION** describes data owned by someone else: the caller's parameters or the JCL PARM. `PROCEDURE DIVISION USING` lists the items in the same order the caller passes them. Names may differ; only position and layout must match.

KEY POINTS

- LINKAGE items have no storage until a call maps them
- Parameters are matched by position, not name
- A main program can receive the JCL `PARM` as a length + text
- `RETURNING` gives function-style results (2002+)

SUBPROGRAM AND JCL PARM

```
      *> Subprogram receiving two parameters
       IDENTIFICATION DIVISION.
       PROGRAM-ID. VALIDATE.
       DATA DIVISION.
       LINKAGE SECTION.
       01  LK-RECORD          PIC X(200).
       01  LK-RESULT.
           05 LK-VALID        PIC X.
              88 LK-IS-VALID          VALUE "Y".
           05 LK-REASON       PIC X(30).
       PROCEDURE DIVISION USING LK-RECORD LK-RESULT.
           SET LK-IS-VALID TO TRUE
           IF LK-RECORD(1:10) NOT NUMERIC
              MOVE "N" TO LK-VALID
              MOVE "BAD ACCOUNT" TO LK-REASON
           END-IF
           GOBACK.

      *> Main program receiving PARM='2026-11'
       LINKAGE SECTION.
       01  LK-PARM.
           05 LK-PARM-LEN     PIC S9(4) COMP.
           05 LK-PARM-TEXT    PIC X(100).
       PROCEDURE DIVISION USING LK-PARM.
           IF LK-PARM-LEN > 0
              DISPLAY "RUN PERIOD " LK-PARM-TEXT(1:LK-PARM-LEN)
           END-IF
```

CODE EXPLAINED

→ The caller passes `CALL "VALIDATE" USING IN-REC WS-RESULT`

→ VALIDATE writes LK-RESULT, which is the caller's WS-RESULT

→ z/OS passes the PARM as a halfword length followed by the text

→ Only use the first LK-PARM-LEN bytes: the rest is not yours

MODERN EQUIVALENT

↔ **Python** `def validate(record, result):` positional parameters; `sys.argv` for run args

↔ **Java** `static void validate(String rec, Result r)`; `main(String[] args)`

↔ **JavaScript** `function validate(rec, result)`; `process.argv` for run args

PREFER / AVOID

⇄ reading past the PARM length → reference-modify with LK-PARM-LEN

⇄ LINKAGE layouts typed by hand → shared copybook

RULES

RULE	WHY
`Same order as caller`	Matched by position
`Same lengths / types`	Bytes are shared
`01 or 77 levels`	Required for USING items
`No VALUE clauses`	No storage of its own
`Do not touch unpassed items`	No address = abend (SOC4)

WAYS IN

SOURCE	ARRIVES AS
`CALL USING`	Caller's fields
`JCL PARM=`	Length + text
`CICS`	DFHCOMMAREA
`SET ADDRESS OF`	Any pointer target

⚠ COMMON MISTAKES

- Different parameter order in caller and callee
- Reading the full 100-byte PARM area when only 7 bytes were passed
- Giving LINKAGE items VALUE clauses

✓ TIPS & BEST PRACTICES

- Name LINKAGE items with an `LK-` prefix
- Check the PARM length before using it
- Return status in a parameter field rather than RETURN-CODE for subprograms

QUICK SUMMARY

- `LINKAGE` — caller's data
- `USING` — by position
- PARM — length + text
- `GOBACK` — return

Nested Programs & User-Defined Functions

PROGRAMS INSIDE PROGRAMS

A source file can hold **nested programs**, each closed with `END PROGRAM`. Only the parent can call them unless they are `COMMON`. `GLOBAL` data is visible to nested programs, and `EXTERNAL` data is shared by any program that declares it. COBOL 2002 also added user-defined functions with `FUNCTION-ID`.

KEY POINTS

- Nested programs keep helper logic private to one source file
- `GLOBAL` items can be read and changed by nested programs
- `EXTERNAL` items are shared across separately compiled programs
- User-defined functions need a `REPOSITORY` entry in the caller

NESTED PROGRAM AND A FUNCTION

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. MAINPGM.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01  WS-TOTAL     PIC 9(7)V99 GLOBAL VALUE 0.
       PROCEDURE DIVISION.
           CALL "ADD-ITEM" USING BY CONTENT 19.99
           DISPLAY WS-TOTAL                    *> 19.99
           GOBACK.

         IDENTIFICATION DIVISION.
         PROGRAM-ID. ADD-ITEM.
         DATA DIVISION.
         LINKAGE SECTION.
         01  LK-AMT     PIC 9(5)V99.
         PROCEDURE DIVISION USING LK-AMT.
             ADD LK-AMT TO WS-TOTAL          *> GLOBAL item
             GOBACK.
         END PROGRAM ADD-ITEM.
       END PROGRAM MAINPGM.

      *> User-defined function (2002; check compiler support)
       IDENTIFICATION DIVISION.
       FUNCTION-ID. DOUBLE-IT.
       DATA DIVISION.
       LINKAGE SECTION.
       01  LK-IN        PIC 9(5).
       01  LK-OUT       PIC 9(6).
       PROCEDURE DIVISION USING LK-IN RETURNING LK-OUT.
           COMPUTE LK-OUT = LK-IN * 2
           GOBACK.
       END FUNCTION DOUBLE-IT.
```

CODE EXPLAINED

- → ADD-ITEM sits inside MAINPGM and closes with END PROGRAM
- → WS-TOTAL is GLOBAL, so ADD-ITEM can update it directly
- → A caller declares `REPOSITORY. FUNCTION DOUBLE-IT.` then uses `FUNCTION DOUBLE-IT(5)`
- → Indentation of nested programs is style only

MODERN EQUIVALENT

- ↔ **Python** Nested functions (closures) and module globals
- ↔ **Java** Private static methods or nested classes; static fields
- ↔ **JavaScript** Inner functions and module-scope variables

PREFER / AVOID

- ⇄ GLOBAL for everything → pass parameters; keep GLOBAL rare
- ⇄ EXTERNAL shared data → explicit parameters (easier to trace)

VISIBILITY

ITEM	SEEN BY
`Nested program`	Its parent only
`COMMON program`	Siblings too
`GLOBAL data`	Nested programs
`EXTERNAL data`	Any program declaring it
`FUNCTION-ID`	Programs with REPOSITORY entry

⚠ COMMON MISTAKES

- Missing END PROGRAM, so the compiler treats the rest as part of one program
- Calling a nested program from outside its parent (not found)
- Overusing GLOBAL until nobody knows who changes what

✓ TIPS & BEST PRACTICES

- Use nested programs for helpers that no other program needs
- Check Enterprise COBOL and GnuCOBOL support before using FUNCTION-ID
- Prefer separately compiled subprograms for shared logic

QUICK SUMMARY

- nested program — private helper
- `END PROGRAM` — required close
- `GLOBAL` / `EXTERNAL` — shared data
- `FUNCTION-ID` — your own functions

Embedded SQL with DB2

EXEC SQL … END-EXEC · host variables :WS-NAME · SQLCA / SQLCODE · DECLARE CURSOR / OPEN / FETCH / CLOSE · DCLGEN · precompile & BIND

TALKING TO DB2 FROM COBOL

SQL statements sit inside EXEC SQL … END-EXEC blocks and use COBOL fields as **host variables** prefixed with a colon. The DB2 precompiler (or coprocessor) turns them into calls, and after each statement **SQLCODE** reports the result: 0 success, 100 not found, negative an error.

KEY POINTS

- `INCLUDE SQLCA` defines SQLCODE and related fields
- DCLGEN generates host-variable layouts from a table
- Multi-row results need a **cursor**
- Programs are bound into a package/plan before they run

UNDER THE HOOD

- The SQL coprocessor (`SQL` compiler option) replaces the separate precompile step
- BIND stores the access path; the plan/package name is used at run time
- Batch DB2 programs run under IKJEFT01 `DSN RUN` (see the JCL set)

SINGLE ROW AND CURSOR

```
           EXEC SQL INCLUDE SQLCA END-EXEC.
           EXEC SQL INCLUDE DCLCUST END-EXEC.      *> DCLGEN output
       01  WS-STATE            PIC XX VALUE "IA".

      *>   Single-row SELECT
           MOVE 104217 TO CUST-ID
           EXEC SQL
              SELECT NAME, BALANCE
                INTO :CUST-NAME, :CUST-BALANCE
                FROM CUSTOMER
               WHERE ID = :CUST-ID
           END-EXEC
           EVALUATE SQLCODE
              WHEN 0      DISPLAY CUST-NAME
              WHEN 100    DISPLAY "NOT FOUND"
              WHEN OTHER DISPLAY "SQL ERROR " SQLCODE
           END-EVALUATE

      *>   Cursor for many rows
           EXEC SQL DECLARE C1 CURSOR FOR
              SELECT ID, NAME FROM CUSTOMER
               WHERE STATE = :WS-STATE ORDER BY NAME
           END-EXEC
           EXEC SQL OPEN C1 END-EXEC
           PERFORM UNTIL SQLCODE NOT = 0
              EXEC SQL FETCH C1 INTO :CUST-ID, :CUST-NAME
              END-EXEC
              IF SQLCODE = 0 PERFORM 2000-PRINT END-IF
           END-PERFORM
           EXEC SQL CLOSE C1 END-EXEC
```

CODE EXPLAINED

- → `:CUST-ID` is a COBOL field used inside SQL
- → SQLCODE 100 means no row (or no more rows for FETCH)
- → The cursor loop stops at 100 or any error
- → A SELECT INTO that finds two rows returns -811

MODERN EQUIVALENT

- ↔ **Python** `cur.execute("SELECT … WHERE id = ?", (cid,)); cur.fetchone()`
- ↔ **Java** `PreparedStatement ps = conn.prepareStatement("… WHERE id = ?")`
- ↔ **JavaScript** `await client.query("… WHERE id = $1", [cid])` (node-postgres)

PREFER / AVOID

- ⇄ ignoring negative SQLCODEs → check SQLCODE after every statement
- ⇄ `SELECT *` → list columns to match host variables

COMMON SQLCODES

CODE	MEANING
`0`	Success
`+100`	Not found / end of cursor
`-180 / -181`	Bad date/time value
`-305`	NULL without indicator
`-803`	Duplicate key on insert
`-811`	SELECT INTO returned >1 row
`-911 / -913`	Deadlock / timeout
`-922`	Not authorized

BUILD STEPS

STEP	DOES
`DCLGEN`	Table → copybook
`Precompile / coprocessor`	SQL → calls + DBRM
`Compile + link`	Load module
`BIND`	DBRM → package / plan
`Run`	IKJEFT01 DSN RUN

⚠ COMMON MISTAKES

- Nullable columns fetched without an indicator variable (-305)
- SELECT INTO a query that can return several rows (-811)
- Treating +100 as an error in a FETCH loop

✓ TIPS & BEST PRACTICES

- Use DCLGEN layouts so host variables match column types
- Add indicator variables (`:NAME :NAME-IND`) for nullable columns
- COMMIT at regular intervals in long batch updates

QUICK SUMMARY

- `EXEC SQL … END-EXEC`
- `:host-var` — COBOL field in SQL
- `SQLCODE` — 0 / 100 / negative
- cursor — many rows

CICS Basics

ONLINE TRANSACTIONS ON THE MAINFRAME

CICS runs online transactions: a user presses Enter on a 3270 screen, CICS starts a COBOL program, and the program sends a screen back. Programs are **pseudo-conversational**: they end after each screen and pass state in a **COMMAREA**, so no memory is held while the user types.

KEY POINTS

- `EXEC CICS` commands replace file I/O and DISPLAY/ACCEPT
- BMS maps define screens; their copybooks give field layouts
- `EIBCALEN = 0` means first time in (no COMMAREA yet)
- `RESP` captures the result instead of abending

PSEUDO-CONVERSATIONAL SKELETON

```
       WORKING-STORAGE SECTION.
           COPY ACCTMAP.                      *> BMS map layout
       01  WS-RESP              PIC S9(8) COMP.
       01  WS-COMM.
           05 WS-STEP           PIC X.
       LINKAGE SECTION.
       01  DFHCOMMAREA          PIC X(1).
       PROCEDURE DIVISION.
           IF EIBCALEN = 0                    *> first time
              MOVE LOW-VALUES TO ACCTMAPO
              EXEC CICS SEND MAP("ACCTMAP") MAPSET("ACCTSET")
                   ERASE
              END-EXEC
           ELSE
              EXEC CICS RECEIVE MAP("ACCTMAP") MAPSET("ACCTSET")
                   RESP(WS-RESP)
              END-EXEC
              EVALUATE EIBAID
                 WHEN DFHPF3   EXEC CICS RETURN END-EXEC
                 WHEN DFHENTER PERFORM 2000-LOOKUP-ACCOUNT
              END-EVALUATE
           END-IF
           EXEC CICS RETURN TRANSID("ACCT")
                COMMAREA(WS-COMM) LENGTH(1)
           END-EXEC.
```

CODE EXPLAINED

- → First entry sends an empty screen
- → On the next Enter, CICS restarts the program with the COMMAREA
- → EIBAID tells which key the user pressed (PF3 = exit)
- → `RETURN TRANSID` ends the task but keeps the conversation going

MODERN EQUIVALENT

- ↔ **Python** A web request handler: stateless, with state in a session or form field
- ↔ **Java** A servlet or Spring controller with session state
- ↔ **JavaScript** An Express route handler with state in cookies or the client

PREFER / AVOID

- ⇄ conversational programs that wait for input → pseudo-conversational RETURN TRANSID
- ⇄ unchecked commands → `RESP` + `EVALUATE` instead of HANDLE CONDITION

COMMON COMMANDS

COMMAND	DOES
`SEND MAP / RECEIVE MAP`	Screen I/O
`READ / WRITE / REWRITE FILE`	VSAM file I/O
`LINK / XCTL`	Call / transfer to program
`RETURN [TRANSID]`	End task
`SYNCPOINT`	Commit
`ASKTIME / FORMATTIME`	Date and time

EIB FIELDS

FIELD	HOLDS
`EIBCALEN`	COMMAREA length
`EIBAID`	Key pressed
`EIBTRNID`	Transaction ID
`EIBRESP`	Last response
`EIBDATE / EIBTIME`	Task start

⚠ COMMON MISTAKES

- Using DISPLAY, ACCEPT or COBOL file I/O inside CICS programs
- Not checking EIBCALEN, so first-time logic runs every time
- Holding a terminal conversation open while the user types

✓ TIPS & BEST PRACTICES

- Check `RESP` after every command and EVALUATE the result
- Keep the COMMAREA small; use channels/containers for large state
- Test maps with the CICS-supplied CEDF debugging transaction

QUICK SUMMARY

- `EXEC CICS` — CICS commands
- BMS map — screen layout
- pseudo-conversational — end per screen
- COMMAREA — state between screens

Compiler Options & Listings

OPTIONS THAT CHANGE BEHAVIOR

Enterprise COBOL options control both performance and **results**: TRUNC decides whether binary fields are truncated to their PIC digits, NUMPROC how signs are handled, and SSRANGE whether subscripts are checked. The **compile listing** shows options, messages, cross-references and the generated code.

KEY POINTS

- Options come from shop defaults, the JCL PARM, or `CBL` / `PROCESS` in source
- `CBL` statements must come before IDENTIFICATION DIVISION
- Message severities: I (info), W (4), E (8), S (12), U (16)
- Changing TRUNC or NUMPROC can change program output

UNDER THE HOOD

- `TRUNC(STD)` truncates COMP to PIC digits; `TRUNC(BIN)` keeps the full binary value but is slower
- `OPTIMIZE(2)` can reorder code, making some debug output look out of order
- `LIST` shows the machine code for each statement: useful for S0C7 offsets

OPTIONS IN SOURCE AND JCL

```
       CBL SSRANGE,TRUNC(STD),ARITH(EXTEND),OPTIMIZE(0)
       CBL LIST,MAP,XREF,OFFSET
       IDENTIFICATION DIVISION.
       PROGRAM-ID. PAYCALC.

      *> JCL PARM on the compile step:
      *> //COBOL EXEC PGM=IGYCRCTL,
      *> //  PARM='SSRANGE,XREF,MAP,LIST,RENT'

      *> GnuCOBOL equivalents:
      *> cobc -x -debug           (runtime checks like SSRANGE)
      *> cobc -x -fbinary-truncate=no   (like TRUNC(BIN))
      *> cobc -x -t listing.lst   (listing file)
```

CODE EXPLAINED

- → Two CBL lines set options for this compile only
- → SSRANGE catches bad subscripts at run time
- → MAP and XREF list every field's offset and every reference
- → OPTIMIZE(0) keeps code in source order while debugging

MODERN EQUIVALENT

- ↔ **Python** Interpreter flags: `python -O`, `-W error`
- ↔ **Java** `javac -Xlint:all -g`; JVM flags like `-ea`
- ↔ **JavaScript** `tsc --strict`, `node --trace-warnings`

PREFER / AVOID

- ⇄ changing TRUNC/NUMPROC per program → keep shop-wide defaults
- ⇄ testing without SSRANGE → SSRANGE in test, measured in prod

KEY OPTIONS

OPTION	EFFECT
SSRANGE	Check subscripts / ref-mod
TRUNC(STD|OPT|BIN)	Binary truncation rules
NUMPROC(NOPFD|PFD)	Sign handling / speed
ARITH(COMPAT|EXTEND)	18 vs 31 digits
OPTIMIZE(0|1|2)	Optimization level
RENT	Reentrant code (CICS)
DYNAM	Literal CALLs are dynamic
SQL / CICS	Coprocessors

LISTING SECTIONS

SECTION	SHOWS
Options	Options in effect
Source	Numbered lines, copybooks
MAP	Field offsets, lengths
XREF	Where names are used
LIST / OFFSET	Code offsets per statement
Messages	Diagnostics by severity

⚠ COMMON MISTAKES

- Putting CBL after IDENTIFICATION DIVISION (it is ignored or an error)
- Ignoring W-level messages about truncation
- Changing TRUNC and getting different results from binary fields

✓ TIPS & BEST PRACTICES

- Keep LIST/OFFSET listings for production modules to map abend offsets
- Read the options page of the listing when results differ between environments
- Treat compile RC 4 warnings as work to fix

QUICK SUMMARY

- `CBL` / `PARM` — set options
- `SSRANGE` — bounds checks
- `TRUNC` / `NUMPROC` — change results
- listing — MAP, XREF, offsets

Debugging & Abends

S0C7 data exception · S0C4 protection exception · S0CB divide by zero · S806 module not found · CEEDUMP · `-ftrace` / `-debug` · z/OS Debugger

WHEN A PROGRAM ABENDS

An **abend** (abnormal end) stops a program with a code. Most COBOL abends come from bad data or bad addressing: **S0C7** means arithmetic on invalid numeric data, **S0C4** means touching storage the program does not own. The Language Environment **CEEDUMP** shows the failing statement and field values.

KEY POINTS

- System abends start with S, user abends with U
- CEEDUMP shows the offset; the LIST listing maps it to a line
- Most S0C7s come from spaces in numeric fields
- GnuCOBOL reports the source line directly

UNDER THE HOOD

- S0C7 = the hardware decimal instruction (AP, ZAP, CP…) saw a non-packed byte
- S0C4 often means a LINKAGE item was used without being passed
- Find the line: CEEDUMP offset → LIST/OFFSET listing → statement

CAUSES AND CHECKS

```
      *>   S0C7: spaces in a numeric field
       01  WS-AMT         PIC S9(5)V99 COMP-3.
           MOVE SPACES TO WS-REC          *> WS-AMT now X'4040…'
           ADD 1 TO WS-AMT                *> S0C7

      *>   Fix: validate or initialize first
           IF WS-AMT IS NUMERIC
              ADD 1 TO WS-AMT
           ELSE
              DISPLAY "BAD AMT AT REC " WS-REC-COUNT
           END-IF

      *>   S0CB: divide by zero
           DIVIDE WS-TOTAL BY WS-COUNT GIVING WS-AVG
              ON SIZE ERROR MOVE 0 TO WS-AVG
           END-DIVIDE

      *>   Debug output on GnuCOBOL
      *>   cobc -x -debug -ftrace prog.cbl
      *>   COB_SET_TRACE=Y ./prog     (paragraph trace)
      *>   prog.cbl:57: libcob: error: subscript of 'TBL' out of bounds
```

CODE EXPLAINED

- → Moving SPACES to a group puts X'40' (EBCDIC space) in the packed field
- → IS NUMERIC catches bad data before the arithmetic
- → ON SIZE ERROR prevents the divide-by-zero abend
- → GnuCOBOL `-debug` gives file and line in the error

MODERN EQUIVALENT

- ↔ **Python** `decimal.InvalidOperation` / `ValueError` with a traceback
- ↔ **Java** `NumberFormatException` / `NullPointerException` with a stack trace
- ↔ **JavaScript** `NaN` silently, or a `TypeError` with a stack trace

PREFER / AVOID

- ⇄ DISPLAY debugging everywhere → CEEDUMP + listing, or z/OS Debugger
- ⇄ retrying abended jobs blindly → find the bad record first

COMMON ABENDS

CODE	USUAL CAUSE
S0C7	Bad numeric (packed/zoned) data
S0C4	Bad address, unpassed LINKAGE
S0CB	Divide by zero
S806	Load module not found
S322	CPU time limit exceeded
S013	Dataset open error (DCB, member)
SB37 / SD37 / SE37	Out of space
U4038	LE detected severe error

DEBUG TOOLS

TOOL	USE
CEEDUMP	LE dump: traceback, fields
LIST/OFFSET listing	Offset → statement
z/OS Debugger	Breakpoints, step
CEDF (CICS)	Step CICS commands
-debug / -ftrace	GnuCOBOL checks, trace
DISPLAY	Quick checks

⚠ COMMON MISTAKES

- Reading the abend code only and not the CEEDUMP traceback
- Fixing the abend by skipping the record instead of fixing the bad data source
- Initializing records with MOVE SPACES when they contain packed fields

✓ TIPS & BEST PRACTICES

- Keep the compile listing for every production module version
- Log the record number and key before risky arithmetic
- Use INITIALIZE instead of MOVE SPACES on mixed records

QUICK SUMMARY

- S0C7 — bad numeric data
- S0C4 — bad address
- S806 — module not found
- CEEDUMP + listing — find the line

JSON, XML & Interop

COBOL MEETS MODERN FORMATS

Enterprise COBOL 6 can turn a group item straight into **JSON** or **XML** and parse documents back into a group, which is how mainframe programs feed REST APIs. COBOL can also call C functions and Java methods; Object-Oriented COBOL exists but is rarely used in shops.

KEY POINTS

- `JSON GENERATE` uses data names as keys (hyphens kept)
- `NAME OF` renames keys; `SUPPRESS` drops fields
- `JSON PARSE` matches keys to field names
- GnuCOBOL support for JSON/XML statements is limited: check your version

JSON ROUND TRIP

```
       01  CUSTOMER.
           05 CUST-ID         PIC 9(6)     VALUE 104217.
           05 CUST-NAME       PIC X(20)    VALUE "ADA LOVELACE".
           05 CUST-BAL        PIC S9(5)V99 VALUE 1250.50.
           05 CUST-INTERNAL   PIC X(10).
       01  JSON-DOC           PIC X(500).
       01  JSON-LEN           PIC 9(4) COMP.

           JSON GENERATE JSON-DOC FROM CUSTOMER
                COUNT IN JSON-LEN
                NAME OF CUST-ID IS "id", CUST-NAME IS "name",
                        CUST-BAL IS "balance"
                SUPPRESS CUST-INTERNAL
                ON EXCEPTION DISPLAY "JSON ERROR " JSON-CODE
           END-JSON
      *>   {"customer":{"id":104217,"name":"ADA LOVELACE",
      *>    "balance":1250.50}}

           JSON PARSE JSON-DOC(1:JSON-LEN) INTO CUSTOMER
                NAME OF CUST-ID IS "id", CUST-NAME IS "name",
                        CUST-BAL IS "balance"
           END-JSON

      *>   Calling C:  CALL "strlen" USING BY REFERENCE WS-CSTR
      *>               RETURNING WS-LEN
```

CODE EXPLAINED

- → The group name becomes the outer object key
- → Trailing spaces in CUST-NAME are trimmed in the output
- → COUNT IN gives the generated length
- → `JSON-CODE` holds the error code when generation fails

MODERN EQUIVALENT

- ↔ **Python** `json.dumps(asdict(cust))` / `json.loads(doc)`
- ↔ **Java** Jackson: `mapper.writeValueAsString(cust)` / `readValue`
- ↔ **JavaScript** `JSON.stringify(cust)` / `JSON.parse(doc)`

PREFER / AVOID

- ⇄ hand-built JSON with STRING → `JSON GENERATE`
- ⇄ parsing JSON with UNSTRING → `JSON PARSE`

STATEMENTS

STATEMENT	DOES
`JSON GENERATE`	Group → JSON text
`JSON PARSE`	JSON text → group
`XML GENERATE`	Group → XML
`XML PARSE`	Event-based XML parser
`NAME OF … IS`	Rename keys
`SUPPRESS`	Omit fields
`CONVERTING (6.3+)`	Booleans etc.

INTEROP

TARGET	HOW
`C`	CALL with BY VALUE / BY REFERENCE
`Java`	OO COBOL INVOKE, or JNI wrappers
`REST APIs`	z/OS Connect + JSON
`MQ`	CALL MQPUT / MQGET

⚠ COMMON MISTAKES

- Receiving field too small for the generated document (exception)
- Expecting keys like `customerName` without NAME OF (you get CUST-NAME)
- Assuming GnuCOBOL handles JSON statements like Enterprise COBOL

✓ TIPS & BEST PRACTICES

- Add NAME OF clauses so APIs get camelCase keys
- Size the output field generously and check COUNT IN
- Use z/OS Connect to expose COBOL programs as REST APIs

QUICK SUMMARY

- `JSON GENERATE` — group → JSON
- `JSON PARSE` — JSON → group
- `NAME OF` / `SUPPRESS` — shape output
- `CALL` — C interop

Testing & DevOps for COBOL

MODERN PRACTICES FOR OLD CODE

COBOL can be unit tested, kept in Git and built in CI like any other code. **COBOL Check** (open source) and **IBM zUnit** test individual paragraphs, mocking files and calls. **Zowe CLI** and **IBM DBB** let pipelines upload source, compile on z/OS and run tests from GitHub Actions or Jenkins.

KEY POINTS

- COBOL Check inserts test code into a copy of the program
- Mock file I/O and CALLs so tests do not need datasets
- DBB builds only programs affected by a change (incl. copybooks)
- Git branches replace "copy the PDS member" workflows

A COBOL CHECK TEST SUITE

```
*> File: src/test/cobol/PAYCALC/PayCalc.cut
      TestSuite "Overtime calculation"

      TestCase "40 hours or less has no overtime"
      MOVE 38 TO WS-HOURS
      MOVE 20.00 TO WS-RATE
      PERFORM 2000-CALC-PAY
      Expect WS-OVERTIME-PAY TO BE 0
      Expect WS-GROSS-PAY TO BE 760.00

      TestCase "Hours over 40 paid at 1.5x"
      MOVE 45 TO WS-HOURS
      MOVE 20.00 TO WS-RATE
      PERFORM 2000-CALC-PAY
      Expect WS-OVERTIME-PAY TO BE 150.00

      TestCase "Read error is reported"
      MOCK READ EMP-FILE
         MOVE "30" TO WS-EMP-FS
      END-MOCK
      PERFORM 8000-READ-EMP
      Expect WS-ERROR-FLAG TO BE "Y"
```

CODE EXPLAINED

→ Each TestCase sets fields, PERFORMs a paragraph, then checks results

→ MOCK replaces a READ so no dataset is needed

→ 5 overtime hours × 20.00 × 1.5 = 150.00

→ Run with the cobolcheck script; it compiles with GnuCOBOL or on z/OS

MODERN EQUIVALENT

↔ **Python** pytest: `def test_overtime(): assert calc_pay(45, 20) == …`

↔ **Java** JUnit: `@Test void overtime() { assertEquals(…) }`

↔ **JavaScript** Vitest/Jest: `test("overtime", () => expect(…).toBe(…))`

PREFER / AVOID

⇄ manual testing in a shared test region → unit tests in Git + CI

⇄ whole-library rebuilds → DBB impact builds

TOOLS

TOOL	ROLE
COBOL Check	Open-source unit tests
IBM zUnit	Unit tests in IBM tooling
Zowe CLI	Upload, submit, download from CI
IBM DBB	Dependency-aware builds
Git + GitHub/GitLab	Source control, reviews
SonarQube COBOL	Static analysis

CI PIPELINE

STAGE	EXAMPLE
Checkout	Git
Unit test	COBOL Check on GnuCOBOL
Upload	zowe files upload dir-to-pds
Build	DBB or JCL compile job
Integration test	Submit test JCL, check RC
Deploy	Promote load modules

⚠ COMMON MISTAKES

- Paragraphs that do I/O and logic together, which are hard to test
- Tests that depend on shared test datasets other people change
- Keeping source only in PDS libraries with no history

✓ TIPS & BEST PRACTICES

- Split calculation paragraphs from I/O paragraphs
- Start tests on the most-changed business rules
- Run COBOL Check on GnuCOBOL in CI before touching z/OS

QUICK SUMMARY

- COBOL Check / zUnit — unit tests
- MOCK — fake I/O and CALLs
- Zowe + DBB — CI on z/OS
- Git — history and reviews

Sheet 63

Quick Reference — Verbs & Syntax

divisions & sections · data verbs · arithmetic verbs · control flow · file I/O verbs · string verbs

STRUCTURE (04, 09)

ITEM	HOLDS
IDENTIFICATION DIVISION	PROGRAM-ID
ENVIRONMENT DIVISION	FILE-CONTROL
DATA DIVISION	All data
FILE SECTION	FD / SD records
WORKING-STORAGE	Persistent vars
LOCAL-STORAGE	Per-call vars
LINKAGE SECTION	Parameters
PROCEDURE DIVISION	Paragraphs

DATA VERBS (17–20)

VERB	DOES
MOVE a TO b	Copy / convert
MOVE CORR g1 TO g2	Matching names
INITIALIZE g	Zeros + spaces
SET cond TO TRUE	Set 88
SET idx TO / UP BY	Index
ACCEPT x [FROM …]	Input / date
DISPLAY a b	Output line

ARITHMETIC (25–28)

VERB	RESULT
ADD a TO b	b += a
ADD a b GIVING c	c = a + b
SUBTRACT a FROM b	b −= a
MULTIPLY a BY b	b *= a
DIVIDE a INTO b	b /= a
DIVIDE a BY b GIVING c REMAINDER r	c, r
COMPUTE x ROUNDED = …	Formula
ON SIZE ERROR	Overflow / ÷0

CONTROL FLOW (29–34)

STATEMENT	LIKE
IF … ELSE … END-IF	if / else
EVALUATE TRUE / WHEN	switch
PERFORM para	call
PERFORM … UNTIL c	while (!c)
WITH TEST AFTER	do-while
PERFORM VARYING i FROM 1 BY 1 UNTIL i > n	for
EXIT PERFORM [CYCLE]	break [continue]
EXIT PARAGRAPH	return
GOBACK / STOP RUN	End
CONTINUE	No-op

FILE I/O (40–47)

STATEMENT	DOES
OPEN INPUT/OUTPUT/EXTEND/I-O	Open
READ f [INTO w] AT END	Next record
READ f KEY IS k INVALID KEY	By key
START f KEY >= k	Position
READ f NEXT	Browse
WRITE r [FROM w]	Output
REWRITE r / DELETE f	Update
CLOSE f	Close
SORT / MERGE	Order files
RELEASE / RETURN	Sort procs

STRINGS & TABLES (21–24, 35–38)

STATEMENT	DOES
STRING … DELIMITED BY … INTO	Join
UNSTRING … DELIMITED BY … INTO	Split
INSPECT … TALLYING	Count
INSPECT … REPLACING	Replace
INSPECT … CONVERTING	Translate
x(start:len)	Substring
OCCURS n [INDEXED BY i]	Table
SEARCH / SEARCH ALL	Lookup
FUNCTION TRIM / UPPER-CASE	Functions

PROGRAMS (53–58)

STATEMENT	DOES
COPY m [REPLACING …]	Include source
CALL p USING a b	Call subprogram
BY REFERENCE / CONTENT / VALUE	Passing
PROCEDURE DIVISION USING	Receive
CANCEL p	Reset sub
EXEC SQL … END-EXEC	DB2
EXEC CICS … END-EXEC	CICS

CODING FORMAT (05)

COLUMNS	USE
1–6	Sequence
7	* / - D indicator
8–11	Area A: names, 01, FD
12–72	Area B: statements
73–80	Ignored

Sheet 64

Quick Reference — PIC, USAGE & File Status

PICTURE symbols · edit symbols · USAGE sizes in bytes · FILE STATUS codes · common abends · figurative constants

PICTURE (11)

SYMBOL	MEANING
9	Digit
X	Any character
A	Letter / space
S	Sign
V	Implied point
P	Scaling zero
(n)	Repeat

EDITING (12, 19)

SYMBOL	EFFECT
Z	Zero → space
*	Zero → *
$ $$$	Currency / floating
, .	Comma, point
+ - CR DB	Sign display
B 0 /	Insert

USAGE (13)

USAGE	BYTES FOR S9(N)
DISPLAY	n
COMP-3	(n + 1) / 2 rounded up
COMP 1–4 digits	2
COMP 5–9 digits	4
COMP 10–18 digits	8
COMP-1 / COMP-2	4 / 8

FILE STATUS (43)

CODE	MEANING
00	OK
02	OK, dup alternate key
04	Length mismatch
10	End of file
21	Sequence error
22	Duplicate key
23	Not found
30	Permanent I/O error
35	File not found
37	Bad open mode
39	Attribute mismatch
41 / 42	Already open / not open
46	Read past EOF

ABENDS (60)

CODE	CAUSE
S0C7	Bad numeric data
S0C4	Bad address
S0CB	Divide by zero
S806	Module not found
S013	Open / DCB error
S322	CPU time
SB37 / SD37 / SE37	Out of space
U4038	LE severe error

SQLCODE (57)

CODE	MEANING
0	OK
+100	Not found / end
-305	NULL, no indicator
-803	Duplicate key
-811	More than one row
-911	Deadlock
-922	Not authorized

FIGURATIVE CONSTANTS (14)

CONSTANT	VALUE
ZERO(S)	0
SPACE(S)	Blanks
HIGH-VALUES	X'FF'
LOW-VALUES	X'00'
QUOTE(S)	"
ALL "x"	Repeat

LEVEL NUMBERS (10, 15, 16)

LEVEL	USE
01	Record
02–49	Children
66	RENAMES
77	Standalone
88	Condition name

RETURN CODES (34)

RC	MEANING
0	Success
4	Warning
8	Error
12	Severe
16	Fatal